VERSION
14.0

AMS GRA ARCHITECTURE VOLUMES

Radio Frequency (RF) Multi-Function Aperture (MFA) Encapsulation Layer (MEL) Interface Description Document (IDD)

AFLCMC/WA

April 24, 2026

U.S. GOVERNMENT NOTICES: Furnished to the U.S. Government with UNLIMITED RIGHTS as defined in DFARS 252.227-7013, this takes precedence over any restrictive marking herein.

Table of Contents

1 Common RF MEL **1**
1.1 Introduction . . . 1
1.2 RF MEL Context . . . 1
1.3 RF MEL Functional Decomposition . . . 2
1.4 Implementation Compliance . . . 4
1.5 Deprecated Function Compliance . . . 7

2 Application Programming Interface (API) Overview **8**
2.1 RF MEL Domain Library . . . 8
2.1.1 Admin MEL Domain Library . . . 8
2.1.2 Command and Control (C2) MEL Domain Library . . . 9
2.1.3 COSITE MEL Domain Library . . . 9
2.1.4 Data MEL Domain Library . . . 10
2.1.5 Direct Endpoint Access (DEA) MEL Domain Library . . . 10
2.1.6 Monitor MEL Domain Library . . . 10
2.2 RF MEL Class . . . 10
2.3 RF MFA Info Class . . . 10
2.4 Virtual Aperture (VA) Class . . . 11
2.5 Virtual Aperture Database (VADB) . . . 16
2.6 Job Classes . . . 17
2.7 Endpoint Classes . . . 18

3 Standards and Protocols **20**
3.1 Jobs-based Protocol . . . 20
3.1.1 Jobs-based Frame Structure . . . 20
3.1.2 Jobs-based Packet Types . . . 21
3.2 VITA Messaging . . . 41
3.2.1 VITA 49.1 Frame Structure . . . 41
3.2.2 VITA 49.2 Packet Types . . . 41
3.2.3 Customized VITA 49 Standards . . . 43
3.2.4 Constraining Direct Endpoint Access: Allowable Control and Data Packets . . . 44

4 Operations Enabled by the RF MEL **46**
4.1 Load RF MEL . . . 46

4.2 Get MFA Characteristics at Design Time via VADB . . . 48
4.3 Get MFA Characteristics At Run Time . . . 52
4.4 Request VA . . . 55
4.5 Create Data Transfer Endpoint . . . 58
4.6 Associate Endpoints . . . 62
4.7 Submit Job Request . . . 64
4.8 Add Job Interval . . . 67
4.9 Direct Endpoint Access (DEA) . . . 70
4.9.1 Overview . . . 71
4.9.2 DEA Initialization . . . 73
4.9.3 Service-based Skill Execution . . . 77
4.9.4 Coprocessor Based Skill Execution . . . 80
4.9.5 Concluding DEA . . . 84
4.10 Get Health and Status . . . 87
4.11 Local Function Control . . . 94
4.12 Inhibit VA . . . 97
4.13 Request RF Activity Notifications . . . 100

5 Advanced Features Use Cases 103
5.1 Multi-Beam Use Case . . . 103
5.2 Multi-Channel Use Case . . . 106
5.3 Multi-Polarization Use Case . . . 109
5.4 Round Robin Use Case . . . 112
5.5 Adaptive Pulse Repetition Frequency . . . 116

A Acronyms 119

B Revision History 122

List of Figures

1.2.1 RF MEL Context . . . 2
1.3.1 RF MEL Enabled Functions . . . 3

2.4.1 Example VAs and Element Groups . . . 12
2.4.2 Example VA Instance . . . 13
2.4.3 Multi-Face VA Designs . . . 14
2.4.4 Multi-Face VAI Designs . . . 15
2.6.1 Relationship of Job Details and Job Intervals . . . 18

3.1.1 Frame Structure Diagram . . . 21
3.1.2 Jobs Based Packet Types Diagram . . . 22
3.1.3 Protocol Metadata . . . 23
3.1.4 MEL Protocol Version ID . . . 24
3.1.5 Interval Packet Diagram . . . 25
3.1.6 Data Packet Format . . . 30
3.1.7 Item Packing Diagram . . . 31
3.1.8 Event Packet Diagram . . . 33
3.1.9 AGC, Tx/Rx Flags and Repetitions Diagram . . . 36
3.1.10 Phase Offset Diagram . . . 38
3.2.1 Framed VITA 49.2 Packet Stream . . . 43

4.1.1 Load RF MEL . . . 48
4.2.1 Communicate MFA Characteristics at Design Time . . . 51
4.3.1 Communicate MFA Characteristics at Run Time . . . 54
4.4.1 Get Virtual Aperture . . . 57
4.5.1 Create Data Transfer Endpoint . . . 61
4.6.1 Associate Endpoints . . . 64
4.7.1 Submit Job Request . . . 67
4.9.1 Direct Endpoint Access . . . 73
4.9.2 Initialize Direct Endpoint Access . . . 76
4.9.3 Request Virtual Aperture with Direct Endpoint Access . . . 77
4.9.4 Service-based Skill Execution . . . 80
4.9.5 Coprocessor-based Skill Execution . . . 83
4.9.6 Concluding Direct Endpoint Access . . . 86

4.10.1 Health and Status Reporting: SubsystemStatus . . . 88
4.10.2 Health and Status Reporting: AntennaStatus . . . 89
4.10.3 Health and Status Reporting: SecurityAuditRecord . . . 90
4.10.4 Health and Status Reporting: LocalFunction . . . 91
4.10.5 Fault and Status Reporting . . . 93
4.11.1 Local Function Control . . . 96
4.11.2 Local Function Control Scheduling . . . 97
4.12.1 Invoke COSITE Mitigation . . . 100
4.13.1 Report RF Utilization . . . 102

5.1.1 Multi-Beam . . . 105
5.2.1 Multi-Channel . . . 108
5.3.1 Multi-Polarization . . . 111
5.4.1 Round Robin Physical Structure Example . . . 112
5.4.2 Round Robin . . . 115
5.5.1 Adaptive Pulse Repetition Frequency . . . 118

List of Tables

1.4.1 RF MEL API Reference Table . . . 7

3.1.1 MEL Protocol Version ID . . . 24
3.1.2 Job Details ID . . . 25
3.1.3 Tx Power Mode ID . . . 26
3.1.4 VA Definition ID . . . 26
3.1.5 VA Instance ID . . . 26
3.1.6 Job Interval ID . . . 27
3.1.7 Local Function Type ID . . . 27
3.1.8 Local Function Instance ID . . . 27
3.1.9 Capability ID . . . 28
3.1.10 Activity ID . . . 28
3.1.11 Sequence Start Count . . . 29
3.1.12 Sequence Repeat Count . . . 29
3.1.13 Calibration Duration . . . 29
3.1.14 Data Packet Format . . . 30
3.1.15 Data Packet Fields . . . 30
3.1.16 Real/Complex Types . . . 31
3.1.17 Data Item Format . . . 32
3.1.18 Event ID . . . 34
3.1.19 Weights ID . . . 34
3.1.20 Data Pipe ID . . . 34
3.1.21 Rx Element Group ID . . . 35
3.1.22 Duration . . . 35
3.1.23 Time Stamp Of First Sample . . . 35
3.1.24 Event Start Count Count . . . 36
3.1.25 AGC, Tx/Rx Flags and Job Event Repetitions . . . 37
3.1.26 Sample Frequency . . . 37
3.1.27 Requested Frequency . . . 37
3.1.28 True Center Frequency . . . 38
3.1.29 Phase Offset . . . 38
3.1.30 Pointing Vector Index . . . 39
3.1.31 Polarization . . . 39

3.1.32 Number of Samples 39
3.1.33 Payload Size 40
3.1.34 Trailer Size 40
3.1.35 User Defined Data 40

B.0.1 Revision History Table 122

1.0 Common RF MEL

1.1 Introduction

The Multi-Function Aperture (MFA) Encapsulation Layer (MEL) is a critical interface within the Agile Mission Suite (AMS) Government Reference Architecture (GRA) taxonomy, which provides an Application Programming Interface (API) definition to govern the interaction between an MFA and an AMS GRA Service in the Multi-Function Processor (MFP). Two MELs are delivered with the AMS GRA Starter Kit (SK): the Common Infrared (IR) MEL and the Common Radio Frequency (RF) MEL. The MELs achieve the following business and technical objectives of the AMS GRA:

- Provide a common, abstracted interface to a broad set of digital MFA types and designs
- Enable significant advances in service portability through the application of common interfaces for interactions with the MFAs
- Enable rapid insertion of new and updated capabilities through the use of software without the need to modify the MFA
- Abstracts MFA designs from the services, promoting low coupling between the design of the MFA and the implementation of services executing in the MFP

Every AMS GRA compliant IR MFA is delivered with an IR MEL implementation, and every AMS GRA compliant RF MFA is delivered with an RF MEL implementation. For more information on the IR MEL interface, see the IR MEL Interface Description Document (IDD). The MEL implementation is the MFA-specific realization of the Common IR MEL API or Common RF MEL API. The MEL implementation is logically part of the MFA; however, the MEL implementation is physically used by a service at runtime as a statically or dynamically linked library. AMS GRA Services use an MFA's specific MEL implementation to communicate with the MFA. The interface between the MEL implementation and the rest of the MFA is implementation specific and is not prescribed by the AMS GRA.

The Common RF MEL is referred to henceforth as the RF MEL for convenience.

1.2 RF MEL Context

The RF MEL, shown in **Figure 1.2.1**, is an interface used by services to interact with AMS GRA-compliant RF MFAs. The RF MEL interface is specified by the RF MEL API and AMS GRA selected data formats including a Jobs-based format and a tailored set of VME bus International Trade Association (VITA) 49.2 Signal Data, Command, and Context packets. The vendor of each RF MFA provides one or more shared libraries that implement the RF MEL API, which must be loaded by each operating system process of the service that directly interacts

with the RF MFA. Furthermore, when a service consists of multiple processes, or a Skill consists of multiple services acting in concert, the Skills/services are responsible for coordinating their own interactions with a given RF MFA and associated Virtual Apertures (VAs) and endpoints. Communication between the services and the RF MFA, including Command and Control (C2) along with data transfers is performed over the Avionics Backplane Bus (ABB) using RF MFA implementation specific messaging and data transfer mechanisms.

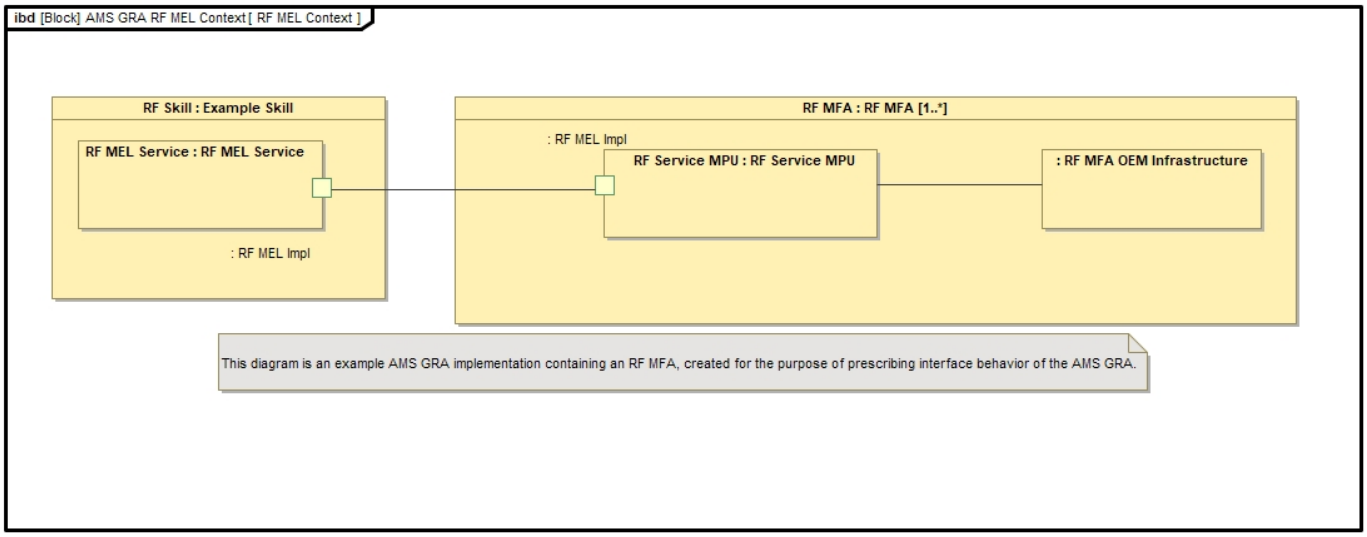


Figure 1.2.1: This Diagram shows an Example AMS GRA Implementation Containing an RF MFA.

1.3 RF MEL Functional Decomposition

The functional decomposition in **Figure 1.3.1** shows the high level operations supported by the RF MEL. These operations elementary and higher level operations, from ones used to create data stream endpoints between an MFA and an MFP, to operations that have the ability to command an MFA to perform multiple sub-tasks simultaneously. These operations were derived from the RF MEL use cases identified by the AMS GRA consortium. See **Chapter 4** for more detailed descriptions.

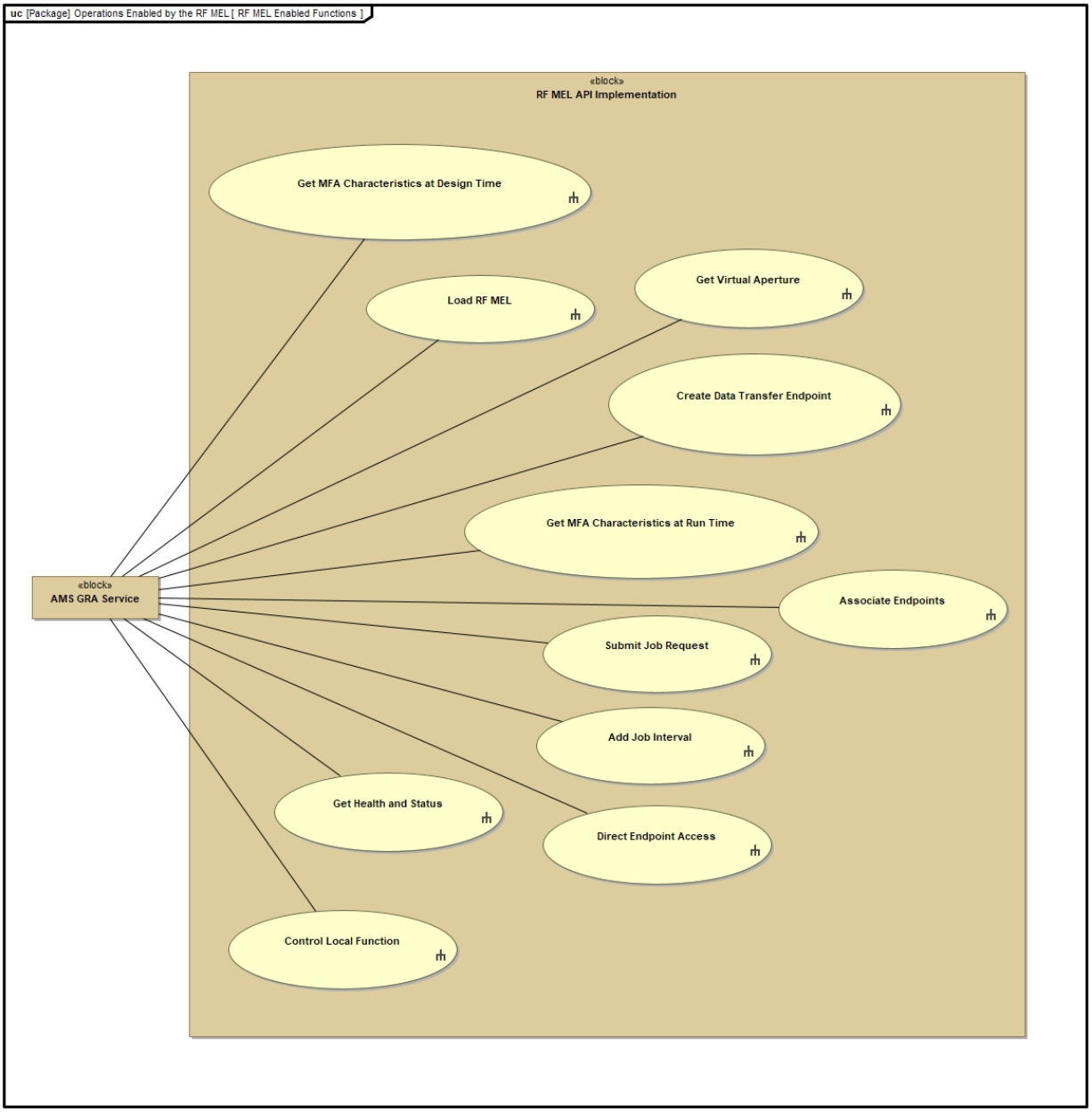


Figure 1.3.1: This Use Case Diagram Displays a High Level Functional Decomposition of the Operations Enabled by the RF MEL.

1.4 Implementation Compliance

The AMS GRA SK contains C++ APIs for Common interfaces. Doxygen is a Hypertext Markup Language (HTML) output delivered with the APIs containing brief descriptions of the Common classes and methods. Within the Doxygen documentation, an adopter will find prescriptive tag markings that indicate when API elements are required, conditionally required, or optional within MEL implementations. Compliant implementations must provide the corresponding content as directed by the tags. Below are descriptions of each Doxygen tag and their optionality:

- `@Required`: This element is required for all API implementations provided by a Minimum Procurable Unit (MPU).
  - When applied to a virtual function, this tag indicates that an implementation of the function must be provided in all RF MEL implementations.
  - When applied to other elements, this tag indicates that the associated data definition is used to support required MEL functions and any MEL implementation must include the element as defined in the Common API definitions.
- `@RequiredIf<capability X>`: If the corresponding capability is supported by a MPU, then the API element must be used in the provided API implementation.
  - When applied to a virtual function, this tag indicates that an implementation of the function must be provided in an RF MEL that supports the corresponding capability.
  - When applied to other elements, this tag indicates that the associated data definition is used to support required MEL functions and any MEL implementation must include the element as defined in the Common API definitions if it supports the corresponding capability.
- `@Optional`: This element is optional for an MPU.
  - When applied to a virtual function, this tag indicates that an implementation of the function may be provided in an RF MEL implementation.
  - When applied to other elements, this tag indicates that the associated data definition may be used to support an RF MEL implementation as defined in the Common API definitions.

Compliant implementations of the RF MEL interface adhere to the definitions (e.g., functions, classes, structures, macros, typedefs, and enumerations) of the Common RF MEL API. In the case of Common RF MEL API functions, compliant implementations must provide at least a minimal implementation that conforms to the function prototype defined by the Common RF MEL API. Each class, enumeration, and public member function in the Common RF MEL API is marked with a Doxygen tag stating whether it is required to be supported by all RF MEL implementations. The full list of classes, enumerations, and functions that are required, required under certain conditions, or optional are provided in the Common RF MEL Doxygen documentation. See **Table 1.4.1** for instructions on downloading the Common RF MEL Doxygen documentation. The conditions under which some API elements are

required are the following:

- **RequiredIfAGC** – The tagged API elements are required if the associated MFA provides automatic gain control.
- **RequiredIfBaselineRelativePointing** – The tagged API elements are required if the associated MFA supports pointing vectors relative to the baseline of the aperture in radians.
- **RequiredIfBeamTaperingWeights** – The tagged API elements are required if the associated MFA supports controlling beam tapering of job events through the use of the service setting aperture weights. For more information about using aperture weights, see **Section 4.8**.
- **RequiredIfCachedWaveformJobInterface** – The tagged API elements are required if the associated MFA supports the use of cached waveforms for transmit jobs.
- **RequiredIfCalibrationInJob** – The tagged API elements are required if the associated MFA supports performing calibration within a Job.
- **RequiredIfCommandErase** – The tagged API elements are required if the associated MFA supports data erasure on command outside the normal shutdown process.
- **RequiredIfCosite** – The tagged API elements are required if the associated MFA supports Inhibit VAs, supporting RF compatibility for a platform.
- **RequiredIfCustomWaveform** – The tagged API elements are required if the associated MFA supports custom waveforms.
- **RequiredIfDEA** – The tagged API elements are required if the associated MFA supports Direct Endpoint Access (DEA). For more information about DEA, see **Section 4.9**.
- **RequiredIfDynamicWaveformTransmitJobInterface** – The tagged API elements are required if the associated MFA supports transmit of dynamic waveforms through jobs.
- **RequiredIfDynamicWeightsSupported** - The tagged API elements are required if the associated MFA supports controlling beam tapering of job events through dynamically setting aperture weights.
- **RequiredIfECEFPointing** – The tagged API elements are required if the associated MFA supports pointing vectors given in the Earth-Centered, Earth-Fixed (ECEF) coordinate system.
- **RequiredIfInterruptableJobInterface** – The tagged API elements are required if the associated MFA supports interruptable jobs.
- **RequiredIfJobEventDurationExtension** – The tagged API elements are required if the associated MFA supports job event duration extensions.
- **RequiredIfLLAPointing** – The tagged API elements are required if the associated MFA supports pointing vectors given in the WGS-84 coordinates.
- **RequiredIfLFM** – The tagged API elements are required if the associated MFA supports Linear Frequency Modulation (LFM).

- **RequiredIfLFSupport** – The tagged API elements are required if the associated MFA supports Local Functions.
- **RequiredIfOOK** – The tagged API elements are required if the associated MFA supports On-Off Keying (OOK).
- **RequiredIfPhaseCode** – The tagged API elements are required if the associated MFA supports phase modulation.
- **RequiredIfPlatformRelativePointing** – The tagged API elements are required if the associated MFA supports pointing vectors relative to the platform of reference.
- **RequiredIfRDMA** – The tagged API elements are required if the associated MFA supports the use of Remote Direct Memory Access (RDMA). For more information about RDMA, see **Section 5.4**.
- **RequiredIfReceive** – The tagged API elements are required if the associated MFA supports receive.
- **RequiredIfTransmit** – The tagged API elements are required if the associated MFA supports transmit.
- **RequiredIfUserDefinedContextData** – The tagged API elements are required if the associated MFA supports the ability to provide user defined context to MFA provided data.
- **RequiredIfWaveformStreamSupport** – The tagged API elements are required if the associated MFA supports the use of waveform streams for transmit jobs.
- **RequiredIfOEMChannelizer** – The tagged API elements are required if the associated MFA supports the channelization Original Equipment Manufacturer (OEM) function. For more information about OEM functions, see the MFA Volume.
- **RequiredIfOEMPulseDetection** – The tagged API elements are required if the associated MFA supports the pulse detection OEM function. For more information about OEM functions, see the MFA Volume.
- **RequiredIfMFADrivenControls** – The tagged API elements are required if the associated MFA supports the use of MFA driven controls.

The referenced RF MEL API documentation detailed in Table 1.4.1 provides further information on the API and its design.

Name	Programming Language	Description
rfmel-doxygen-v.tar.gz	C++	HTML API documentation. Extract from the SK for the selected interface, then navigate to the folders "doxygen" then "html" within the folder structure. Open the file index.html. The file name for extraction will be appended with the released version number (e.g., rfmel-doxygen-v[version].tar.gz).

Table 1.4.1: This Table Provides Reference Information for the RF MEL API.

1.5 Deprecated Function Compliance

AMS GRA uses deprecation, denoted by the `@deprecated` tag, to identify elements that are targeted for deletion from the API. Methods that are deprecated may continue to be used while present but are considered optional, even if required in previous releases. Usage of these elements will produce a warning at compile-time to notify adopters. Deprecated elements will be removed as a part of major release cycles. Continued use of elements after they have been deleted from the released MEL API will result in non-compliance.

2.0 Application Programming Interface (API) Overview

The RF MEL API defines the software interface used by a service to interact with an RF MFA. Services use the RF MEL API to command and control MFAs and MFAs may respond to the services with output data (e.g., In-phase/Quadrature (I/Q) or Pulse Descriptor Words (PDWs)) using AMS GRA tailored VITA 49.2 or Jobs-based Protocol packets. See **Section 3.1** for more details on the Jobs-based Protocol and **Section 3.2** for more details on VITA 49.2. Additional key concepts and API classes are also described in more detail in this chapter.

The RF MEL API exists as a set of C++ classes and C++ is the only language currently supported. The API depends on the C++ Standard Template Library (STL) and Boost as well as the AMS GRA Common MEL and Math libraries. Many API operations are asynchronous; that is, the caller of the function receives an answer immediately, and the result of the function provides methods to wait/poll for completion of the operation and check whether it was successful. These operations are built on the `std::future` class, while the indication of success or failure is built using the `std::variant` class.

The RF MEL implementation is the MFA specific implementation of the RF MEL API provided for a specific MFA. A single RF MEL implementation is provided for a single MFA.

2.1 RF MEL Domain Library

In AMS GRA Starter Kit 12.0, a modularized RF MEL API is being instated. This modularized RF MEL API is partitioned into functional components, called domain libraries, that enable services to use them independently based on the needs of that service. RF MEL API partitioning provides Mission System Integrators (MSIs) and services with the ability to manage and control access to critical API functions. The modularized RF MEL API replaces the RF MEL API from previous AMS GRA Starter Kit releases. The current API creates the following domain libraries.

2.1.1 Admin MEL Domain Library

The admin RF MEL domain library provides skill-independent configuration, command, and status of the RF MFA. This functionality covers MFA state management, Built-In Test (BIT) and Calibration, run-time configuration management, and security as well as functions that govern system state data that impact MFA functionality including navigation, Emissions Control (EMCON) and status reporting. For functionality within the Monitor MEL domain

library, see the Monitor MEL domain library **Section 2.1.6**.

The Universal Command and Control Interface (UCI) Interface Controls (UCI Control) initializes UCI Control which includes the following class files (inheriting Base classes and adding controller methods):

- BIT_Control
- CalibrationControl
- EMCON_Control
- Position Navigation and Time (PNT)_Control
- SettingsControl
- StatusControl

The UCI Base class files, located in the folder uci_common which contains common functionality:

- BIT_Base
- CalibrationBase
- EMCON_Base
- PNT_Base
- StatusBase

2.1.2 C2 MEL Domain Library

The C2 MEL domain library within the RF MEL definition is responsible for the access to the command and control and waveform transmission functionality of the MFA. The core functions of the C2 MEL domain library are Job Request, Job Detail, Job Interval, and Element Group Request.

2.1.3 COSITE MEL Domain Library

The COSITE MEL domain library provides access to the COSITE and blanking functionality of the MFA. To accomplish this functionality, the MEL domain library allows a Skill/Skills to request access to an Inhibit Virtual Aperture for Antenna Jobs.

2.1.4 Data MEL Domain Library

The Data MEL domain library within the RF MEL definition is responsible for transferring data from the MFA to the applicable service. This domain library provides several functions. It provides access to an RDMA implementation, endpoint management, memory allocation, and data movement.

2.1.5 DEA MEL Domain Library

The DEA MEL domain library within the RF MEL definition is responsible for providing direct access functionality (i.e. DEA, RDMA) of the MFA. The domain library provides a DEA Endpoint function for an MFA provider to adopt and implement.

2.1.6 Monitor MEL Domain Library

The Monitor MEL domain library within the RF MEL definition is responsible for status monitoring of the MFA. The partition provides several core functions, including UCI Interface Status, RF Activity, Security Events, and Aperture Status for an MFA provider to adopt and implement. As part of the RF MEL split, the UCI interfaces were separated into Admin, Monitor, and UCI Common interfaces. As a result, the Monitor MEL domain library provides status of BIT, Calibration, EMCON, and other status. For functionality within the Admin MEL domain library, see the Admin MEL domain library **Section 2.1.1**.

2.2 RF MEL Class

The RF MEL class is the top-level container of the RF MEL API, providing methods to allocate concrete instances of other classes. From this class, a service retrieves detailed information about the MFA, establishes endpoints, and requests VAs.

The RF MEL class is instantiated via the `DynamicLoader::loadMEL()` function. This function requires the input of a path to the external module and an implementation specific configuration string and returns a pointer to the instantiated `RFMEL`.

2.3 RF MFA Info Class

The RF MFA Info class provides the details needed for a service to use a particular MFA. These details include the time resolution of the scheduler, limits of the RF frequency range, beam/bandwidth limitations, maximum sample bandwidth, maximum data bandwidth, and maximum beams. In addition, the service can query for scheduler resolution to account for quantization of pulse widths and other resolution dependent parameters.

2.4 Virtual Aperture (VA) Class

A VA is an abstract set of MFA resources. A VA is composed of the collection of MFA hardware, OEM software, and Local Functions. A Skill can support the loading of a Local Function by providing Local Function file information in the VA Request. However, the Local Function information can be left empty if no Local Functions are available. AMS GRA also includes the concept of an Inhibit VA which allows Skills to support COSITE. See the Resource Management and MFA volumes for more information.

A VA is physically realized by one or more Virtual Aperture Instances (VAIs). A VAI includes one or more Element Groups. An Element Group is the atomic unit of the MFA which the service can command to Transmit (Tx) or Receive (Rx). An Element Group has a single-phase center. An Element Group is restricted to elements on a single MFA face. An Element Group consists of one or more MFA elements and may contain noncontiguous elements. An Element Group may be used to perform Tx, Rx, or both. **Figure 2.4.1** shows three example VAs containing different Element Group configurations.

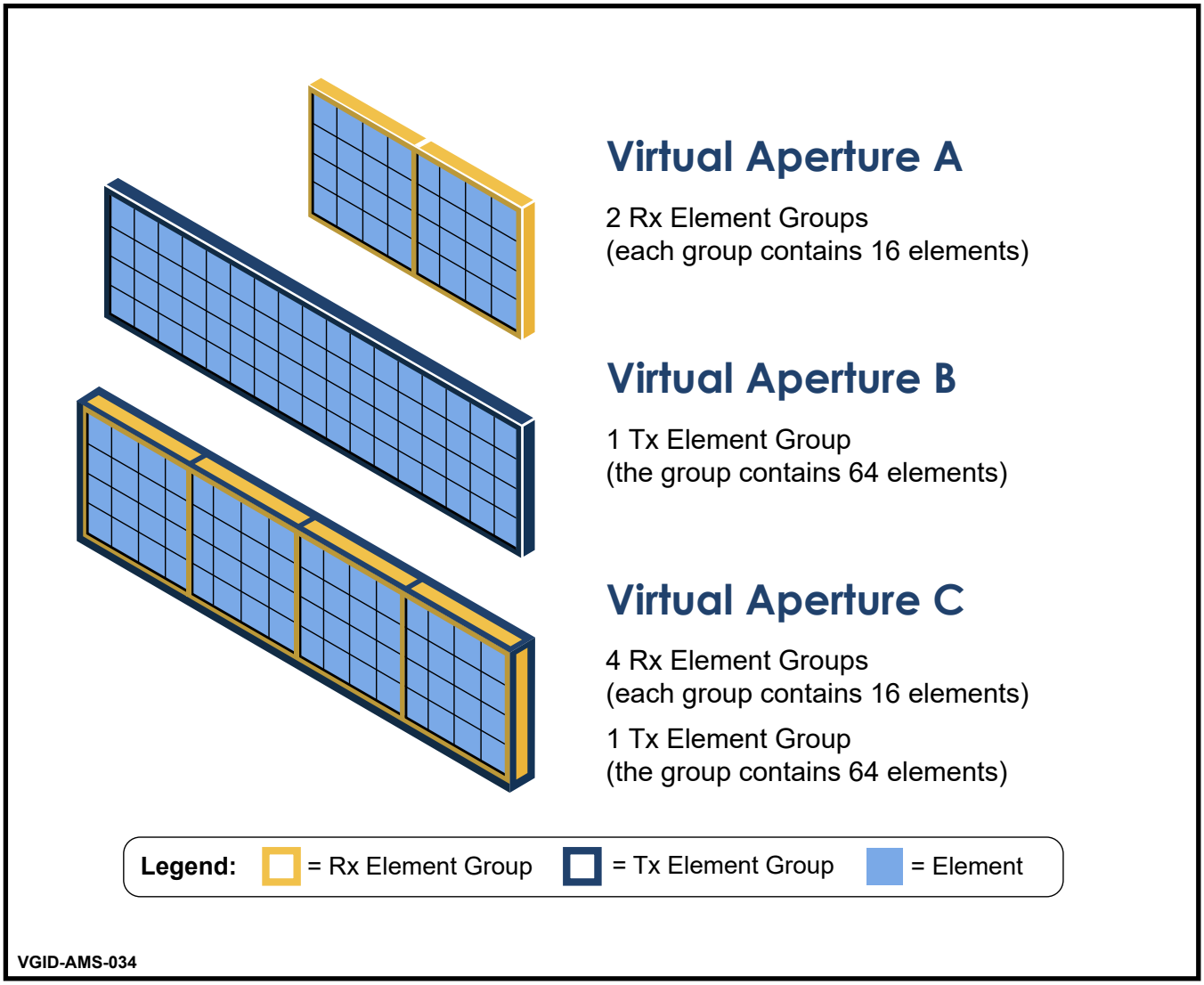


Figure 2.4.1: This Figure Shows Several Example Relationships Between VAs and MFA Faces.

The relationship between VAs and MFA faces for a particular MFA is MFA specific. An MFA may provide VAs which include Element Groups on multiple different MFA faces, VAs which only include Element Groups on a single MFA face, or a combination of both.

Figure 2.4.2 shows an example VA consisting of half of an MFA face, which is realized by four possible VAIs denoted VAI 1 through VAI 4.

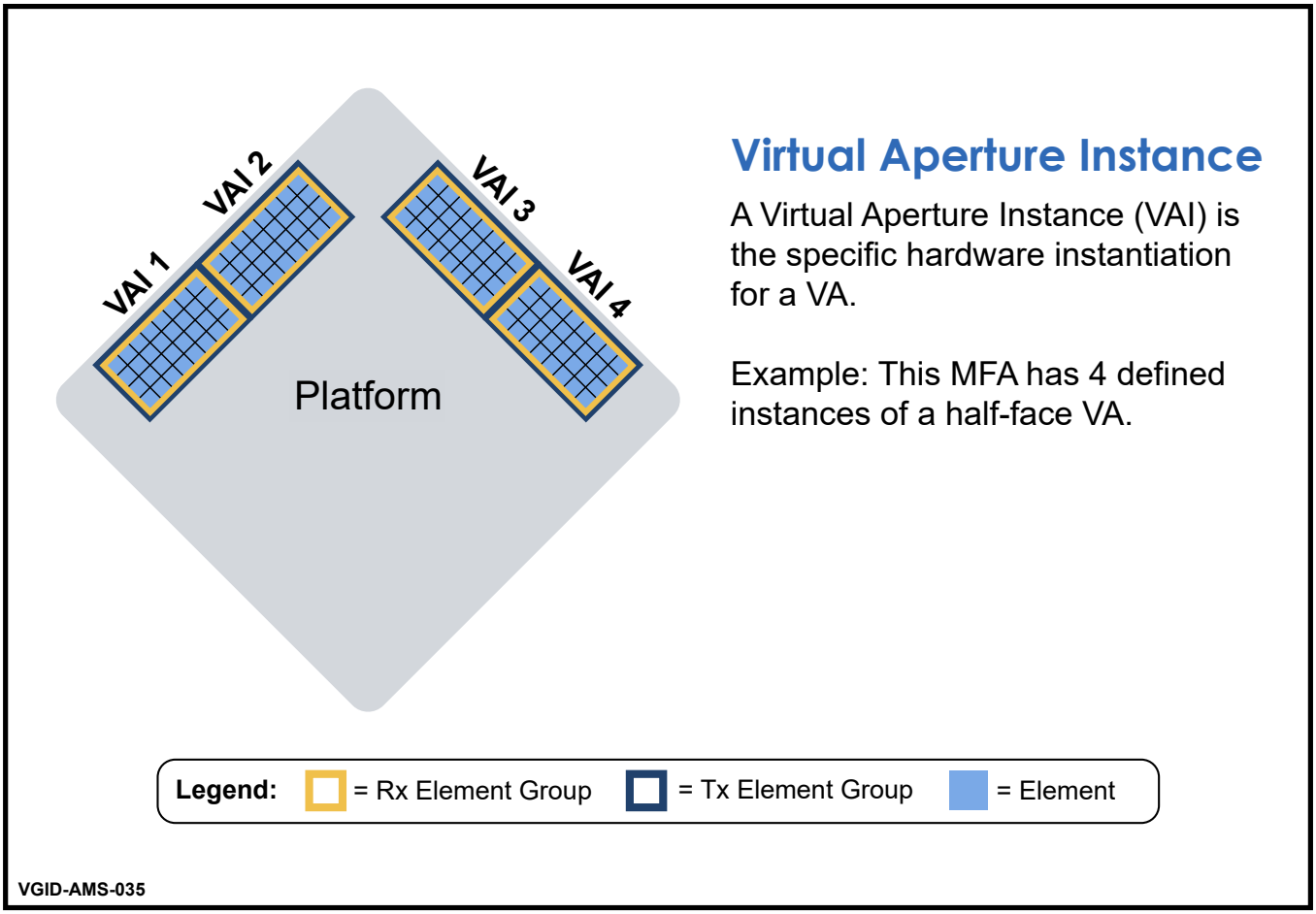


Figure 2.4.2: This Figure Shows an Example of an Instantiation of a VA Spanning Multiple Faces of an MFA.

Figure 2.4.3 shows an example VA which spans multiple faces.

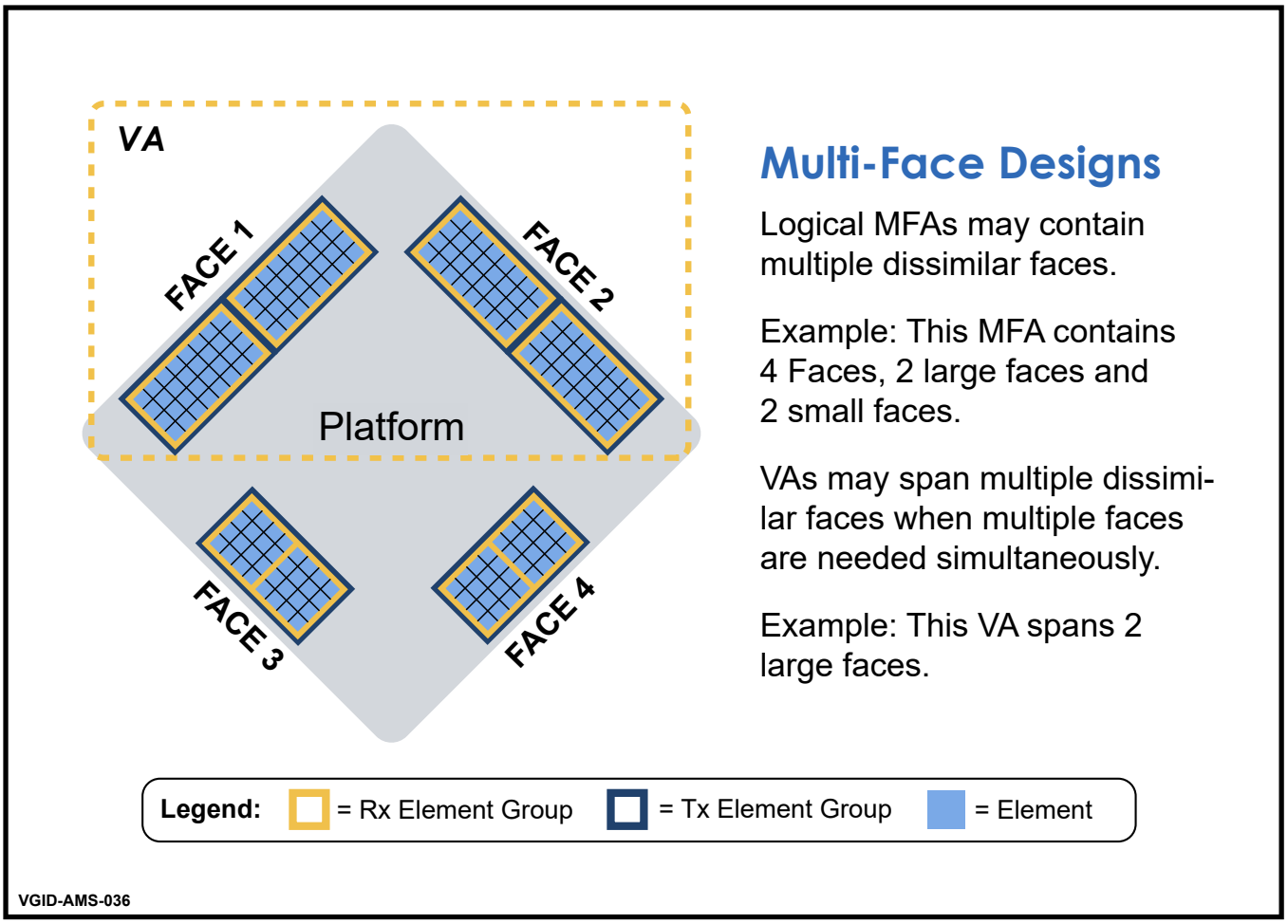


Figure 2.4.3: This Figure Shows an Example of a Multi-Face VA that Spans Multiple Dissimilar Faces.

Figure 2.4.4 shows example VAIs which include combinations of dissimilar MFA faces.

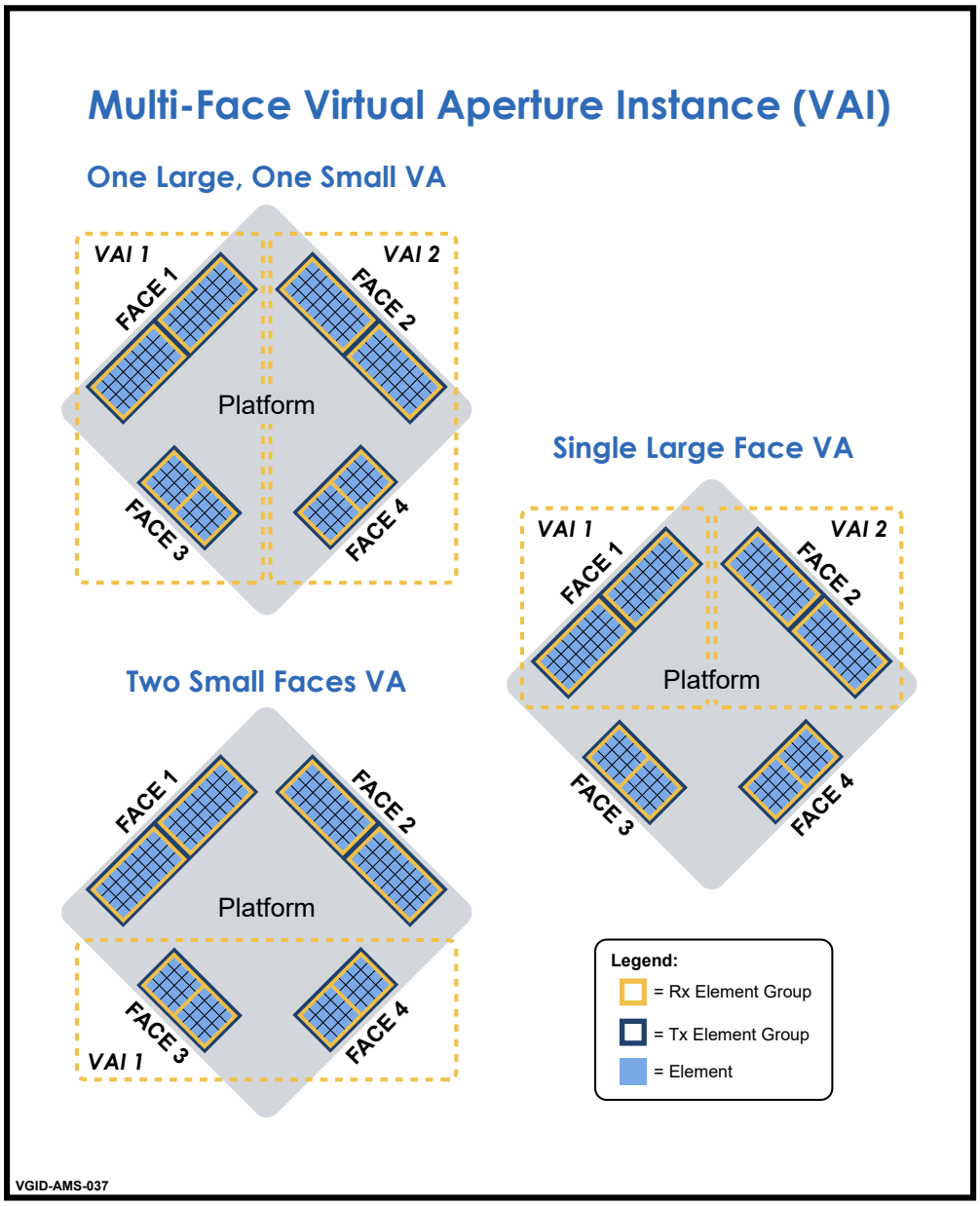


Figure 2.4.4: This Figure Shows Three Examples of Multi-Face VAI Designs to Incorporate Different Usage Patterns and Service Needs.

The VA class represents the service's interface to the abstract set of MFA resources that make up a particular VA. The VA class can be used to query for a particular VA's partitioning of the array into Element Groups and to retrieve the status of any associated Local Functions. The VA class also provides the API with an interface to create coefficient tables that can be used in tapering operations and structures to describe pulse modulation and other functions.

2.5 Virtual Aperture Database (VADB)

In addition to the RF MEL API available to services at runtime, a separate interface named the Virtual Aperture Database (VADB) API defines functions available to Skill developers at design time. This library is used by Skill providers to extract statically available VA information needed to create a tailored package for use by the Skill at runtime. The library provides information about the MFA, its available VAs, and the available instances for each VA.

- MFA: The library provides information about the MFA through `getRFMFAInfo`.
- VAs: The library provides the list of all available VAs for the MFA. The library describes each VA through a `VirtualApertureDefinition` which includes a unique numerical `VADefinitionID`.
- VA Instances: The library also provides information about each instance of that VA in the form of the `VirtualApertureInstanceDescriptor`. Each instance is associated with a unique numerical `VirtualApertureInstanceID`.

For example, in the case where the MFA provides a predefined VA with `VADefinitionID` = 1, which consists of one half of one MFA face, which can be instantiated as one instance on the left half of the face, and another instance on the right half of the face. Then the associated VADB would contain a `VirtualApertureDefinition` for `VADefinitionID` = 1, and it would contain two `VirtualApertureInstanceDescriptors`, one for the left half and one for the right half. The individual `VirtualApertureInstanceDescriptors` are used to communicate any differences in different instances of the same VA, for example, a different `RxPhaseCenter`.

Due to the lack of information for element failures or performance degradation at design time, this static information assumes as-delivered performance. For specific VADB API documentation, see the Doxygen documentation referenced in **Table 1.4.1**.

2.6 Job Classes

The main interaction between a service and VA is to request Jobs. A job represents access to a particular VA for some duration of time in order to use MFA resources (e.g., make requests to Rx or Tx). A Job Request of a VA cannot be granted unless all of the components of the VA are available for use by the Skill during the requested job duration. The Job Interface refers to the set of classes and functions within the RF MEL which allow an AMS GRA Service to submit jobs to the VA and receive the resulting products. At a high level, this process includes the service submitting a Job Request, receiving a Job Detail, and submitting Job Intervals. A job consists of one or more Job Intervals. A Job Interval contains a Sequence with a repeat count. A Sequence contains one or more events, which can be Tx Events or Rx Events. Different parameters are available for each of these components of a job. For more information on Job Intervals, Sequences, and Job Events, see the RF MEL Doxygen Documentation referenced in **Table 1.4.1**.

The Job Request class is used by the service to set up basic parameters of its requested job, such as the job's duration, minimum start time, and maximum complete time. The Job Request also contains a `CapabilityID` and `ActivityID` which allows the Job Request to be correlated to the higher level Open Mission Systems (OMS) commands that prompted the Skill to create the job.

The Job Detail class represents the specific MFA resources available to the service as a result of a successful job request to the VA. It supports querying for the allocated time windows and the selected VA instance.

The Job Interval is analogous to a collection interval or dwell. The Job Interval consists of a Sequence of a specified duration and repeat count, which contains Tx or Rx Events. These Job Intervals are sent to the MFA by the service using the Job Detail. **Figure 2.6.1** shows the temporal relationship inside of the Job Detail between a Job Interval, a Sequence, and a Job Event. The service may submit all of the desired Job Intervals for a single job at once, or submit groups of Job Intervals as desired. The `flush()` method is used to submit the Job Intervals added so far to be executed by the MFA, with the implication that more Job Intervals will be added. The `finalize()` method indicates that the last Job Interval has been specified and returns a placeholder for the eventual status of the job. That status indicates whether the job was rejected by job validation logic, interrupted by higher priority jobs, or the job executed to completion. If desired, the service may also receive status on a per Job Interval basis through the use of a callback, either for all Job Intervals or only upon exception.

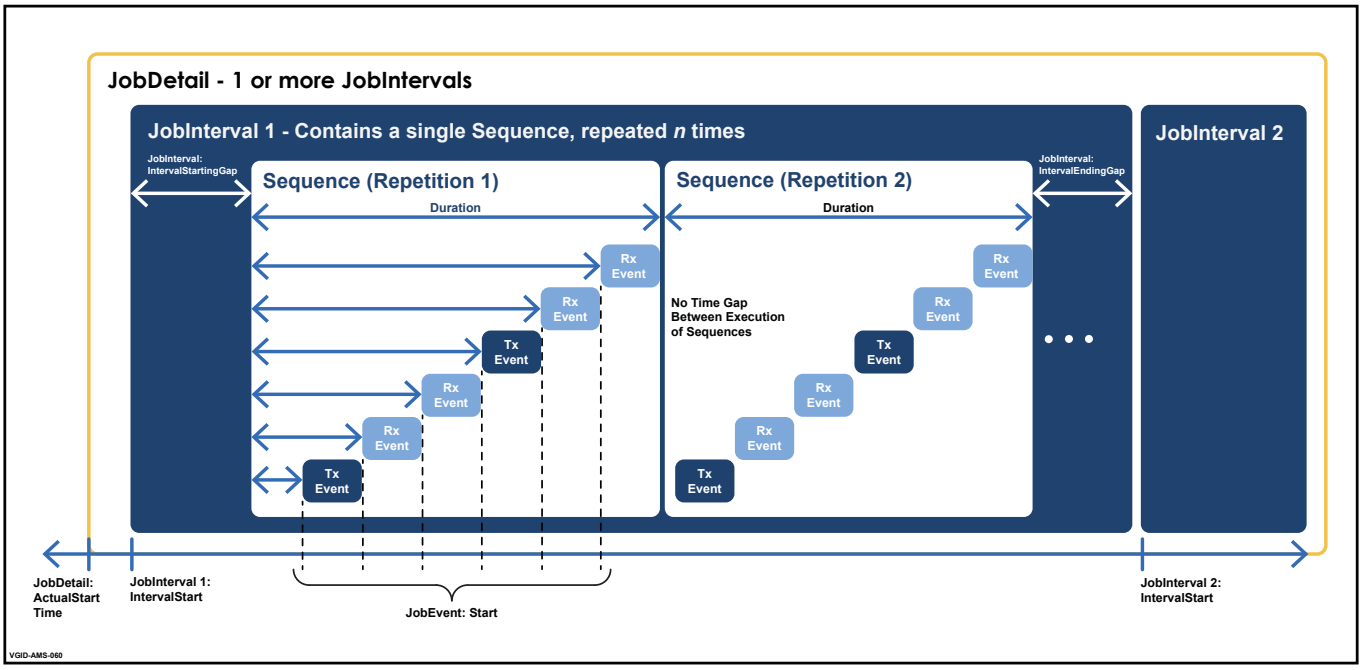


Figure 2.6.1: This Figure Shows the Relationship Between Job Details and Job Intervals. This Includes the Patterns at Which Services Can Submit Job Intervals to be Executed by the MFA.

For more detail on using the Job Interface, see **Section 4.4**, **Section 4.5**, **Section 4.7**, and **Section 4.8**.

2.7 Endpoint Classes

The Product Rx Endpoint and Waveform Tx Endpoint classes encapsulate the high throughput, low latency ABB interface to the MFA. They provide the `EndpointIDs` as an abstraction of all the links between the service's processes within the MFP and the endpoints in the MFA. The memory behind the endpoint might be Random Access Memory (RAM) connected directly to the Central Processing Unit (CPU), memory inside a Field-Programmable Gate Array (FPGA), or Graphics Processing Unit (GPU) memory, if supported by the MFP. The following is a list of Endpoint Types derived from the Base Endpoint class:

- **Product Rx** – This class represents a communications channel from the MFA to the service by which outputs from a job are received. If RDMA is supported, this class creates the protection domain/memory region, etc. to allow the MFA remote access to memory resident in the Skill's process space. Any number of devices within the MFA may need to access this local memory region, so this class is responsible for communicating with the

infrastructure in the MFA for setting up all sockets, queue pairs, etc. necessary for any MFA device, selected by the Virtual Aperture Scheduler, to access the local memory region according to the commands in the MFA Job Request.

- **Waveform Tx** – This class represents a communications channel to the MFA from the service by which dynamic waveform definitions are sent. If RDMA is supported, this class creates the protection domain/memory region, etc. to allow the MFA remote access to memory resident in the Skill's process space. Any number of devices within the MFA may need to access this local memory region, so this class is responsible for communicating with the infrastructure in the MFA for setting up all sockets, queue pairs, etc. necessary for any MFA device, selected by the Virtual Aperture Scheduler, to access the local memory region according to the commands in the MFA Job Request.
- **External** – This class represents a communications channel to the MFA to an external endpoint. This class supports external endpoints for Rx, Tx, and Direct Access.
- **Direct Access** – This class provides direct communication with the MFA over an abstracted communications bus for both command and data packets that are constrained to a limited collection of VITA 49.2 Control and Data messages.

3.0 Standards and Protocols

The AMS GRA supports two RF message structures to support data transfer between MFAs and AMS GRA Services: the Jobs-based Protocol, defined in **Section 3.1**, or AMS GRA-tailored VITA 49.2 packets, described in **Section 3.2**. Both of these structures, or data types, rely on a common endpoint implementation. While the DEA exclusively uses a custom VITA 49.2 format, the `ProductRxEndpoint`, `WaveformTxEndpoint`, and `ExternalEndpoints` allow for both the Jobs-based Protocol and AMS GRA-tailored VITA 49.2 data format.

The Jobs-based Protocol is typically used when MEL API services are required to schedule and execute jobs on the MFA. The MEL provides a Jobs API for services that use a common message structure to command and control, and receive output data packets from the MFA. A use case showing the advantage of the Jobs-based Protocol is enabling the split of header content and data content between a CPU and GPU respectively.

The AMS GRA-tailored VITA 49.2 protocol is typically used to support DEA. The VITA standard provides an existing modified message structure that allows for commonality across the frequency spectrum. While services use the RF MEL API to command and control MFAs, MFAs use AMS GRA-tailored VITA 49.2 packets to provide alternative control and output data (e.g., I/Q or PDWs) to the services.

3.1 Jobs-based Protocol

The Jobs-based Protocol provides a message structure that allows for commonality. Services use this protocol to command and control MFAs, which can then be used by MFAs to provide output data (e.g., I/Q or PDWs) in the form of packets to the services. See **Chapter 2** for more details on the RF MEL API.

3.1.1 Jobs-based Frame Structure

All Jobs-based frames consist of three Header Types (Command), one Data Payload Type, and one optional Trailer Type. The frame is packaged in the order shown in **Figure 3.1.1**.

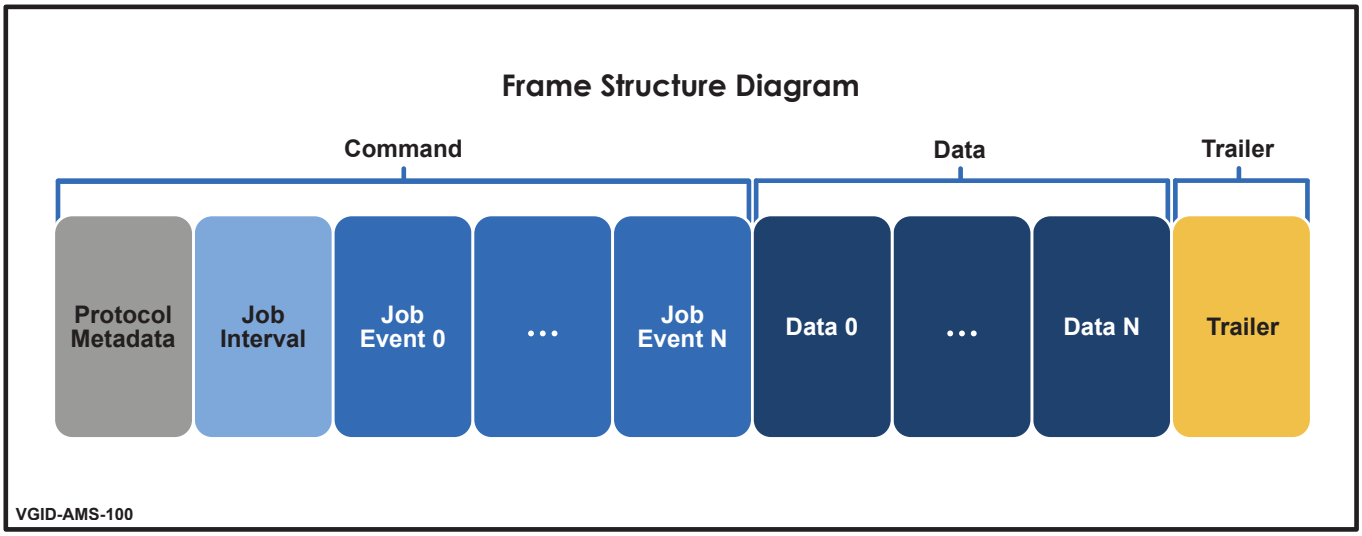


Figure 3.1.1: This Figure Shows the Jobs-Based Frame Structure, Which Consists of a Command Header Type, a Data Type, and an Optional Trailer Type.

3.1.2 Jobs-based Packet Types

The Jobs-based format supports three types of content: Command, Data, and Trailer. Command is captured within Command Information which convey the control mechanism used to set the Protocol Metadata, Job Interval, and Job Events. Data is captured within the Data Information to convey a digitized signal. Trailer is captured within the Trailer Information for optional trailer data.

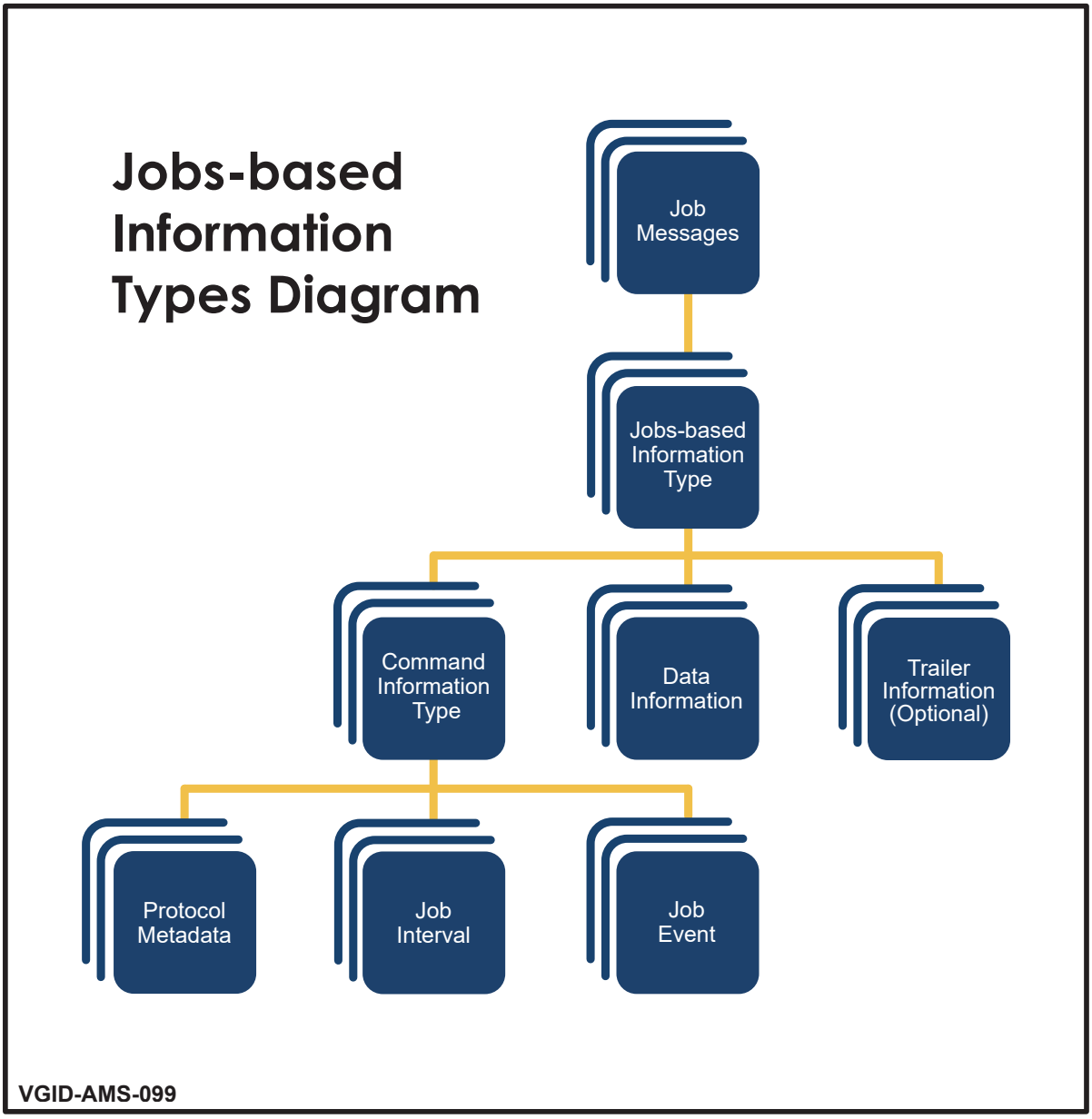


Figure 3.1.2: This Figure Represents a Flow Down of Jobs-Based Packet Types From Job Messages Down to the Header Information Within the Jobs-Based Frame Structure.

3.1.2.1 Protocol Metadata

The Protocol Metadata consists of a single 32-bit word and is the first header within the Jobs-based Frame Structure.

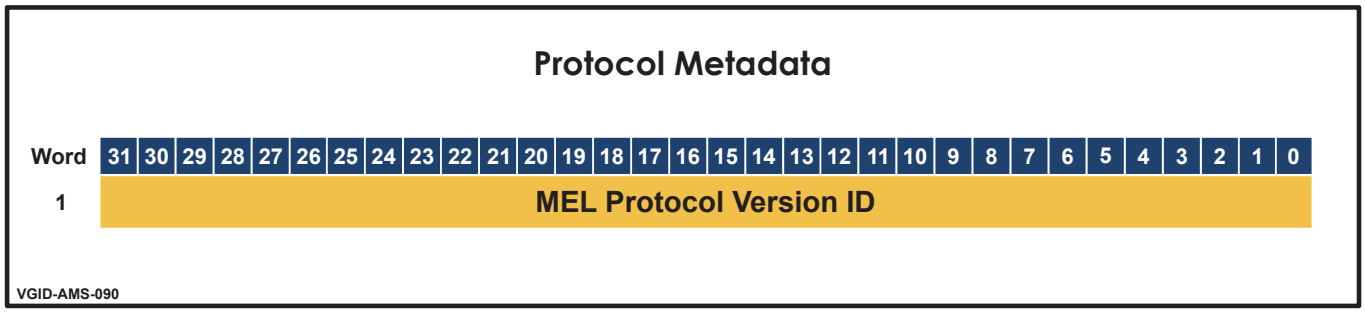


Figure 3.1.3: This Figure Shows the 32-bit Word Structure of the Protocol Metadata Within the Jobs-Based Frame Structure.

3.1.2.1.1 MEL Protocol Version ID

`melProtocolVersionID`: The MEL Protocol Version Identification (ID) shall be a generic 32-bit identifier.

The MEL Protocol Version ID shall be used as linkage to the MEL Protocol version used within the Jobs-based Protocol.

The MEL Protocol Version ID shall parallel the Common MEL API version number. **Figure 3.1.4** shows the construction of MEL Protocol Version ID. In this construction, bits 31 to 17 inclusive comprise the MEL Protocol Version Major ID and bits 16 to 0 inclusive comprise the MEL Protocol Version Minor ID.

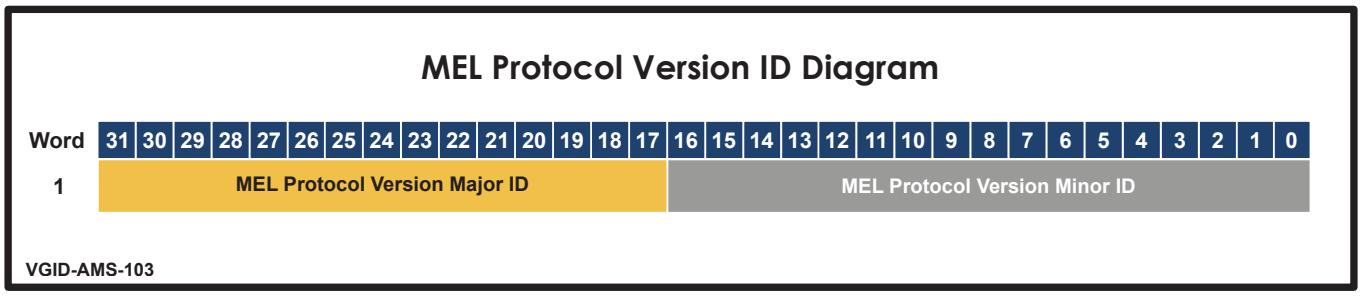


Figure 3.1.4: This Diagram shows MEL Protocol Version ID as a Single 32-Bit Word. This Word consists of MEL Protocol Version Major ID and MEL Protocol Version Minor ID.

Field Name	No. of Words	Packet Byte Number	C++ Type
`melProtocolVersionID`	1	0	uint32_t

Table 3.1.1: This Table Displays That the `melProtocolVersionID` is Defined by the Generic 32-bit Identifier.

3.1.2.2 Job Interval Information

The Interval Packet is the second header within the Jobs-based Frame Structure with a total of 20 32-bit words.

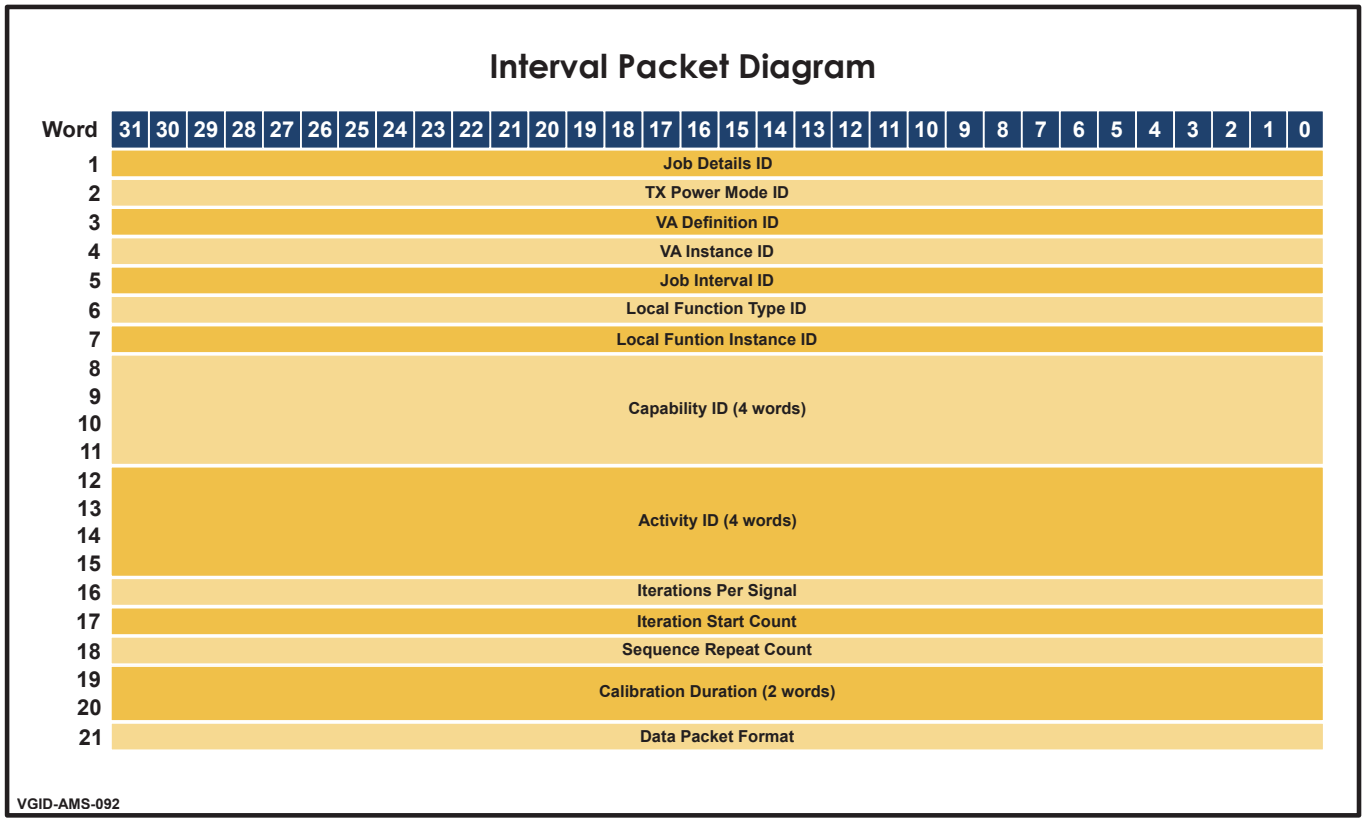


Figure 3.1.5: This Diagram Shows the 20 32-bit Words Available Within the Interval packet Diagram.

3.1.2.2.1 Job Details ID

`jobDetailsID`: The Job Details ID shall be a generic 32-bit identifier.

The Job Details ID shall be used as linkage to the associated Job Detail requested within the Jobs-based Protocol.

Field Name	No. of Words	Packet Byte Number	C++ Type
`jobDetailsID`	1	0	uint32_t

Table 3.1.2: This Table Displays That the Jobs Details ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.2 Tx Power Mode ID

`txPowerModeID`: The Tx Power Mode ID shall be a generic 32-bit identifier.

The Tx Power Mode ID shall be used as linkage to the uniquely identifiable MFA-specific commandable Transmit Power Configurations.

Field Name	No. of Words	Packet Byte Number	C++ Type
`txPowerModeID`	1	4	uint32_t

Table 3.1.3: This Table Displays That the Tx Power Mode ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.3 VA Definition ID

`vaDefinitionID`: The VA Definition ID shall be a generic 32-bit identifier.

The VA Definition ID shall be used as linkage to the uniquely identifiable single VA.

Field Name	No. of Words	Packet Byte Number	C++ Type
`vaDefinitionID`	1	8	uint32_t

Table 3.1.4: This Table Displays That the VA Definition ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.4 VA Instance ID

`vaInstanceID`: The VA Instance ID shall be a generic 32-bit identifier.

The VA Instance ID shall be used as linkage to the uniquely identifiable instance of a single VAI.

Field Name	No. of Words	Packet Byte Number	C++ Type
`vaInstanceID`	1	12	uint32_t

Table 3.1.5: This Table Displays That the VA Instance ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.5 Job Interval ID

`jobIntervalID`: The Job Interval ID shall be a generic 32-bit identifier.

The Job Interval ID shall be used as linkage to the uniquely identifiable Job Interval corresponding to this status entry.

Field Name	No. of Words	Packet Byte Number	C++ Type
`jobIntervalID`	1	16	uint32_t

Table 3.1.6: This Table Displays That the Job Interval ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.6 Local Function Type ID

`lfTypeID`: The Local Function Type ID shall be a generic 32-bit identifier.

The Local Function Type ID shall be used in combination with `lfInstanceID` as linkage to the uniquely identifiable Local Function in a VA Instance.

Field Name	No. of Words	Packet Byte Number	C++ Type
`lfTypeID`	1	20	uint32_t

Table 3.1.7: This Table Displays That the Local Function Type ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.7 Local Function Instance ID

`lfInstanceID`: The Local Function Instance ID shall be a generic 32-bit identifier.

The Local Function Instance ID shall be used in combination with Local Function Type ID as linkage to the uniquely identifiable Local Function in a VA Instance.

Field Name	No. of Words	Packet Byte Number	C++ Type
`lfInstanceID`	1	24	uint32_t

Table 3.1.8: This Table Displays That the Local Function Instance ID is Defined by the Generic 32-bit Identifier.

3.1.2.2.8 Capability ID

`capabilityID`: The Capability ID shall be a Universally Unique Identifier (UUID) contained within four 32-bit Words.

The Capability ID shall be used as linkage for the Capability UUID associated with the Job Request. This is the Capability UUID that corresponds to a capability previously commanded by the Mission Processing Subsystem (MPS) using a UCI Command message.

Field Name	No. of Words	Packet Byte Number	C++ Type
`capabilityID`	4	28	std::vector<uint8_t>

Table 3.1.9: This Table Displays That the Capability ID is Contained Within Four 32-bit Words.

3.1.2.2.9 Activity ID

`activityID`: The Activity ID shall be a UUID contained within four 32-bit Words.

The Activity ID shall be used as linkage for the Activity UUID associated with the Job Request. This is the Activity UUID that corresponds to a capability previously published by the Skill or service using a UCI Activity message.

Field Name	No. of Words	Packet Byte Number	C++ Type
`activityID`	4	44	std::vector<uint8_t>

Table 3.1.10: This Table Displays That the Activity ID is Contained Within Four 32-bit Words.

3.1.2.2.10 Sequence Start Count

`sequenceStartCount`: Sequence Start Count shall be a 32-bit word.

The Sequence Start Count shall define the number of iterations that must be completed before this sequence will be executed. For example, if the Sequence Start Count is equal to five, then this event will not occur until the sixth iteration of the sequence.

Field Name	No. of Words	Packet Byte Number	C++ Type
sequenceStartCount	1	60	size_t

Table 3.1.11: This Table Displays That the Iteration Start Count is Defined by a 32-bit Word.

3.1.2.2.11 Sequence Repeat Count

`sequenceRepeatCount`: Sequence Repeat Count shall be a 32-bit word.

The Sequence Repeat Count shall define the number of times the sequence should execute to perform data collection.

Field Name	No. of Words	Packet Byte Number	C++ Type
sequenceRepeatCount	1	64	size_t

Table 3.1.12: This Table Displays That the Sequence Repeat Count is Defined by a 32-bit Word.

3.1.2.2.12 Calibration Duration

`calDuration`: Calibration Duration shall be two 32-bit words.

The Calibration Duration shall define the amount of time after the end of the sequence loop to perform calibration with an Least Significant Bit (LSB) of 1 femtosecond even if the MFA cannot quantize to that level.

Field Name	No. of Words	Packet Byte Number	C++ Type
calDuration	2	68	int64_t

Table 3.1.13: This Table Displays That the Calibration Duration is Defined by Two 32-bit Words.

3.1.2.2.13 Data Packet Format

`dataPacketFormat`: Data Packet Format shall be a 32-bit word.

Field Name	No. of Words	Packet Byte Number	C++ Type
`dataPacketFormat`	1	76	uint32_t

Table 3.1.14: This Table Displays That the Data Packet Format is Defined by a 32-bit Word.

The Data Packet Format shall define the parameters that specify the packing and content of the data payload and is derived from the Data Packet Payload Format Examples, Section B.9, of the VITA 49.2 standard and Section 7.1.5.18 The Data Packet Payload Format Field of the VITA 49.0 standard.

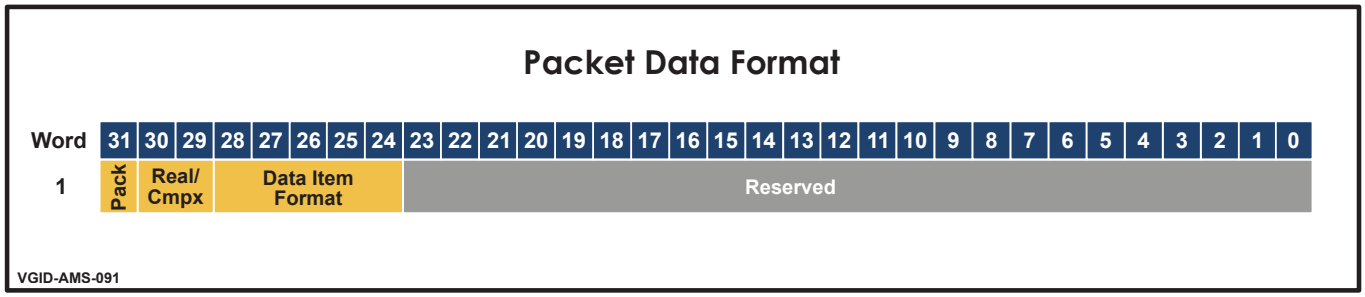


Figure 3.1.6: This Figure Displays the 32-bit Structure of the Packet Data Format.

Bit Position	Field	Field Width (bits)
[31]	Packing Method	1
[30..29]	Real/Complex Type	2
[28..24]	Data Item Format	5
[23.0]	Reserved (set to zero)	24

Table 3.1.15: This Table Shows the Field Width for Different Bit Positions Within the Data Packet Fields.

The Packing Method field shall be set to one when link-efficient packing is used in the paired Data Packet Stream. The Packing Method field shall be set to zero when processing-efficient packing is used.

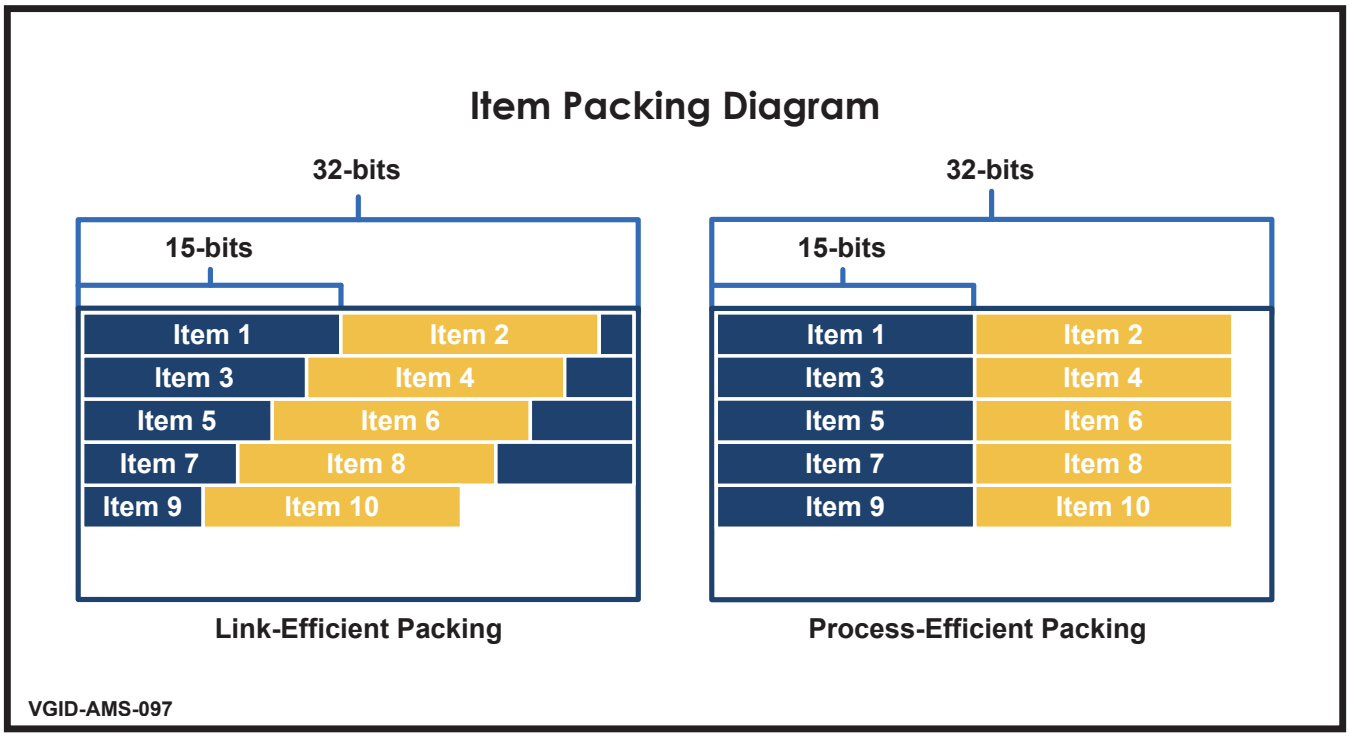


Figure 3.1.7: This Figure Shows Item Packing Diagrams for Both Link-efficient Packing and Process-efficient Packing.

The Real/Complex Type field shall indicate whether the Data Samples are real or one of the complex types using the appropriate code listed in the following table and referenced by Table 7.5 of VITA 49.0.

Code	Data Sample Type
00	Real
01	Complex, Cartesian
10	Complex, Polar
11	Reserved

Table 3.1.16: This Table Shows the Real/Complex Type Field as Defined by Code Values in the VITA 49.0 Standard.

The Data Item Format field shall contain the appropriate 5-bit code to indicate the type of Data Items used in the paired Data Packet Stream. The code shall be chosen according to the following table as referenced by Table 7.6 of VITA 49.0.

Code	Data Sample Type	Code	Data Sample Type
00000	Signed Fixed-Point	10000	Unsigned Fixed-Point
00001	Signed VRT, 1-bit exponent	10001	Unsigned VRT, 1-bit exponent
00010	Signed VRT, 2-bit exponent	10010	Unsigned VRT, 2-bit exponent
00011	Signed VRT, 3-bit exponent	10011	Unsigned VRT, 3-bit exponent
00100	Signed VRT, 4-bit exponent	10100	Unsigned VRT, 4-bit exponent
00101	Signed VRT, 5-bit exponent	10101	Unsigned VRT, 5-bit exponent
00110	Signed VRT, 6-bit exponent	10110	Unsigned VRT, 6-bit exponent
00111	Signed Fixed-Point Non-Normalized	10111	Unsigned Fixed-Point Non-Normalized
01000	Reserved	11000	Reserved
01001	Reserved	11001	Reserved
01010	Reserved	11010	Reserved
01011	Reserved	11011	Reserved
01100	Reserved	11100	Reserved
01101	IEEE-754 Half-Precision Floating-Point	11101	Reserved
01110	IEEE-754 Single-Precision Floating-Point	11110	Reserved
01111	IEEE-754 Double-Precision Floating-Point	11111	Reserved

Table 3.1.17: This Table Shows the Different Code Values and Corresponding Data Sample Types From the VITA 49.0 Standard.

3.1.2.3 Job Event Information

The Event Packet is the third header within the Jobs-based Frame Structure with a total of 30 32-bit words.

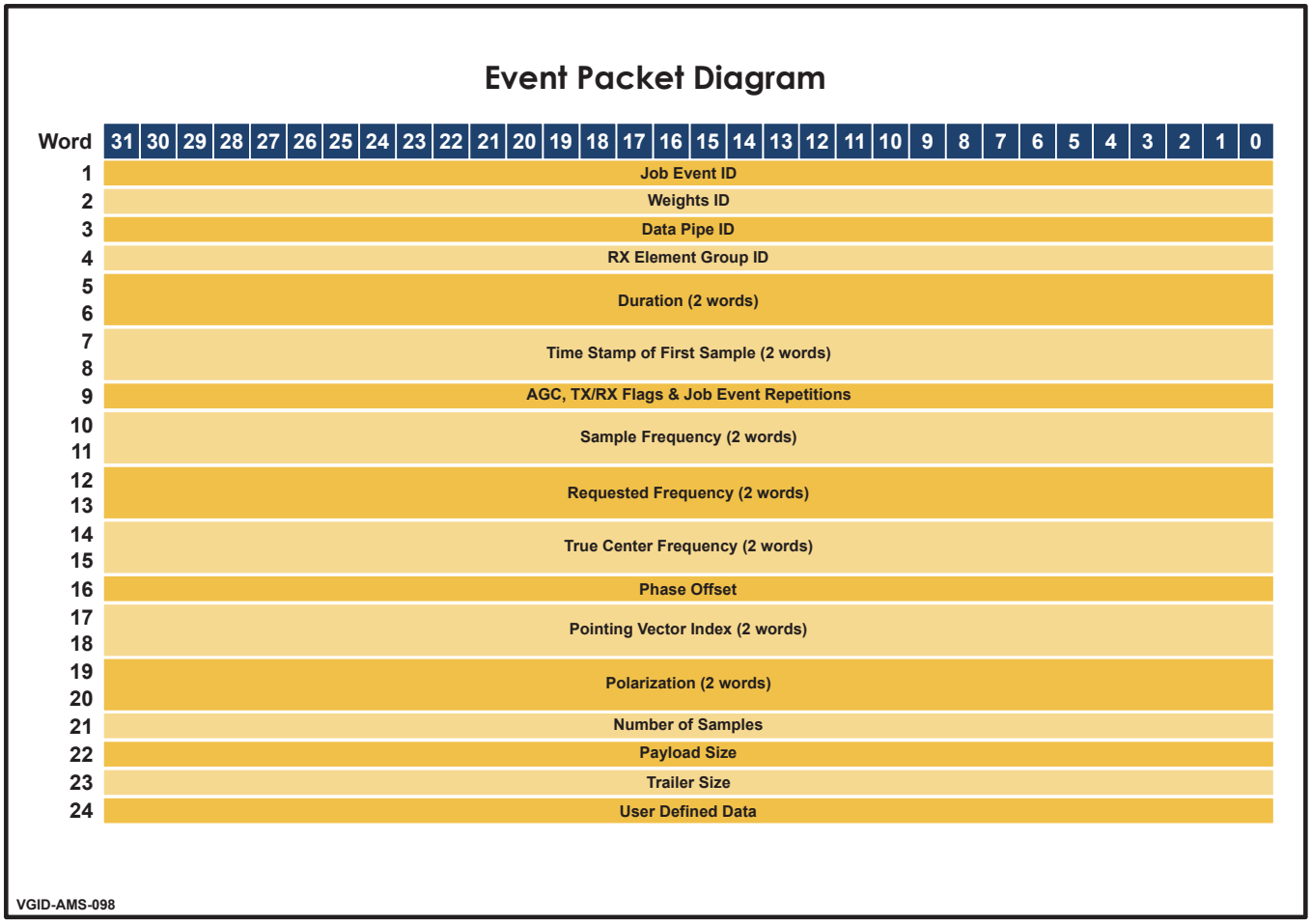


Figure 3.1.8: This Diagram Shows the Breakdown of the Event Packet Header.

3.1.2.3.1 Event ID

`eventID`: The Job Event ID shall be a generic 32-bit identifier.

The Job Event ID shall be used as linkage to the associated Job Event requested within the Jobs-based Protocol.

Field Name	No. of Words	Packet Byte Number	C++ Type
`eventID`	1	0	uint32_t

Table 3.1.18: This Table Displays That the Job Event ID is Defined by the Generic 32-bit Identifier.

3.1.2.3.2 Weights ID

`weightsID`: The Weights ID shall be a generic 32-bit identifier.

The Weights ID shall be used as linkage to the associated MFA-Specific type-ID of the table of weights requested within the Jobs-based Protocol.

Field Name	No. of Words	Packet Byte Number	C++ Type
`weightsID`	1	4	uint32_t

Table 3.1.19: This Table Displays That the Weights ID is Defined by the Generic 32-bit Identifier.

3.1.2.3.3 Data Pipe ID

`dataPipeID`: The Data Pipe ID shall be a generic 32-bit identifier.

The Data Pipe ID shall be used as linkage to the associated Data Pipe used by the MFA for data transfer.

Field Name	No. of Words	Packet Byte Number	C++ Type
`dataPipeID`	1	8	uint32_t

Table 3.1.20: This Table Displays That the Data Pipe ID is Defined by the Generic 32-bit Identifier.

3.1.2.3.4 Rx Element Group ID

`rxElementGroupID`: The Rx Element Group ID shall be a generic 32-bit identifier.

The Rx Element Group ID shall be used as linkage to the associated element group used for the Job Event requested within the Jobs-based Protocol.

Field Name	No. of Words	Packet Byte Number	C++ Type
`rxElementGroupID`	1	12	uint32_t

Table 3.1.21: This Table Displays That the Rx Element Group ID is Defined by the Generic 32-bit Identifier.

3.1.2.3.5 Duration

`duration`: The Duration shall be two 32-bit words.

The Duration shall define the amount of time of the job event with an LSB of 1 femtosecond even if the MFA cannot quantize to that level.

Field Name	No. of Words	Packet Byte Number	C++ Type
`duration`	2	16	int64_t

Table 3.1.22: This Table Displays That the Duration is Defined by Two 32-bit Words.

3.1.2.3.6 Time Stamp of First Sample

`timestampOfFirstSample`: The time stamp of first sample shall be two 32-bit words.

The time stamp of the first sample shall define the time of the first data sample. Use `quantizeDuration()` when setting this value to ensure the MFA is able to execute as requested.

Field Name	No. of Words	Packet Byte Number	C++ Type
`timestampOfFirstSample`	2	24	int64_t

Table 3.1.23: This Table Displays That the Time Stamp Of First Sample is Defined by Two 32-bit Words.

3.1.2.3.7 Event Start Count

`eventStartCount`: The Event Start Count shall be a single 32-bit word.

The Event Start Count shall define the number of iterations that must be completed before this event will be executed within the sequence. For example, if the Event Start Count is equal to five, then this event will not occur until the sixth iteration of the sequence.

Field Name	No. of Words	Packet Byte Number	C++ Type
`eventStartCount`	1	32	size_t

Table 3.1.24: This Table Displays That the Event Start Count is Defined by a 32-bit Word.

3.1.2.3.8 AGC, Tx/Rx Flags and Job Event Repetitions

`agcTxRxFlagsReps`: The AGC Tx/Rx Flags and Job Event Repetitions shall be a single 32-bit word.

The AGC Tx/Rx Flags and Job Event Repetitions shall define when a Job Event enables Automatic Gain Control (AGC) data collections with an AGC Flag at bit 31, 0=disable and 1=enable.

The AGC Tx/Rx Flags and Job Event Repetitions shall define when a Job Event Transmit or Receives with a Tx/Rx Flag at bit 30, 0=Receive and 1=Transmit.

The AGC Tx/Rx Flags and Job Event Repetitions shall define in bits 0 to 15 when a Job Event Repeats with no repetitions equal to 0 and the maximum amount of repetitions equal to 65,535.

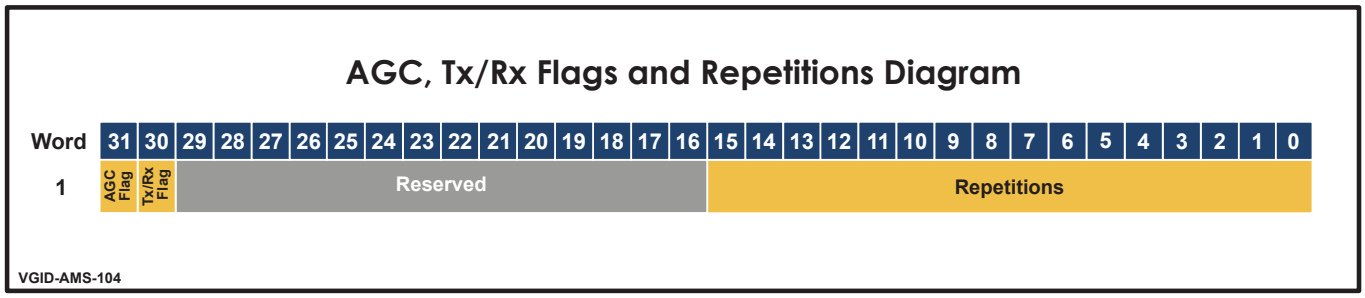


Figure 3.1.9: This Figure Shows the Structure For the AGC Tx/Rx Flags and Job Repetitions For Bits 0-15 and 16-30 For When There are Job Event Repetitions.

Field Name	No. of Words	Packet Byte Number	C++ Type
agcTxRxFlagsReps	1	36	uint32_t

Table 3.1.25: This Table Displays That the AGC, Tx/Rx Flags, and Job Repetitions are Defined by a 32-bit Word.

3.1.2.3.9 Sample Frequency

`sampleFrequencyHz`: The Sample Frequency shall be two 32-bit words.

The Sample Frequency shall define the sample frequency of the event in Hertz.

Field Name	No. of Words	Packet Byte Number	C++ Type
sampleFrequencyHz	2	40	int64_t

Table 3.1.26: This Table Displays That the Sample Frequency is Defined by Two 32-bit Words.

3.1.2.3.10 Requested Frequency

`requestedFrequencyHz`: The Requested Frequency shall be two 32-bit words.

The Requested Frequency shall define the requested center frequency of the event in Hertz.

Field Name	No. of Words	Packet Byte Number	C++ Type
requestedFrequencyHz	2	48	int64_t

Table 3.1.27: This Table Displays That the Requested Frequency is Defined by Two 32-bit Words.

3.1.2.3.11 True Center Frequency

`trueCenterFrequencyHz`: The True Center Frequency shall be two 32-bit words.

The True Center Frequency shall define the actual center frequency in Hertz at which the event was fulfilled.

Field Name	No. of Words	Packet Byte Number	C++ Type
`trueCenterFrequencyHz`	2	56	int64_t

Table 3.1.28: This Table Displays That the True Center Frequency is Defined by Two 32-bit Words.

3.1.2.3.12 Phase Offset

`phaseOffset`: The Phase Offset shall be a single 32-bit word.

The Phase Offset shall define the initial phase in radians for phase-modulated waveforms.

The phase field shall be expressed in units of radians (rad). The phase value shall be expressed in two's-complement format in the lower 16 bits of the phase field as shown in the table below. This field has an integer and a fractional part with the radix point to the right of bit 7 of the field.

Figure 3.1.10 shows the construction of Phase Offset. In this construction, bits 31 to 16 inclusive are marked Reserved, while the value of Phase Offset (15...0) takes up bits 15 to 0 inclusive.

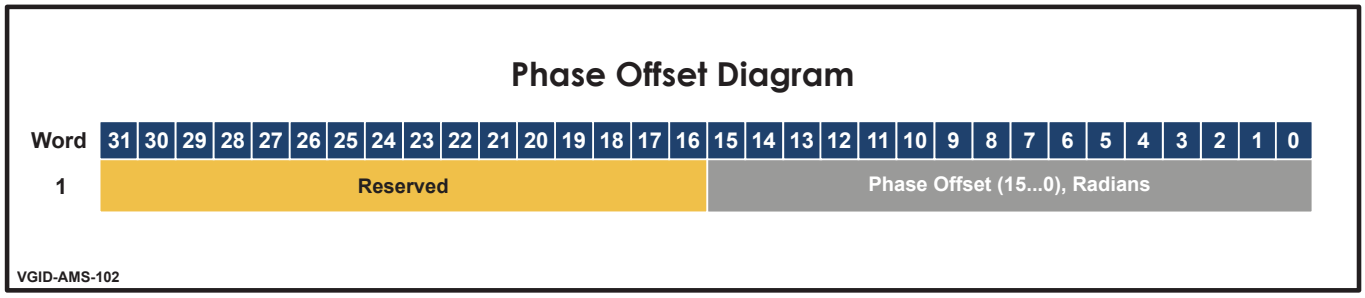


Figure 3.1.10: This Diagram Shows Phase Offset as a Single 32-bit Word. Bits 31 to 16 Inclusive Are Reserved While Bits 15 to 0 Inclusive Define the Phase Offset in Units of Radians.

Field Name	No. of Words	Packet Byte Number	C++ Type
`phaseOffset`	1	64	int64_t

Table 3.1.29: This Table Displays That the Phase Offset is Defined by a 32-bit Word.

3.1.2.3.13 Pointing Vector Index

`pointingVectorIndex`: The Pointing Vector Index shall be a single 32-bit word.

The Pointing Vector Index shall define the reference frame mode of the VA and is defined as the following: ECEF = 0, LLA = 1, platform relative = 2, face relative = 3, baseline relative = 4.

Field Name	No. of Words	Packet Byte Number	C++ Type
`pointingVectorIndex`	1	68	enum

Table 3.1.30: This Table Displays That the Pointing Vector Index is Defined by a 32-bit Word.

3.1.2.3.14 Polarization

`polarization`: Polarization shall be two 32-bit words.

The Polarization shall define the polarization mode implemented on the VA for this job sequence.

The Polarization shall be defined as default, single-pol, or dual-pol dependent on the capabilities of the MFA.

Field Name	No. of Words	Packet Byte Number	C++ Type
`polarization`	8	72	stokesvector(array<double,4>)

Table 3.1.31: This Table Displays That the Polarization is Defined by Two 32-bit Words.

3.1.2.3.15 Number of Samples

`numSamples`: Number of Samples shall be a single 32-bit word.

The Number of Samples shall define the number of samples in the data payload.

Field Name	No. of Words	Packet Byte Number	C++ Type
`numSamples`	1	104	uint32_t

Table 3.1.32: This Table Displays That the Number of Samples is Defined by a 32-bit Word.

3.1.2.3.16 Payload Size

`payloadSize`: Payload Size shall be a single 32-bit word.

The Payload Size shall define the number of bytes in the associated payload.

Field Name	No. of Words	Packet Byte Number	C++ Type
`payloadSize`	1	108	uint32_t

Table 3.1.33: This Table Displays That the Payload Size is Defined by a 32-bit Word.

3.1.2.3.17 Trailer Size

`trailerSize`: Trailer Size shall be a single 32-bit word.

The Trailer Size shall define the size of the trailer data payload in bytes.

Field Name	No. of Words	Packet Byte Number	C++ Type
`trailerSize`	1	112	uint32_t

Table 3.1.34: This Table Displays That the Trailer Size is Defined by a 32-bit Word.

3.1.2.3.18 User Defined Data

`userDefinedData`: User Defined Data shall be a single 32-bit word.

The User Defined Data shall define optional context information that can be used between any control service to any processing service that correlates to this event. For example, if a state value needs to be passed to a processing service for further information on the modulation type of the signal, a 35 could be entered on the command side for Frequency Modulation. The processing service would then receive a 35 and associate that with Frequency Modulation.

Field Name	No. of Words	Packet Byte Number	C++ Type
`userDefinedData`	1	116	uint32_t

Table 3.1.35: This Table Displays That the User Defined Data is Defined by a 32-bit Word.

3.1.2.4 Data Information

The Data Information is the payload packets of digitized signal data packaged according to the Job Event header with a maximum size of 2^{30} 32-bit words within the Jobs-based Frame Structure.

3.1.2.5 Trailer Information

The Trailer Information is an optional with a maximum size of 2^{30} 32-bit words within the Jobs-based Frame Structure.

3.2 VITA Messaging

The VITA standard provides an existing message structure that allows for commonality across the frequency spectrum. Whereas services use the RF MEL API to command and control MFAs, MFAs use AMS GRA-tailored VITA 49.2 packets to provide alternative control and output data (e.g., I/Q or PDWs) to the services. See **Chapter 2** for more details on the RF MEL API.

3.2.1 VITA 49.1 Frame Structure

While the AMS GRA uses the VITA 49.2 standard, the VITA 49.1 standard serves as a frame structure for the VITA 49.2 messages.

VITA 49.1 is a lightweight framing protocol. The primary benefit of VITA 49.1 is the addition of a Frame Alignment Word (FAW). The FAW allows for the linking of multiple packets. The framing capability is the ability of a Skill to identify frame boundaries in a VITA Radio Link Layer (VRL) stream based solely upon the FAW. The framing capability allows for the quick recognition of the start of a new group of packets. The most common usage of the FAW is to group Signal Data Packets. This predetermined word can be easily detected within data streams to allow any parsing software to regain synchronization if it is ever lost.

3.2.2 VITA 49.2 Packet Types

Per the VITA 49.2 standard, there are two different types of packet type formats, standard formats and custom formats for each of the three types of support contents for standard formats:

- **Data** – Data is captured within the Signal Data Packet which is used to convey a digitized signal.
- **Context** – Context is captured within a Context Packet which is used to convey a common format for Signal Data.
- **Command** – Command is captured within Command Packets which is used to convey the control mechanism

used to set the context field types.

Custom formats for each of the three types of contents are enumerated as follows: Data captured in the Extension Data Packet, Context captured within the Extension Context Packet, and Command captured within the Extension Command Packet.

Within the AMS GRA, a Context Packet is sent when Rx data is sent from the MFA to the MFP. The Context Packet provides context for the Signal Data Packets received for a particular job request. The MFP uses the Context Packet as it processes the data payload from the Signal Data Packet. Signal Data Packets are generated by the MFA in response to a Job or Control Packet.

All VITA 49.x packets are limited to a maximum size of 65,536 32 bit words. All VITA 49.2 packets have the same seven-word header. The first two words are mandatory for all packets and the next five words are optional to the developer. Following the header words the information contained is packet specific. The Signal Data packet contains a payload representing the digital signal. The Context and Command packets contain information specific to the job that is being executed, and each word in the packet is designed to carry specific information. Optionally there is also the ability to add a trailer word.

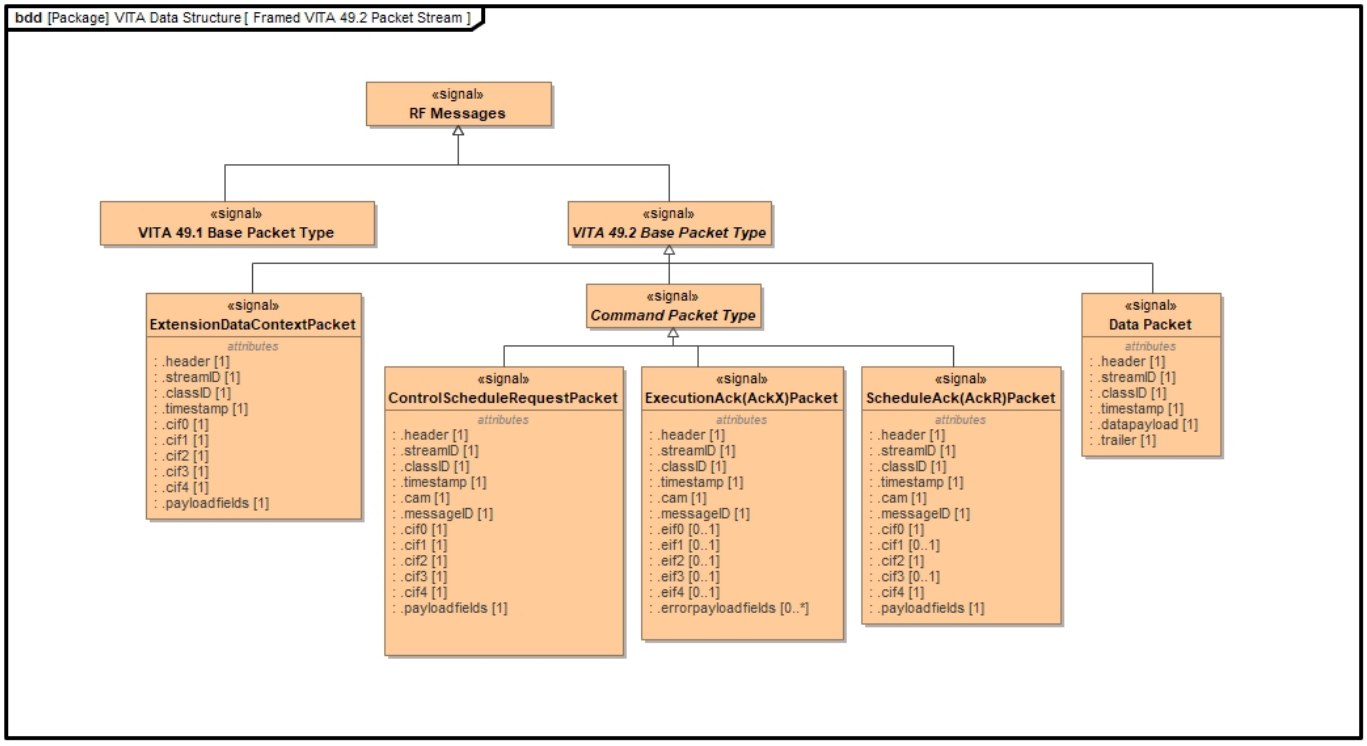


Figure 3.2.1: This Block Definition Diagram Shows the VITA Data Structure for the Planned VITA 49.2 Data Stream.

3.2.3 Customized VITA 49 Standards

VITA 49.1 and 49.2 provide frameworks for programs to create a complete protocol definition for a particular application. The AMS GRA uses this approach to define the packetized datastream portion of the RF MEL definition. The RF MEL definition for data transport selects fields from the VITA 49.2 framework and identifies these as either required, MFA provider defined, or not supported. With this guidance, MFA providers can fully define and document open data transports that are largely common and easily integrated by services in the MFP layer. The current selection of required fields addresses the signal data packets and the context packets as described in VITA 49.2, as well as an additional three words for wrapping the VITA 49.2 standard inside VITA 49.1 for frame alignment. This word arrangement, bit number, bit range, and the specific selected fields are captured in the AMS GRA Model, under Interfaces. These packets are shown at a high level in **Figure 3.2.1**. Note that each signal and context data packet have a header and body section. The .dataPayload itself can be found in the body of the signal data packet.

3.2.4 Constraining Direct Endpoint Access: Allowable Control and Data Packets

DEA is a feature provided by the RF MEL in which a service can designate an ABB endpoint for direct access to the MFA. Set up, initialization, scheduling, status reporting, and other metadata are performed through calls to the RF MEL API. An endpoint designated for DEA may interact with the MFA using a limited collection of VITA 49.2 Control messages. For more information about DEA, see **Section 4.9**. An RF MFA supporting DEA exchanges established through the use of its RF MEL implementation uses the `ControlScheduleRequest`, `ExecutionAck`, and `ScheduleAck` Command Packet types shown at a high level in **Figure 3.2.1**.

The Excel spreadsheet *Common RF MEL VITA 49.2 Control Message Set and Tailoring.xlsx* is delivered with the Common RF MEL repo in the SK as a reference and details the usage of individual fields in control related VITA 49.2 packets used for communication between the service and the MFA by utilizing DEA. These details include per-field definitions of whether fields are required, optional, or unused; along with information regarding when the values are determined. Per-field optionality bounds the control degrees-of-freedom available to a service through DEA, which lessens the burden on the MFA for performing tasks such as validation and safety checking on the real-time packet stream.

The fields explicitly identified in the spreadsheet are for the packet prologue and Context Indicator Field (CIF) sections. The CIF section consists of flags that indicate whether information associated with those flags is present in the Context fields. There are two columns indicating when the value of a field is determined: Time Field Determined and Time Payload Value Determined, along with a Notes column which provides additional information.

The column Time Field Determined relates to when the value of the field explicitly called out in the spreadsheet is determined. For example, the `isAck` field is fixed, because this field simply indicates whether the packet is an acknowledge packet. The `bw` field is fixed per VA definition because each VA has a known bandwidth. The `packetCount` field is Dynamic because it is a unique 4-bit packet sequence number.

The column Time Payload Value Determined only applies to CIF sections and applies to the contents of the payload section. Generally the values for indicator fields are fixed early in the process, while the associated values in the payload section are dynamic. If the associated values in the payload were not dynamic, there would be no need to include them. That is because, in this case, the payload values would not change and could therefore be included in the VA definition, set during VA initialization, or set through some outer loop control process accessed through API calls. For example, the `bw` field's payload is dynamic if allowed per VA definition since the VA definition would define the degrees of freedom and amount of adjustment allowed in the bandwidth control from packet to packet or disallow `bw` adjustment entirely. The payload associated with the `rfRefFreqOffset` field is dynamic within limits of VA definition because the VA may restrict the valid set of values for the RF Reference Frequency Offset.

Fixing the values of the indicator fields at VA definition or earlier ensures that the size and layout of Extension Control Packets is fixed at runtime. This static behavior enables efficient processing in firmware.

All dynamic fields should be assumed to be changeable from one Extension Control Packet to the next for a given VA unless MFA documentation explicitly states otherwise.

The maximum control rates, minimum lead times for the service issuing Extension Control Packets, and maximum lead times for issuing Extension Control Packets should be captured in MFA and/or VA documentation. These times are highly application dependent and implementation dependent.

4.0 Operations Enabled by the RF MEL

The following use cases reflect the expected behaviors of the Skill and MFA interaction.

1. Load RF MEL
2. Get MFA Characteristics at Design Time
3. Get MFA Characteristics at Run Time
4. Request VA
5. Create Data Transfer Endpoint
6. Associate Endpoints
7. Submit Job Request
8. Add Job Interval
9. Enable DEA
10. Get Health and Status
11. Local Function Control
12. Inhibit VA
13. Request RF Activity Notifications

4.1 Load RF MEL

Description

This use case, shown in **Figure 4.1.1**, captures how a service loads the RF MEL to interact with the MFA.

Preconditions

Before the RF MEL can be loaded to interact with the MFA, the following conditions must be true:

- The service has been successfully deployed and is running on an ABB accessible Digital Payload Processor (DPP).
- The service has the RF MEL library filename and configuration string.
- The service is in a state which supports interaction with the RF MEL.

Post-Conditions

After the RF MEL has been loaded to interact, the following conditions should be true:

- The Load RF MEL can request a VA.
- Upon failure to load the RF MEL, the service is unable to interact with the RF MEL.

Assumptions

The following function and condition is assumed true for the RF MEL to be properly loaded:

- The library and configuration file are accessible by the service. If the service resides in a container, the library and configuration file are in the container with the service or are mounted into the container.

Actors

The actors in the Load RF MEL use case are:

- The service
- The `DynamicLoader`

Flow of Events

Figure 4.1.1 shows the RF MEL being loaded by the service:

1. The service uses the `DynamicLoader` to instantiate the RF MEL.
2. The `DynamicLoader` returns the instance of the RF MEL class.

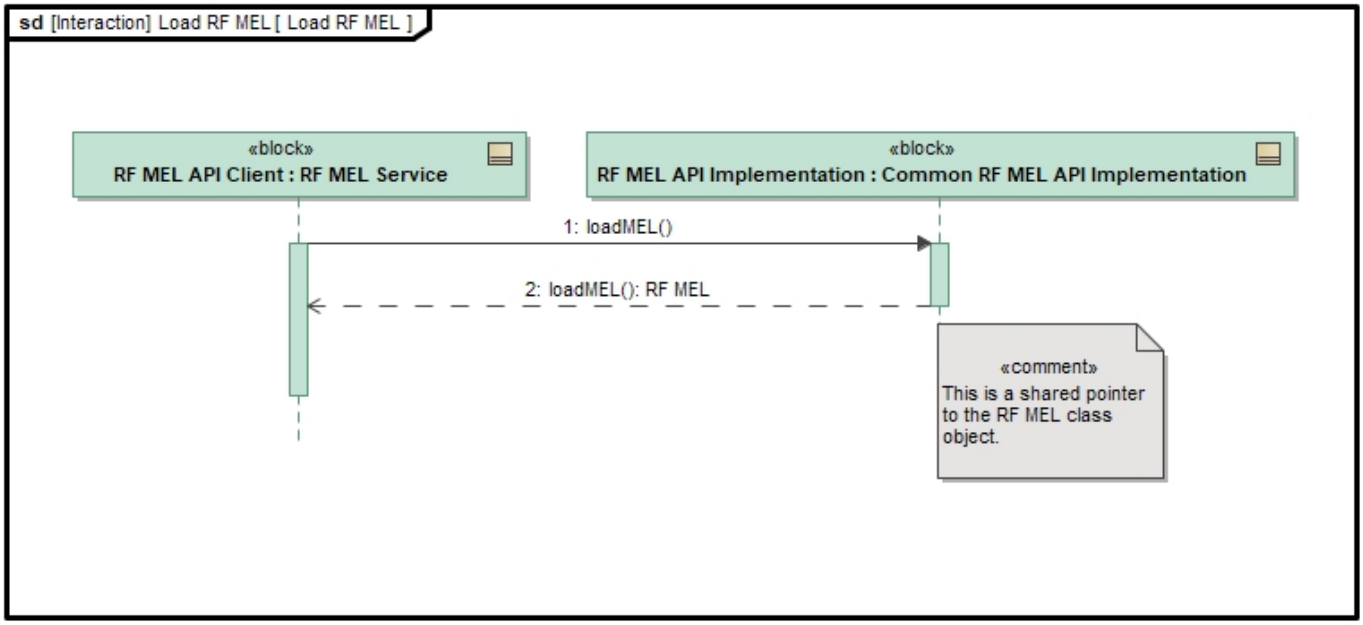


Figure 4.1.1: The Service Loads the RF MEL API to Interact With the MFA.

4.2 Get MFA Characteristics at Design Time via VADB

Description

This use case, shown in **Figure 4.2.1**, captures how a service provider can learn information about the MFA at service design and integration time using the VADB. Every MFA provider provides a VADB with their AMS GRA compliant MFA. While the AMS GRA prescribes the VADB API which is provided in the SK, the actual implementation of the VADB is MFA specific. Therefore, the VADB Client actor in this use case may take different forms for each VADB.

Preconditions

Before the characteristics of the MFA can be pulled from the VADB, the following condition must be true:

- The service provider has locally loaded the VADB for the MFA of interest.

Post-Conditions

After the MFA characteristics have been loaded through the VADB, the following conditions should be true:

- The service provider has been able to evaluate whether predefined VAs satisfy the service provider's needs.
- The service knows which predefined VAs it will use at runtime.
- The service provider will communicate with the MFA provider to work towards a solution.

Assumptions

The following function and condition is assumed true for the MFA characteristics to be properly pulled from the VADB:

- The VADB for the MFA of interest has been provided to the service provider.

Actors

The actors in the Get MFA Characteristics at Design Time via VADB use case are:

- The VADB Client
- The VADB

Flow of Events

Figure 4.2.1 shows the MFA characteristics being loaded by the service through the VADB:

1. The VADB Client loads the `VirtualApertureDatabase`.
2. The VADB returns a pointer to the `VirtualApertureDatabase`.
3. The VADB Client loads the VA definitions.
4. The VADB Client calls `getAvailableVADefinitionIDs`.
5. The VADB returns the IDs of available VAs.
6. The VADB Client calls `get` on a `VirtualApertureDefinitionID` of interest.
7. The VADB returns the `VirtualApertureDefinition` associated with that `VirtualApertureDefinitionID`.
8. Once the Client has this `VirtualApertureDefinition`, the Client can call many functions on the `VirtualApertureDefinition` in order to get more information about the VA. In this example, we will show the Client getting more information about the available instances of this VA. To do this, the Client calls `getNumInstances`.
9. The VADB returns the number of Instances associated with that VA.

10. Now that the Client knows how many instances there are, the Client can query for particular information about each instance. The VADB Client calls `getInstanceDescriptor` on a `VirtualApertureInstanceID` of interest.
11. The VADB returns the `VirtualApertureInstanceDescriptor` associated with that `VirtualApertureInstance`. Once the Client has this `VirtualApertureInstanceDescriptor`, the client can call many functions on the `VirtualApertureInstanceDescriptor`. For available functions, see the Doxygen files located in the SK in interfaces/mel/common_rf_mel.

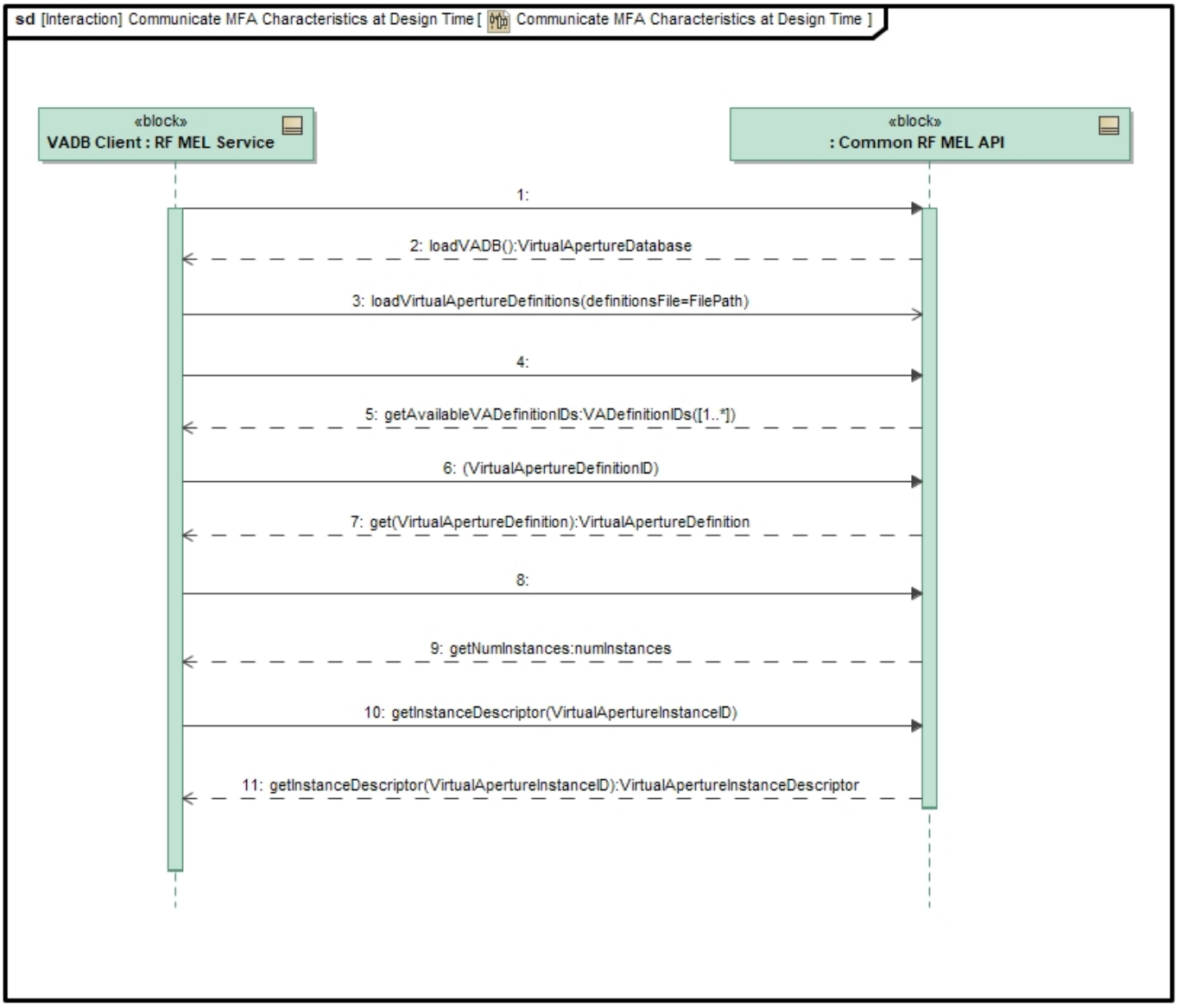


Figure 4.2.1: At Design Time, the Service Retrieves MFA Characteristics Using the VADB.

4.3 Get MFA Characteristics At Run Time

Description

This use case, shown in **Figure 4.3.1**, captures how a service provider can learn information about the MFA at run time.

Preconditions

Before the MFA characteristics can be pulled during run time, the following conditions must be true:

- The service has successfully completed Load RF MEL in **Section 4.1**.
- Before step 9 in this use case's Flow of Events, the service has successfully completed Request VA in **Section 4.4**.

Post-Conditions

After the provider has learned about the MFA characteristics at run time, the following condition should be true:

- The service has information about the MFA and MFA Faces.

Assumptions

- There are no further assumptions to execute the Get MFA Characteristics At Run Time use case.

Actors

The actors in the Get MFA Characteristics At Run Time use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.3.1 shows the MFA characteristics being received during run time:

1. The service calls `getRFMFAInfo`.
2. The MEL returns the `RFMFAInfo`.

3. Now that the service has the `getRFMFAInfo` object, there are many functions that the service can query to get information about the MFA. In this example, we will focus on the MFA Faces. For available functions, see the Doxygen files located in the SK in interfaces/mel/common_rf_mel. The service calls `getNumFaces`.
4. The MEL returns the number of Faces for this MFA.
5. The service calls `getFaceIDs`.
6. The MEL returns the set of Face IDs for this MFA.
7. The service calls `getPhysicalData` for a `FaceID` of interest.
8. The MEL returns the `PhysicalData` for this Face.
9. The service calls `VirtualAperture.getX` where `getX` stands for any of the available functions provided in the `VirtualApertureClass`. For available functions, see the Doxygen files located in the SK in interfaces/mel/common_rf_mel. Functions of interest may include:
    - `getTxRadiatedPower`
    - `getTxPeakRadiatedPower`
    - `getTxApertureGain`
    - `getMaxTxAttenuation`

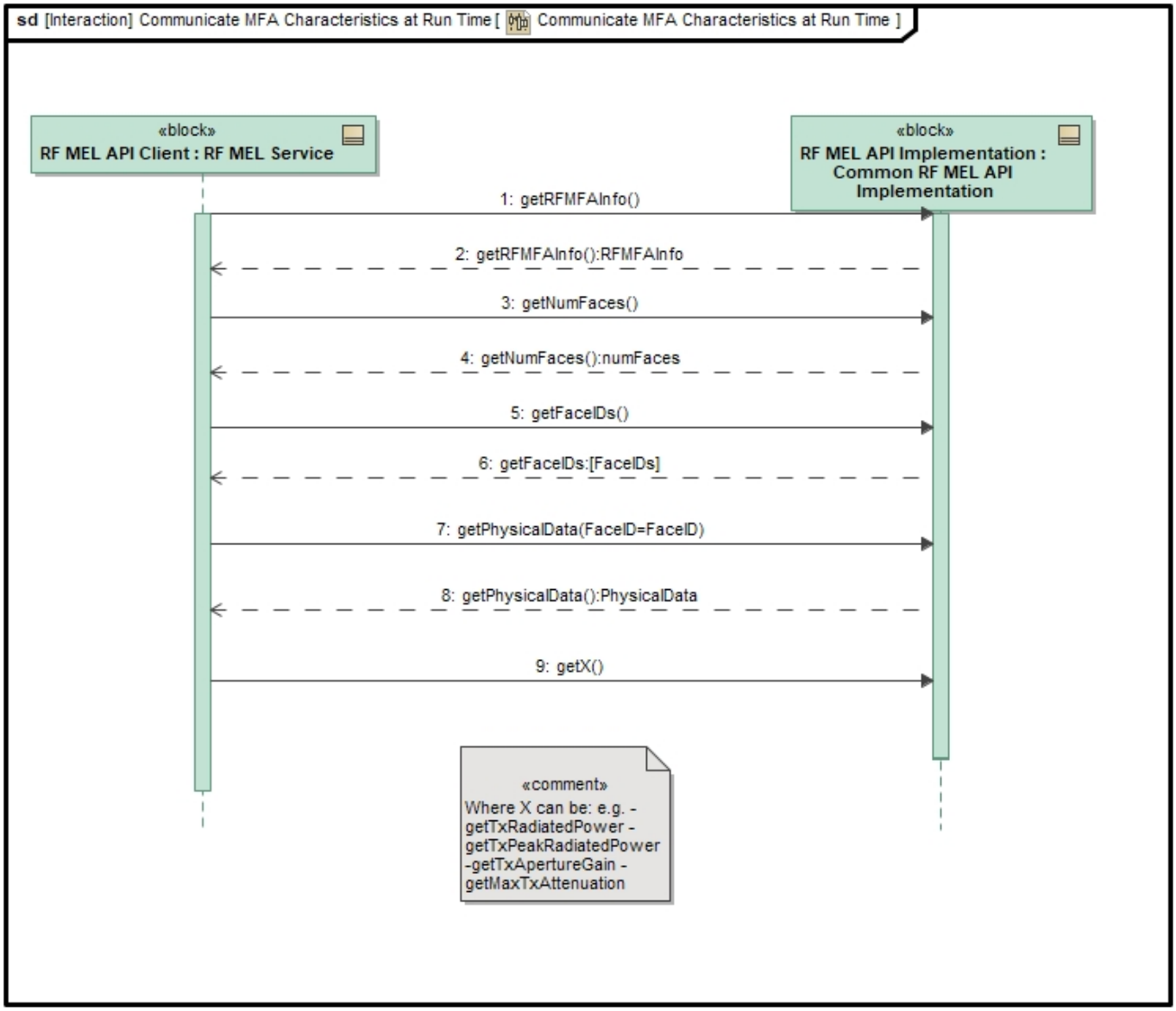


Figure 4.3.1: The Service Retrieves MFA Characteristics at Run Time.

4.4 Request VA

Description

The sequence diagram, shown in **Figure 4.4.1**, captures how a service requests a handle to a VA using the RF MEL. This process must be performed once per constituent process of the service (or larger Skill) that is interacting with the MFA using consistent parameters such that the MFA knows that all the handles refer to the same VA. The service can obtain MFA properties through the `getRFMFAInfo()` function. Endpoint creation and VA requests are independent loops that can happen in parallel. Both scenarios are described in this section. The determination of which `VirtualApertureIDs` to use at runtime is made at design and integration time using the VADB. For more information, see **Section 2.5**. The service requests access to a VA using the predetermined `VirtualApertureID`. The request for a VA also includes a priority. The priority is based on the priority that the service received from the MFP layer (e.g., from an OMS Activity command). Upon success, the service registers a status callback function within each process that needs to be notified of VA status and can obtain properties of the VA through the object handle. This interface provides a simple mechanism for the service to obtain knowledge of whether all of the supporting resource elements in the MFA are in a state to respond to tasking. This approach decouples services from detailed MFA implementations, status, and provides a clean interface to obtain key information. Note that RF MEL users employing the DEA feature request a VA through a slightly different set of RF MEL functions. The DEA concept is documented in **Section 4.9** and requesting a DEA enabled VA is addressed in **Section 4.9.2**.

Preconditions

Before the service can request a handle for a VA using the RF MEL, the following conditions must be true:

- The service has been successfully deployed.
- The RF MEL has been successfully initialized.
- The service has a valid instance of the RF MEL.
- The service has the `VirtualApertureDefinitionID` of the VA that it requests.

Post-Conditions

After the service has requested the VA, the following conditions should be true:

- Upon success, the service can submit `JobRequests` to the MFA through the VA and can process VA status updates.

- Upon failure, the service takes appropriate actions based on the type of failure, as defined by the Service Contract.
- Upon failure, the service and MFA continue in an operational state.

Assumptions

- There are no further assumptions to execute the Request VA use case.

Actors

The actors in the Request VA use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.4.1 shows the VA handle being requested by the service:

1. The service requests a VA from the `RFMEL` by calling the `requestVirtualAperture()` function with the `VirtualApertureDefinitionID` and `priority`.
2. The `RFMEL` returns to the service a `RequestFor<VirtualAperture>` object, which is a `future` type.
3. The service gets the response from the `future` type.
4. If the request was successful, the service gets the `VirtualAperture` object.

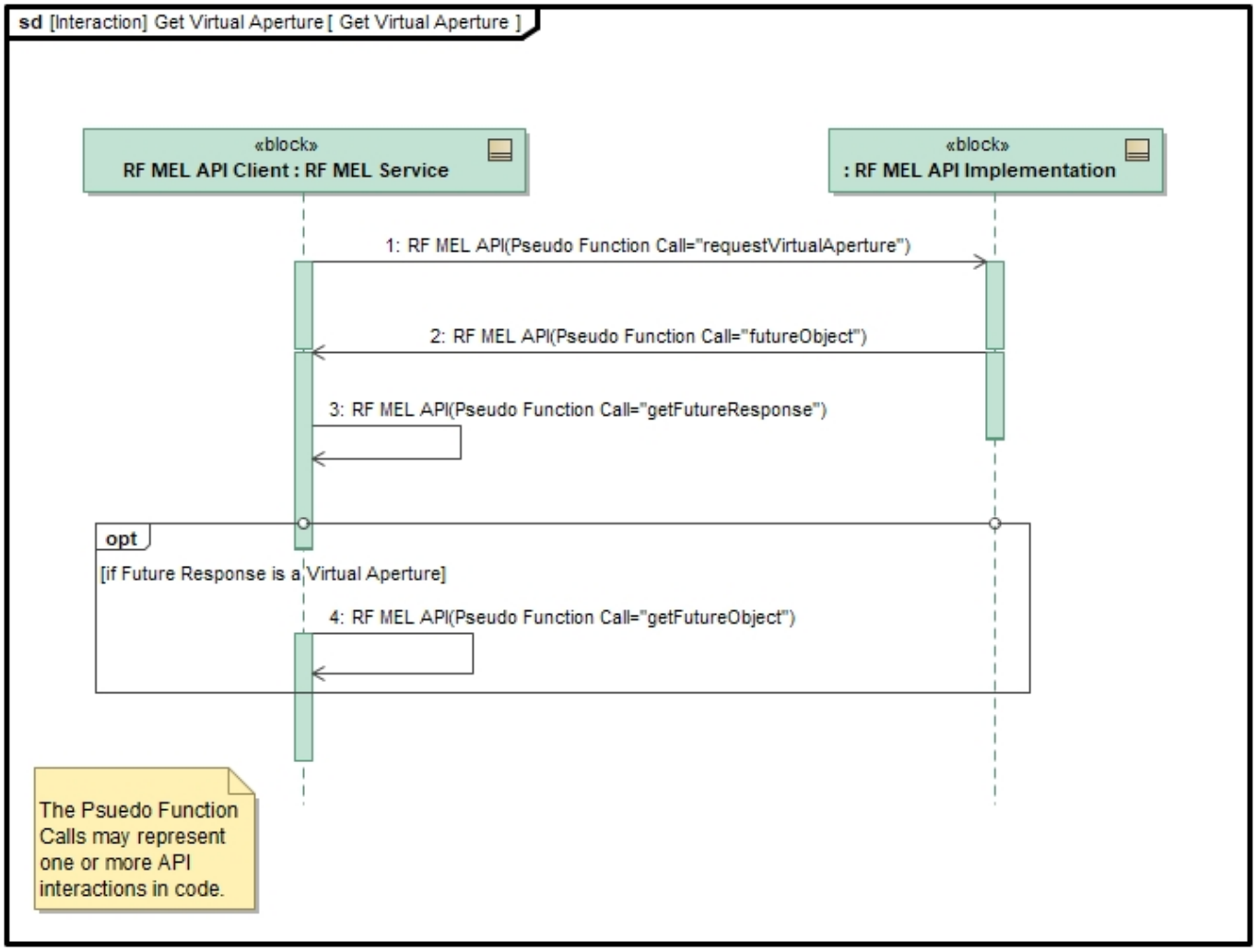


Figure 4.4.1: The Service Requests a Handle for a VA Using the RF MEL.

4.5 Create Data Transfer Endpoint

This use case, shown in **Figure 4.5.1**, captures how a service creates an endpoint for data transfer between the service and MFA. An endpoint is a common abstraction to send and receive data. The endpoint abstraction separates the definition/connection mechanisms and transports (e.g., initialization parameters) from the data transport mechanisms send and receive functions. This operation can happen before or after the Request VA operation.

Although not explicitly discussed in this use case, data transfer endpoints may include those endpoints defined in a VA for data transmitted from a Local Function hosted within an MFA. Similar to any case where a VA contains multiple endpoints, care should be taken to ensure that the service appropriately aligns its endpoints to the endpoints provided in the VA to ensure proper routing of data.

Preconditions

Before the transfer data endpoints can be created, the following conditions must be true:

- The service has been successfully deployed.
- The RF MEL has been loaded successfully.
- The service has obtained valid references to the ABB and MEL.

Post-Conditions

After the creation of the data transfer endpoints between the service and MFA has been attempted, the following conditions should be true:

- Upon success and endpoint association not being required (see **Section 4.6**), endpoint memory can be used to support transmit or receive jobs for buffering I/Q data.
- Alternatively, upon failure, the service takes appropriate actions based on the type of failure, as defined by the service, and may notify the system through `Service_Status` messages.
- Upon failure, the MFA continues in an operational state and services any additional API requests.
- Upon failure, the service and MFA continue in an operational state and performs any additional requests.

Assumptions

The following function and condition are assumed true for the data transfer endpoints to be properly created:

- The I/Q data is packetized per the structures defined in **Chapter 3**.

Actors

The actors in the Create Data Transfer Endpoint use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.5.1 shows the data transfer endpoints being created:

1. If the service wants to allocate the memory, then it allocates the memory region (such as memory mapped peripheral memory) of desired size.
2. The service calls the `createProductRxEndpoint` function with the buffer size and address to the memory region to be used as the endpoint.
3. If the service does not want to preallocate the memory, it can request that an endpoint be created with a specific size from system memory by calling the `createProductRxEndpoint` function without providing the address argument.
4. The service receives a future object.
5. The service attempts to get the response from the future object.
6. Upon successful endpoint creation, the service gets the endpoint object; otherwise, it gets an error.
7. The service registers a data ready callback function to be notified when receive data is available.

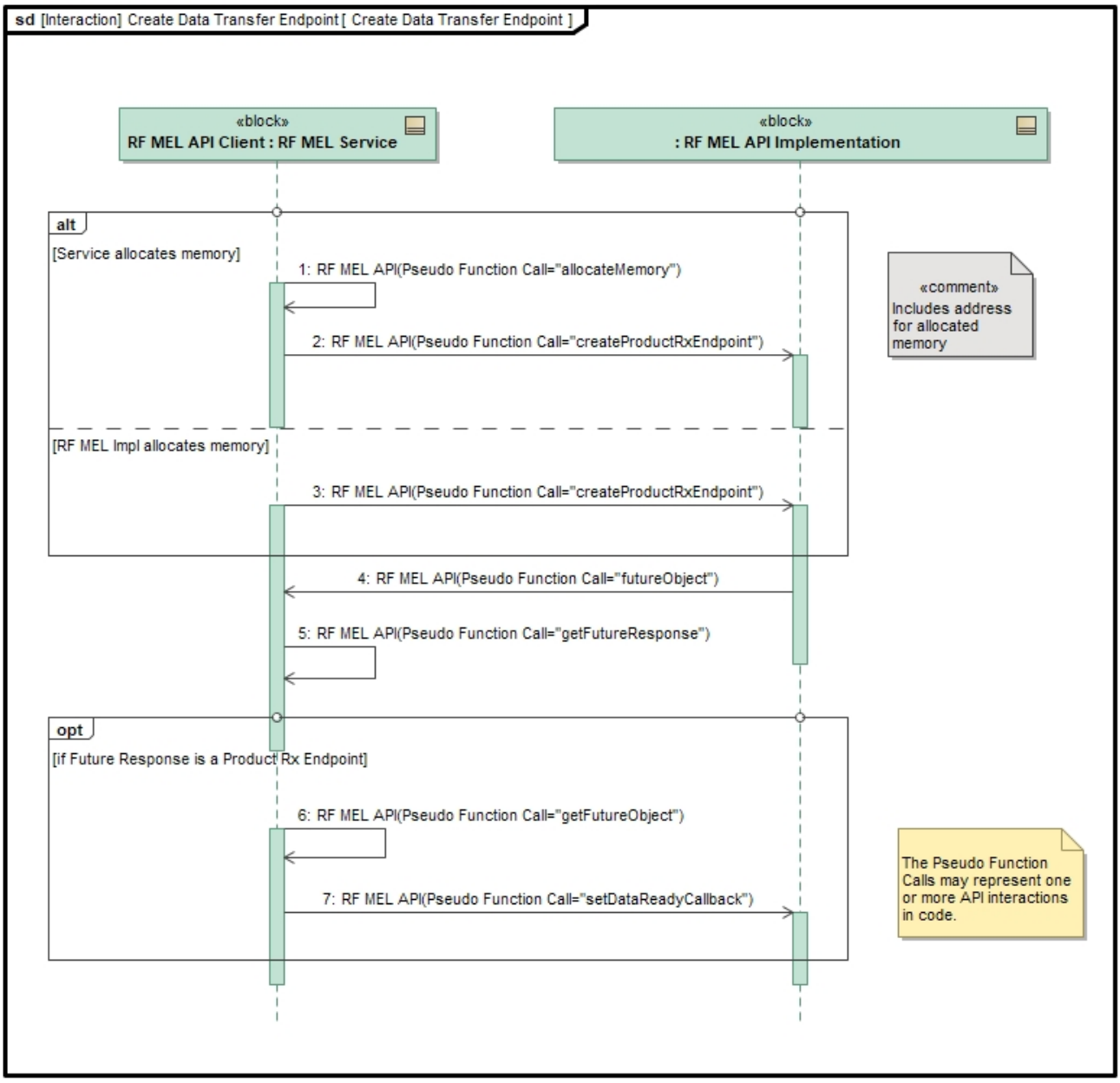


Figure 4.5.1: The Service Creates an Endpoint for Data Transfer Between the Service and the MFA.

4.6 Associate Endpoints

This use case, shown in **Figure 4.6.1**, captures the sequence of interactions between a service and an RF MFA to associate endpoints.

Endpoint association is a feature of MFA's that, if supported, enable early data connections to be established from MFA hardware to service endpoints. These data paths flow between element groups, data pipes, and endpoints. Element groups and data pipes can be identified by a user friendly name through the use of `ElementGroupLabel` and `DataPipeLabel` classes respectively. This operation happens after the Create Data Transfer Endpoint and Request VA operations.

The data pipe in this context is a construct capturing the (possibly multiple) unique hardware connections on which data can be transferred. In scenarios where only one data path is supported by the MFA, endpoint association can be defaulted through the use of the `ams::iface::rfmel::DataPipe::DEFAULT` variable.

Preconditions

Before endpoint association can occur, the following conditions must be true:

- The service is deployed and running on a DPP.
- The service's status supports MFA interactions.
- The service has successfully requested a VA, see Request VA in **Section 4.4**.
- The service has created data transfer endpoints, see Create Data Transfer Endpoint in **Section 4.5**.
- The MFA requires endpoint association.

Post-Conditions

After endpoint association between the service and MFA has been attempted, the following conditions should be true:

- Upon success, endpoint memory can be used to support transmit or receive jobs for buffering I/Q data.
- Alternatively, upon failure, the service takes appropriate actions based on the type of failure, as defined by the service, and may notify the system through `Service_Status` messages.
- Upon failure, the MFA continues in an operational state and services any additional API requests.
- Upon failure, the service and MFA continue in an operational state and services any additional API requests.
- Upon failure, the service and MFA continue in an operational state and performs any additional requests.

Actors

The actors in the Associate Endpoints use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.6.1 shows the endpoint association through the RF MEL:

1. The service queries RF MFA Info: `requiresEndpointAssociation` and verifies it returns true.
2. The service queries the `VirtualAperture` for the available `DataPipe`.
3. The service associates the desired endpoint(s) with the desired `DataPipe`.

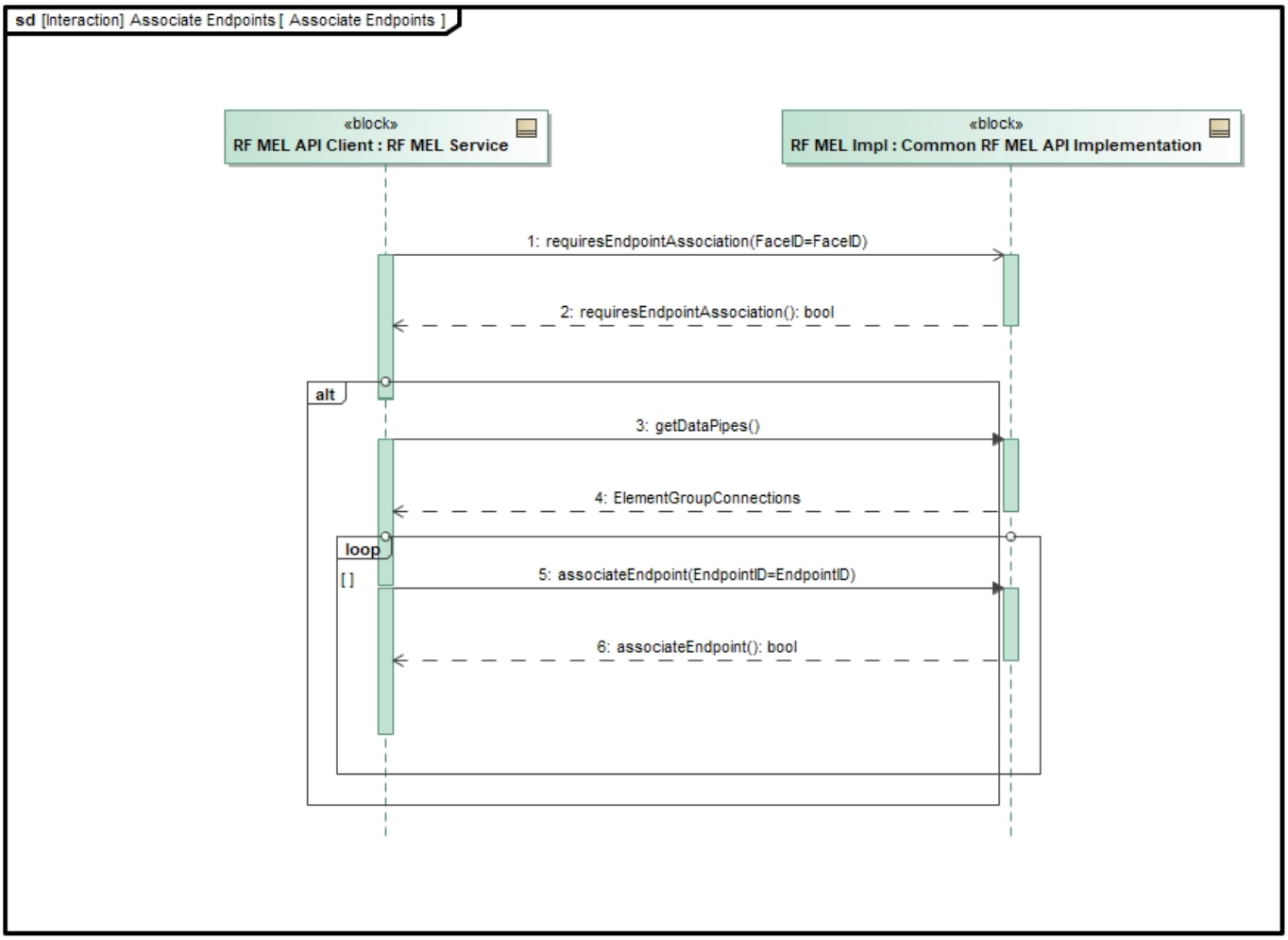


Figure 4.6.1: A Service and an RF MFA Work Together to Associate Endpoints.

4.7 Submit Job Request

Description

This use case, shown in **Figure 4.7.1**, captures the sequence of interactions between a service and an RF MFA, to implement a `JobRequest` through the RF MEL.

Preconditions

Before the `JobRequest` can be sent through the RF MEL, the following conditions must be true:

- The service is deployed and running on a DPP.
- The service's status supports MFA interactions.
- The service has successfully requested a VA, see Request VA in **Section 4.4**.
- The service has successfully associated endpoints if required, see **Section 4.6**.

Post-Conditions

After the `JobRequest` has been sent through the RF MEL, the following condition must be true:

- The service returns to its pre-condition state, ready to interact further with the MFA.

Assumptions

The following functions and conditions are assumed true for a job request to be properly sent through the RF MEL:

- The MFA is not in a degraded state that would prevent or limit any interaction from a service through the RF MEL.
- The service is the only concurrent client for MFA services throughout this use case execution (i.e. this use case does not describe any alternate/exception flows associated with prioritization of requests from multiple clients).
- The service only needs to access a single Element Group in the MFA.
- The primary flow for this use case does not include repetition or recurrence for the `JobRequest`.

Actors

The actors in the Submit Job Request use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.7.1 shows the `JobRequest` being submitted through the RF MEL:

1. The service sets up desired `JobRequest` parameters. Examples of such operations may include:
   (a) The service calls `createWeights()` on its `VirtualAperture` object to retrieve a `Weights` object.
   (b) The service enters data into the `Weights` object In Accordance With (IAW) MFA API Implementation (IMPL) documentation for the specific MFA so that the `Weights` object implements a particular beam taper pattern of interest for the service. *NOTE: The `Weights` object, once defined, can then be applied to multiple `JobEvent` objects without the need to re-define the `Weights` object for each `JobEvent`.*
   (c) The service instantiates a `JobRequest` object.
   (d) The service assigns top-level `JobRequest` attributes (e.g., priority and min_start_time).
   (e) The service adds an `ElementGroupCommand` to the `JobRequest` object.
   (f) The service assigns `ElementGroupCommand` attributes associated with the `JobRequest` (e.g., `center_frequency` and `max_rf_bandwidth`).
   (g) The service assigns the endpoint_id `ElementGroupCommand` attribute to specify endpoint(s) for data transfer.
2. The service calls `requestJob()` on its `virtualAperture` object to request scheduling.
3. The service examines the future object returned from `requestJob()` and determines that the MFA successfully scheduled the `JobRequest`, providing access to a `JobDetail` object.

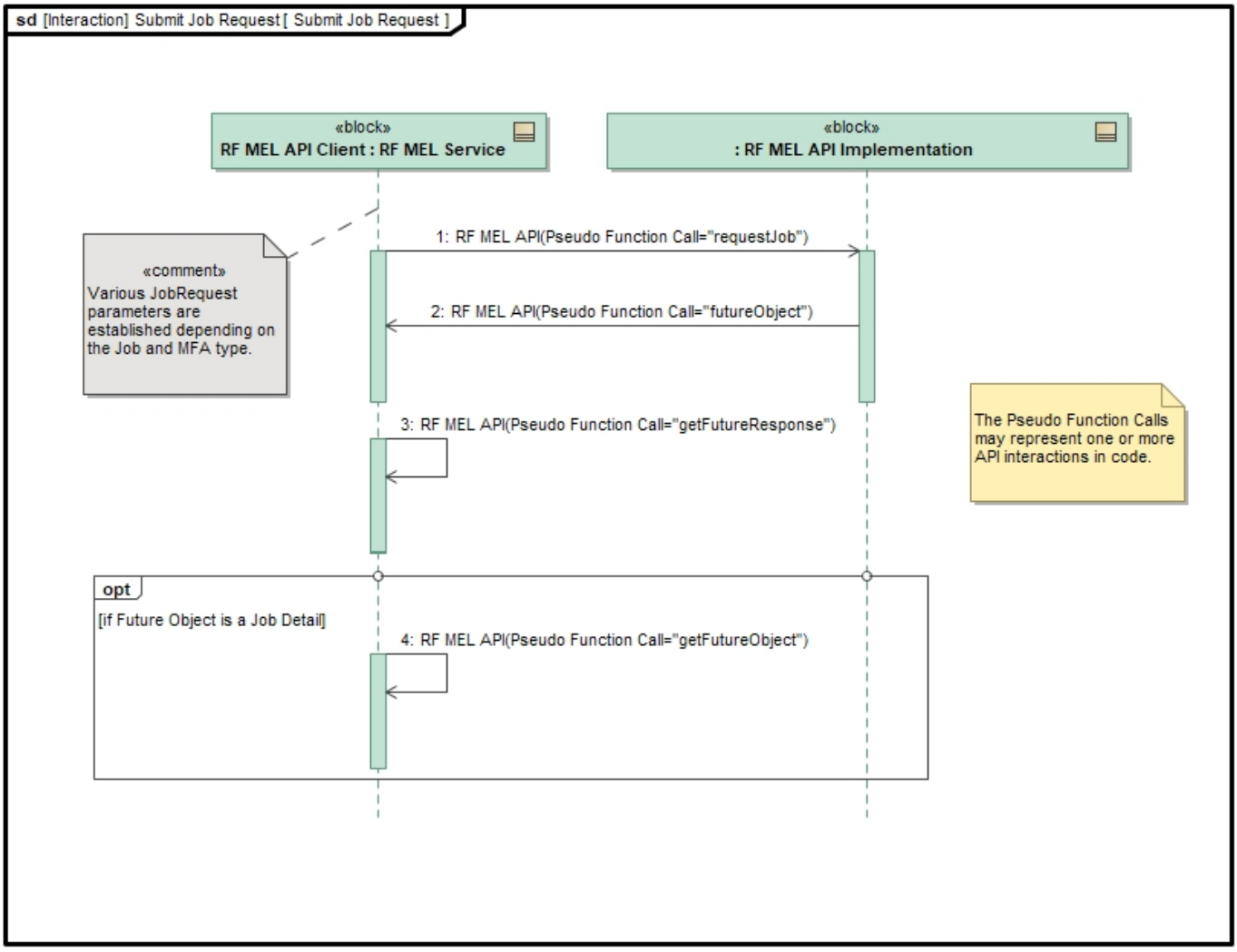


Figure 4.7.1: A Service Schedules a Job Through the RF MEL.

4.8 Add Job Interval

This use case captures how a service adds a `JobInterval`, which can include Tx and/or Rx events, to a job using the RF MEL. The service must have successfully scheduled a job on a VA and obtained a valid `jobDetail` object. A series of contiguous or non-contiguous transmit events can be scheduled by creating a sequence of transmit events. Transmit events include a relative start time, duration, center frequency, direction, power, and additional

optional parameters. Transmit events can take the form of either a parameterized modulated pulse or can be defined by I/Q data from an endpoint. The sequence of transmit events targets one or more Element Groups of the VA and may optionally be scheduled to repeat using parameters. The sequence of events and repeat parameters compose a `JobInterval` which is added to the `JobDetail` object to communicate transmit events to the MFA.

Preconditions

Before the `JobInterval` can be added to a job using the RF MEL, the following conditions must be true:

- The service has been successfully deployed.
- The RF MEL is loaded successfully.
- The service has successfully scheduled a job on a VA. See Request VA in **Section 4.4**.
- The service has successfully associated endpoints if required, see **Section 4.6**.
- The service has successfully created endpoints for transmit I/Q data (if needed).

Post-Conditions

After the `JobInterval` can be added to a job using the RF MEL, the following conditions must be true:

- The service can submit additional `JobRequests` to the MFA by utilizing the VA and can process VA status updates.
- The service takes appropriate actions based on the type of failure, as defined by the service, and may notify the system through `Service_Status` messages.
- Upon failure, the service and MFA continue in an operational state.

Assumptions

The following function and condition is assumed true for the `JobInterval` to be added properly:

- The I/Q data is packetized per the structures defined in **Chapter 3**.

Actors

The actors in the Add Job Interval use case are:

- The service
- The MEL API Implementation

Flow of Events

The following sequence of events below describes the `JobInterval` being added using the RF MEL:

1. The service instantiates one or more `JobInterval` objects.
2. The service adds one or more `JobEvents` (Tx or Rx) to the `JobInterval` object and sets `JobEvent` attributes to desired values.
3. For each `JobEvent` requiring beam tapering, the Client Service sets the `JobEvent's` weights attribute to the tailored `Weights` object created above.
4. For each Tx event requiring pulse modulation, the service sets the Tx event's modulation attribute to specify either streaming I/Q attributes or parameter values associated with amplitude, frequency, or phase modulation.
5. The service adds the `JobInterval` to the `JobDetail` object.
6. The service calls `finalize()` on the `JobDetail` object, which returns a placeholder for the status of the job.
7. The MFA processes the finalized Job for execution IAW timeline description of the `JobInterval` and offsets associated with the `JobEvents`.

- Flow of Events *(Alternate 1)* - Command Pulse Modulation
  - Commanding pulse modulation in Tx events is an alternate flow of the `JobRequest` process described in the optional step in the primary flow, where the actor has the option to specify Tx event modulation parameters as part of populating a `JobInterval` with `JobEvents`. The Tx event definition extends the `JobEvent` base class with the addition of a modulation field of type `WaveformOrModulation`, and the service populates either the waveform parameters to associate the `TransmitEvent` with a `WaveformTxEndpoint` or populates the modulation object with pointers to objects specifying the phase, amplitude, and frequency modulations.

- Flow of Events *(Alternate 2)* - Command Beam Tapering
  - Commanding beam tapering is an alternate flow of the `JobRequest` process described in the optional step in the primary flow, where the actor has the option to customize shape of beam by communicating a collection of beam weight values to the MFA through the use of the weights field of the transmit and/or receive events.
- The RF MEL populates the placeholder for the status of the job, which was returned by `finalize()`, with the job execution result.

Flow of Events for Receive Events

The following sequence of events below describes the `JobInterval` being added using the RF MEL for received events:

1. A series of non-overlapping `ReceiveEvents` are scheduled by setting the sequence `JobInterval` to a `Sequence` of `ReceiveEvents`, each of which is defined by a start time, duration, center frequency, direction, and additional optional parameters.
2. The `Sequence` of receive events apply to one or more Receive Element Groups of the VA and may optionally be scheduled to repeat using `setSequenceRepeatCount`.
3. This sequence of events and repeat parameters compose a `JobInterval` which is added to the `JobDetail` object to communicate receive events to the MFA.
4. Additional `JobIntervals` can be added to the `JobDetail`, typically covering a different time slot of the `JobRequest` or an orthogonal set of element groups of the VA.
5. The destination of receive I/Q data products is defined by the service as a map of Element Group label, endpoint ID, and data pipe.
6. The `ProductStreamParams` that establish the association between end receive endpoints is added to the `JobInterval` by the service.
7. When the final `JobInterval` has been added, the service finalizes the `JobDetail` object and can monitor the progress of the job.

4.9 Direct Endpoint Access (DEA)

MFAs may support DEA, which provides a service the ability to designate an ABB endpoint for direct access to the MFA. The RF MEL defines the common interface to enable this capability. Support for the DEA capability is optional for MFAs.

DEA provides a service the flexibility to use a General Purpose Processor (GPP)-deployed Digital Payload (DP) for API access to the MFA, while designating a different endpoint, such as an FPGA-deployed DP, for higher

throughput or lower latency interactions than the GPP-resident DP can process.

This section details the DEA interface of the RF MEL in a series of sequence diagrams that depict its use by notional service- and coprocessor-based Skill executions.

4.9.1 Overview

The sequence diagram in **Figure 4.9.1** shows the high-level relationship between the remaining sequence diagrams in this section. In particular, the Initialize DEA sequence diagram shows how a service notionally uses the RF MEL to request the VA and then get the list of endpoints with which it can interact. After DEA has been initialized, this first sequence branches: service-based Skills notionally execute per the Service-based Skill Execution sequence diagram, and coprocessor-based Skills notionally execute per the Coprocessor-based Skill Execution sequence diagram.

Preconditions

Before the DEA uses the RF MEL to request the VA and then get the list of endpoints with which it can interact, the following conditions must be true:

- The relevant RF MFA is in a state where it is able to facilitate initialization of DEA through its associated RF MEL.
- The service DP and any DPs intended for DEA are successfully deployed.
- The Skill has access to the RF MEL associated with the RF MFA it intends to interact with.

Post-Conditions

After the DEA uses the RF MEL to request the VA and then get the list of endpoints with which it can interact, the following condition must be true:

- The Skill has executed successfully.

Assumptions

The following function and condition is assumed true for the service to designate an ABB endpoint for direct access to the MFA.

- The service includes its own VITA 49 packetizer capability that is compatible with the underlying network stack.

Actors

The actors in the DEA use case are:

- The service
- The MEL API Implementation
- The DP (DP on coprocessor that is part of the same Skill as the service actor).

Flow of Events

Figure 4.9.1 shows DEA using the RF MEL to request the VA and then get the list of endpoints with which it can interact:

1. **Initialize DEA** – This interaction use case references the Initialize DEA sequence shown in **Figure 4.9.2** which shows the process by which a service notionally initializes DEA via the RF MEL.
2. **Service-based Skill Execution** – This interaction use case references the Service-based Skill Execution sequence shown in **Figure 4.9.4** which shows the process by which a service notionally executes direct endpoint command and control and data exchanges through the RF MEL.
3. **Coprocessor-based Skill Execution** – This interaction use case references the Coprocessor-based Skill Execution sequence shown in **Figure 4.9.5** which shows the process by which a DP executing on a coprocessor notionally executes direct endpoint command and control and data exchanges initialized through the RF MEL.

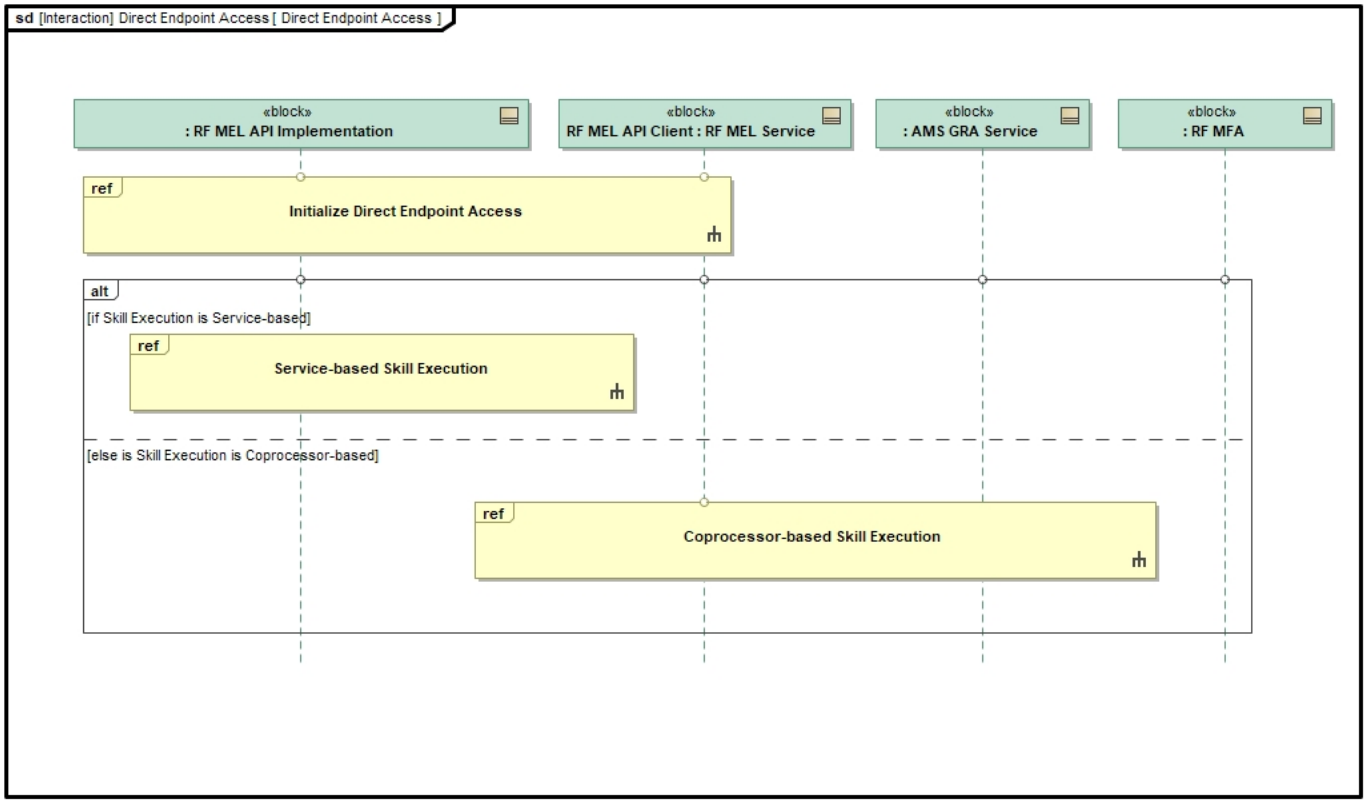


Figure 4.9.1: A Service Requests a VA and Initializes DEA Through the RF MEL. Then, the Skills Begin to Execute.

4.9.2 DEA Initialization

The sequence diagram in **Figure 4.9.2** shows the process by which a service notionally initializes DEA via the RF MEL. In order to reduce the complexity of the diagram, the sequences to request a VA with DEA are further removed to another sequence diagram. The sequences remaining outline the RF MEL operations that the service executes to first obtain the stream ID associated with the VA (sequence 1 and 2), and then get the list of endpoints associated with the VA (sequence 3 and 4). In the event that the RF MFA, the MFA specific RF MEL implementation, and/or the VA do not support these operations, the operations returns an error, namely some indication that the operation is not supported.

The sequence diagram in **Figure 4.9.3** shows the process by which a service requests a VA with DEA via the RF MEL. Sequences 1 and 2 describe the RF MEL operation that the service executes request a VA with DEA.

In the event that DEA is not supported by the RF MFA, the RF MEL implementation, and/or the VA identified by the `vaDefID` input parameter, the operation returns an error, namely some indication that the operation is not supported.

Preconditions

Before the service notionally initializes DEA via the RF MEL, the following condition must be true:

- The RF MEL has been successfully loaded.

Post-Conditions

After the service notionally initializes DEA via the RF MEL, the following condition must be true:

- The Skill has executed successfully.

Assumptions

The following condition is assumed true for the DEA to be properly initialized.

- The MFA supports DEA.

Actors

The actors in the DEA Initialization use case are:

- The service
- The MEL API Implementation

The DP actor shown in other diagrams in this section is not shown in these diagrams because DEA initialization must be executed by a service interacting with the RF MEL.

Flow of Events

Figure 4.9.2 and **Figure 4.9.3** show the DEA being initialized via the RF MEL:

- As the reference box indicates, the Request VA with DEA sequence has completed successfully.
    1. The service requests the stream ID associated with the VA.
    2. The RF MEL responds with the stream ID.

3. The service requests the list of control endpoints.
4. The RF MEL responds with the list of control endpoints. The first control endpoint is the Virtual Aperture Scheduler (VAS).
5. The service requests the list of data endpoints.
6. The RF MEL responds with the list of data endpoints.
   (a) The service registers a function to the `detachEndpointCallback`. If the RF MFA needs to detach any of the endpoints used by the service at any point, the RF MFA alerts the service by providing the `endpointId` of the detached endpoint to the callback. An example of this is shown in **Figure 4.9.6**.
   (b) The service registers a function to the `setExpirationCallback`. As shown in **Figure 4.9.2**, when the service requests a Direct Access VA. If VA access expires, the RF MEL alerts the service by providing the `virtualApertureID` of the expired VA.
   (c) The service may request some information about the expiration of the VA it requested. If desired the service may call `getExpiration`.
   (d) The RF MEL responds with the date and time that the requested VA expires.
   (e) If desired the service may request the time remaining prior to VA expiration.
   (f) The RF MEL responds with the time remaining.

- As the reference box indicates, the Load RF MEL sequence has completed successfully.
  1. The service requests a VA that supports DEA. The details of establishing the endpoints with the MFA are MFA specific, and are implemented by the MFA provider in their RF MEL implementation.
  2. The RF MEL responds with the VA.

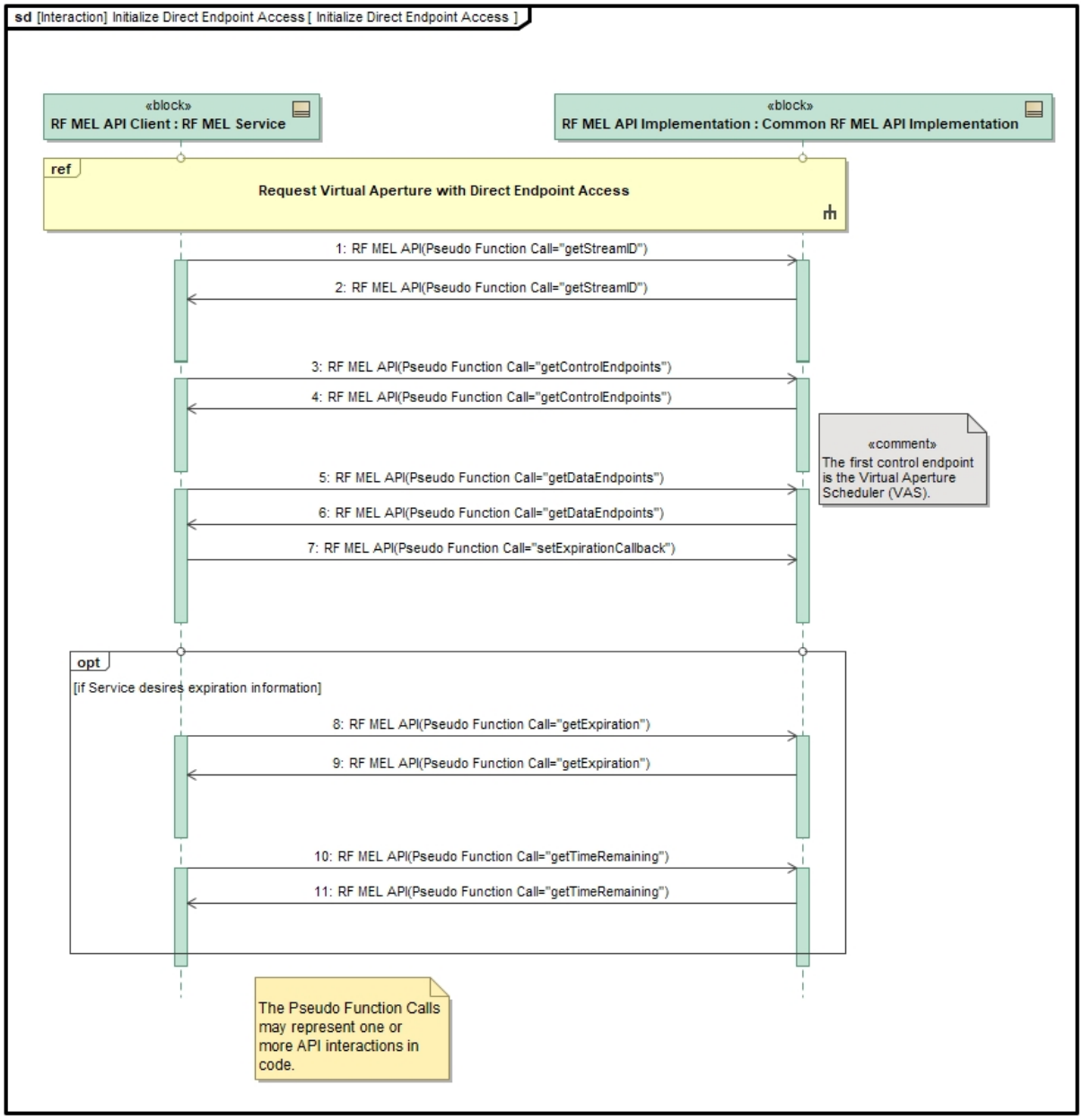


Figure 4.9.2: A Service Uses the RF MEL to Initialize DEA.

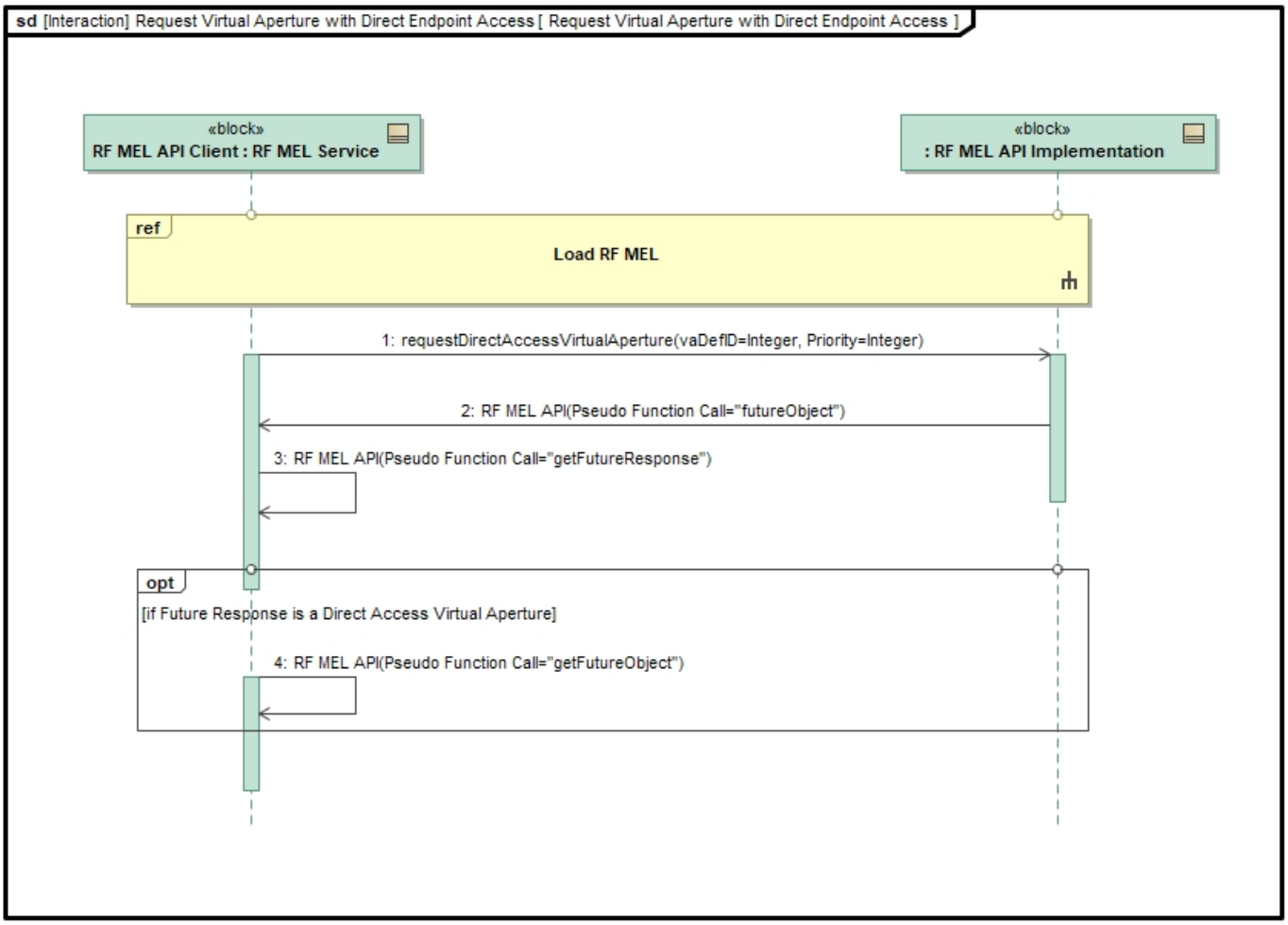


Figure 4.9.3: A Service Requests a VA with DEA.

4.9.3 Service-based Skill Execution

The sequence diagram in **Figure 4.9.4** shows the process by which a service notionally executes direct endpoint command and control and data exchanges via the RF MEL. Sequences 1 and 2 describe the interaction with the list of endpoints and the RF MEL operation that the service executes to send C2 data directly to the endpoint. The first element in the list of control endpoints (i.e., `endpts["ctrl"][0]`) is the VAS. Sequences 3 and 4 describe the interaction with the list of endpoints and the RF MEL operation that the service executes to send data directly to the endpoint. In this diagram, selection of the fifth element in the list of data endpoints (i.e., `endpts["data"][4]`)

is notional. Not shown, endpoints that provide a mechanism for the service to register a callback operation to be invoked when it has received data from the endpoint. In the event that an endpoint does not support the `send()` operation, invocations of this operation should return an error, namely one that indicates the operation is not supported.

Preconditions

Before the service notionally executes direct endpoint command and control and data exchanges via the RF MEL, the following condition must be true:

- The DEA Initialization use case completed successfully, see **Section 4.9.2**.

Post-Conditions

After the service notionally executes direct endpoint command and control and data exchanges via the RF MEL, the following conditions must be true:

- The Skill has executed successfully.

Assumptions

The following functions and conditions are assumed true for the service to notionally execute direct endpoint command and control and data exchanges via the RF MEL:

- The MFA supports DEA.

Actors

The actors in the Service-based Skill Execution use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.9.4 shows the service-based Skill executing:

1. The service sends control packets to the first control endpoint (index 0), which is the VAS. The buffer represents a VITA 49.2 packet. This series of VITA 49.2 exchanges establishes the association between MFA endpoints, stream IDs, and VITA 49.2 citedIDs. The allowed control packets are described in **Section 3.2.4**. Note that the MFA can Acknowledgement (ACK) or Negative-Acknowledgement (NACK) requested events as is standard with VITA 49.2 exchanges.
2. The RF MEL responds as appropriate.
3. The service sends data packets to the data endpoint. The data endpoint is notional here, but represents the endpoint identified for the activity. The buffer represents a VITA 49.2 packet. The data packets are described in **Section 3.2.3**. The activity is executed by the service directly interacting with the MFA via the ABB API endpoint abstraction.
4. The RF MEL responds as appropriate.
5. The service releases the VA and endpoint when execution is complete.

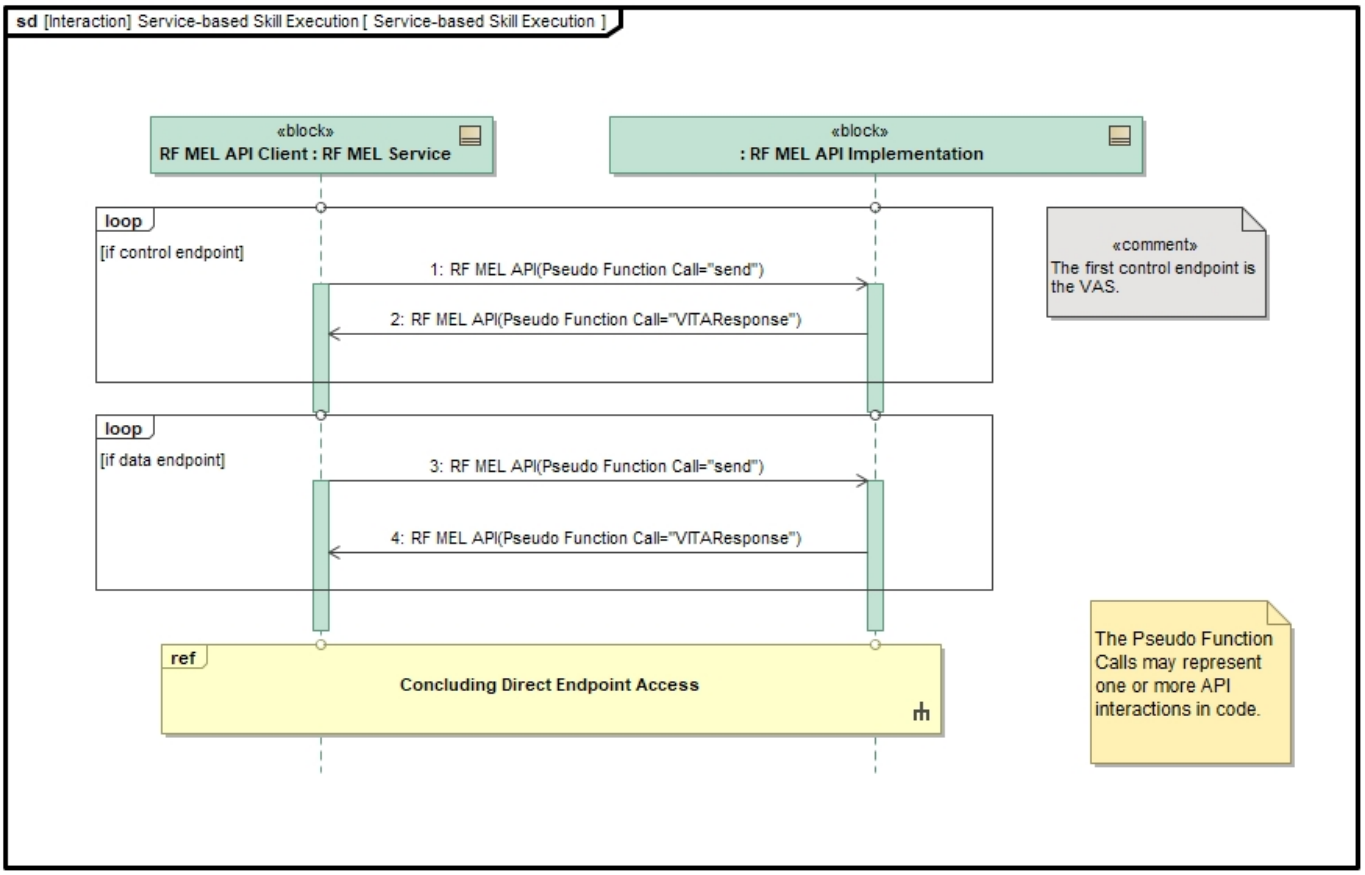


Figure 4.9.4: A Service Uses a VA and an Endpoint to Exchange Data With the Endpoint Via the RF MEL.

4.9.4 Coprocessor Based Skill Execution

The sequence diagram in **Figure 4.9.5** shows one potential example of the process by which a DP executing on a coprocessor executes direct endpoint C2 and data exchanges initialized via the RF MEL. This sequence assumes that all coprocessors are paired with a GPP that runs a service that can initialize the DEA via the RF MEL. **Figure 4.9.5** describes a notional operation, `setupDigitalPayloadLogic()`, that the service uses to pass relevant endpoint information (e.g., Internet Protocol or memory addresses and port numbers) to the coprocessor. At this point in the sequence, the service may use the DEA interface to initialize the Skill execution as described in **Figure 4.9.4**. **Figure 4.9.5** describes another notional operation, `executeSkill()`, that begins execution of the

(remainder of) the Skill by the coprocessor. What follows are the notional exchanges between the coprocessor and the RF MFA via the RF MEL and the information obtained via the DEA interface.

Preconditions

Before the Coprocessor based Skill use case is executed, the following condition must be true:

- The Initialize DEA use case has completed successfully.

Post-Conditions

After the Coprocessor based Skill use case is executed, the following condition must be true:

- The Skill has executed successfully.

Assumptions

The following conditions are assumed true for the Coprocessor based Skill use case to be properly executed:

- All coprocessors are paired with a GPP that runs a service that can broker interactions with the RF MEL.

Actors

The actors in the Coprocessor based Skill Execution use case are:

- The service
- The MEL API Implementation
- The DP (DP on coprocessor)

Flow of Events

Figure 4.9.5 shows the Coprocessor based Skill Execution use case executing:

1. The service provides necessary DP logic.
2. The service triggers execution of the Skill. It is also possible that the service performs some VITA 49.2 message exchanges before handling over execution to the DP.
3. For any desired transmit operations, the DP sends control packets as desired. The allowed control packets are described in the beginning of **Section 4.9**.
4. The MFA responds as appropriate. Note that the MFA can ACK or NACK requested events as is standard with VITA 49.2 exchanges.
5. The DP sends data packets as desired. The data packets are described in **Section 3.2.3**.
6. The MFA responds as appropriate.
7. For any desired receive operations, the DP sends control packets as desired. The allowed control packets are described in the beginning of **Section 4.9**.
8. The MFA responds as appropriate.
9. The MFA sends data packets.
10. The MFA responds as appropriate.

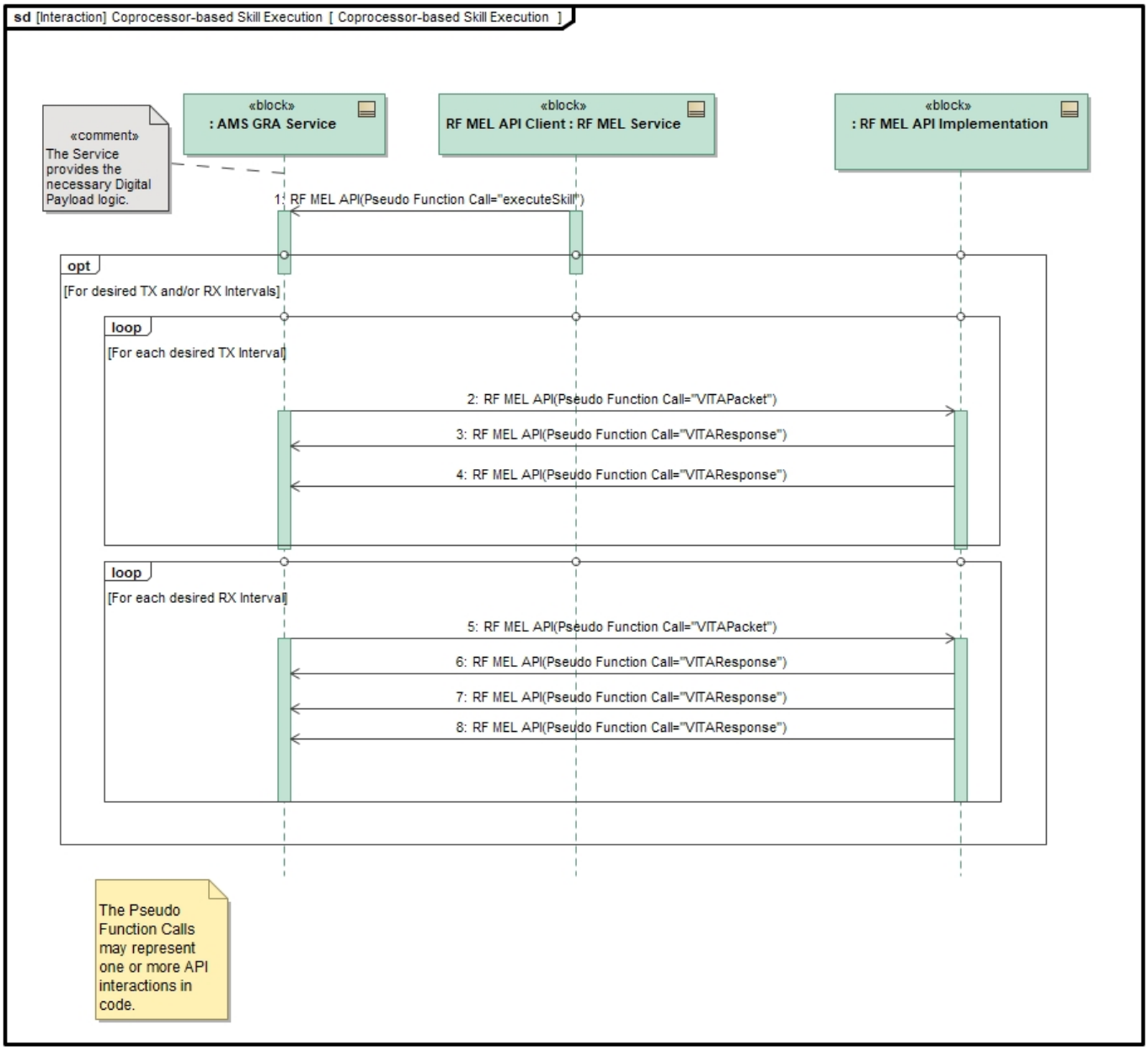


Figure 4.9.5: A DP Runs on a Coprocessor that Communicates With an RF MFA to Execute a Skill.

4.9.5 Concluding DEA

The sequence diagram in **Figure 4.9.6** shows the multiple ways that DEA can be concluded from the service of RF MEL sides.

Preconditions

Before the DEA can be concluded, the following condition must be true:

- The service set up the `detachEndpointCallback` and `setExpirationCallback` DEA was successfully initialized and is executing.

Post-Conditions

After the DEA has concluded, the following condition must be true:

- The MFA is ready to accept tasking.

Assumptions

The following function and condition are assumed true for the DEA to be properly concluded:

- The MFA supports DEA.

Actors

The actors in the Concluding DEA use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 4.9.6 shows the ways the DEA can be concluded from the service of RF MEL sides:

1. If the service is finished using a particular endpoint, it calls reset on the pointer to that endpoint to communicate to the RF MEL that it does not need access to the endpoint anymore.
2. If the service is finished using a particular VA, it calls reset on the pointer to that VA to communicate to the RF MEL that it does not need access to the VA anymore.
3. If an MFA needs to terminate a service's access to a particular endpoint, then the MFA alerts the service by providing a detach message that contains the relevant endpoint ID. As noted in the preconditions, the callback has been set up in initialization.
4. If the DEA enabled VA request has expired, then the MFA alerts the service by providing an expiration message. As noted in the assumptions, this flow assumes the callback has been set up in initialization.
5. In this instance, the service attempts to use an endpoint that has expired through either the MFA terminating it or the duration of the VA request having expired.
6. The service responds with the exception `ResourceUseAfterExpiredException`.

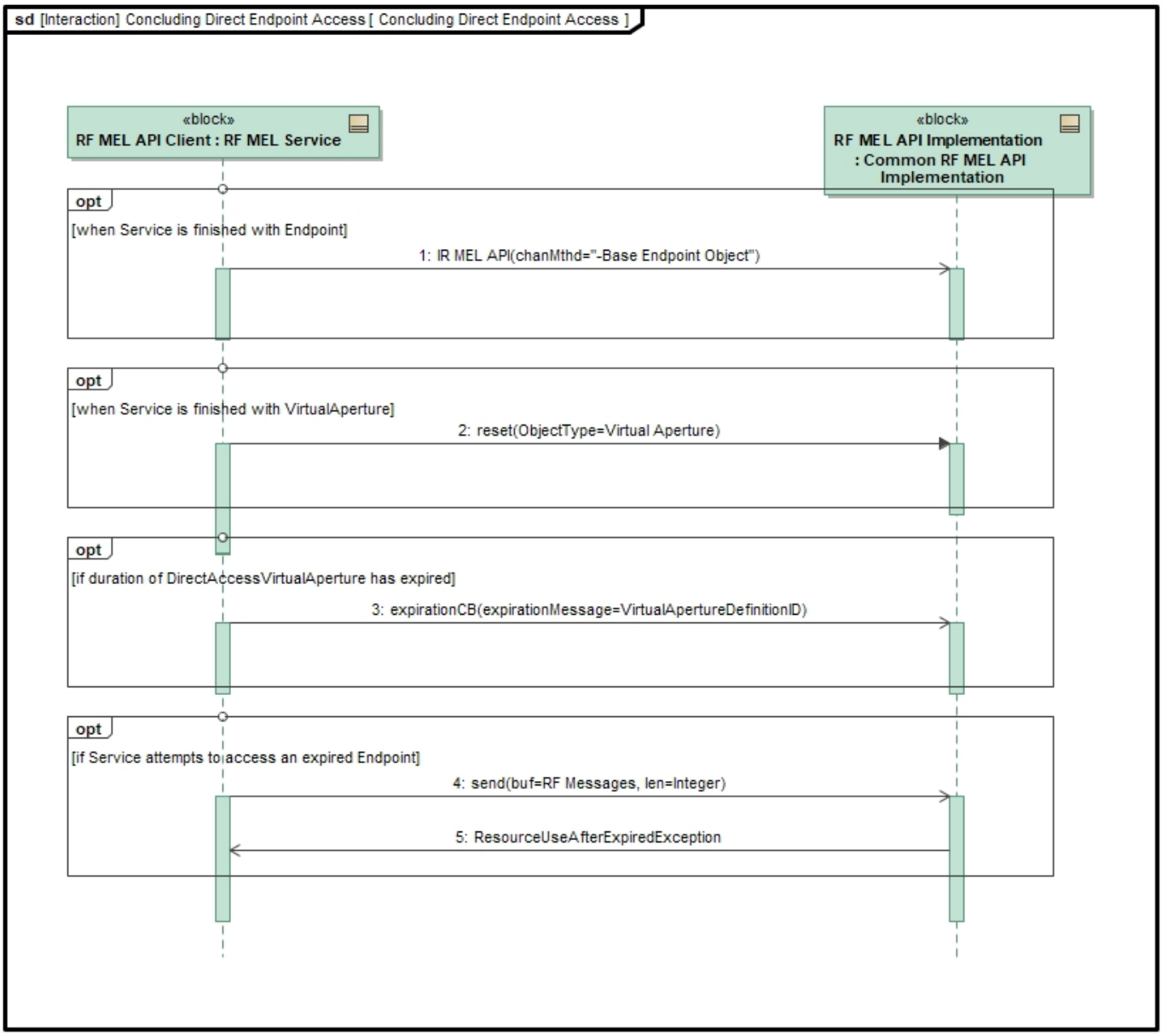


Figure 4.9.6: DEA is Concluded From the Service.

4.10 Get Health and Status

In AMS GRA, an MFA reports its health and status via the MEL API like an OMS Subsystem. The MFA provider can supply their own MFA status service, SK's MEL OMS adapter service, or an MSI make implements a common MFA status service are ways to make MFA status available on the Abstract Service Bus (ASB). They may also make use of the AMS GRA's MEL OMS Adapter service, which translates the message traffic between the ASB and the MFA by means of the Critical Abstraction Layer and IR or RF MEL software interfaces. This adapter pattern serves to isolate the software in each MFA from changes to the OMS Critical Abstraction Layer in the rest of the platform. The API is not isolated from UCI, and future changes in the UCI message structure could require changes in the MEL API to fully represent MFA status on the ASB.

A Service Contract documents the MFA's OMS interactions. Via the MFA status service, the MFA periodically publishes the `AntennaStatus` and `SubsystemStatus` messages. In the case of `SubsystemStatus` an MFA status service receives the status from the MEL in a callback. It also responds to requests for those reports IAW the OMS definition and documentation. Additionally, these reports are published unsolicited whenever their contents change. Similarly, upon detection of a security event, the MFA publishes the `MFA_SecurityAuditRecord` message that contains metadata associated with the security event.[1]The sequences below illustrate these four patterns. Reference the Service Contract for the specific MFA as well as the UCI Schema for definitions of field-level details associated with these messages.

The status of Local Functions inside an MFA is not necessarily reported through the `SubsystemStatus` message. While it is possible to report Local Functions as Subsystem Components, it is typical for each Local Function to only map to a single Skill. Because of this approach, the preferred reporting mechanism is for a service within that Skill to receive VA status and Local Function status via the RF MEL, and update its `CapabilityStatus` on the ASB to reflect any change to its ability to perform its intended tasking.

MFA health and status reporting is shown in 4 diagrams. **Figure 4.10.1** captures the health and status reporting from a `SubsystemStatus` message that can be triggered via an event that causes a change in status, a request for status, or a periodic timer. **Figure 4.10.2** captures the reporting from a `AntennaStatus` message that can be triggered via an event that causes a change in status or a periodic timer. **Figure 4.10.3** captures the reporting from a `MFA_SecurityAuditRecord` message that is triggered by detection of a security event. **Figure 4.10.4** shows the health and status reporting behavior for a Local Function.

[1]While the `MFA_SecurityAuditRecord` is available for implementers to use, it is not required for a given MFA implementation to use or for an MFA implementation to support all of the checks.

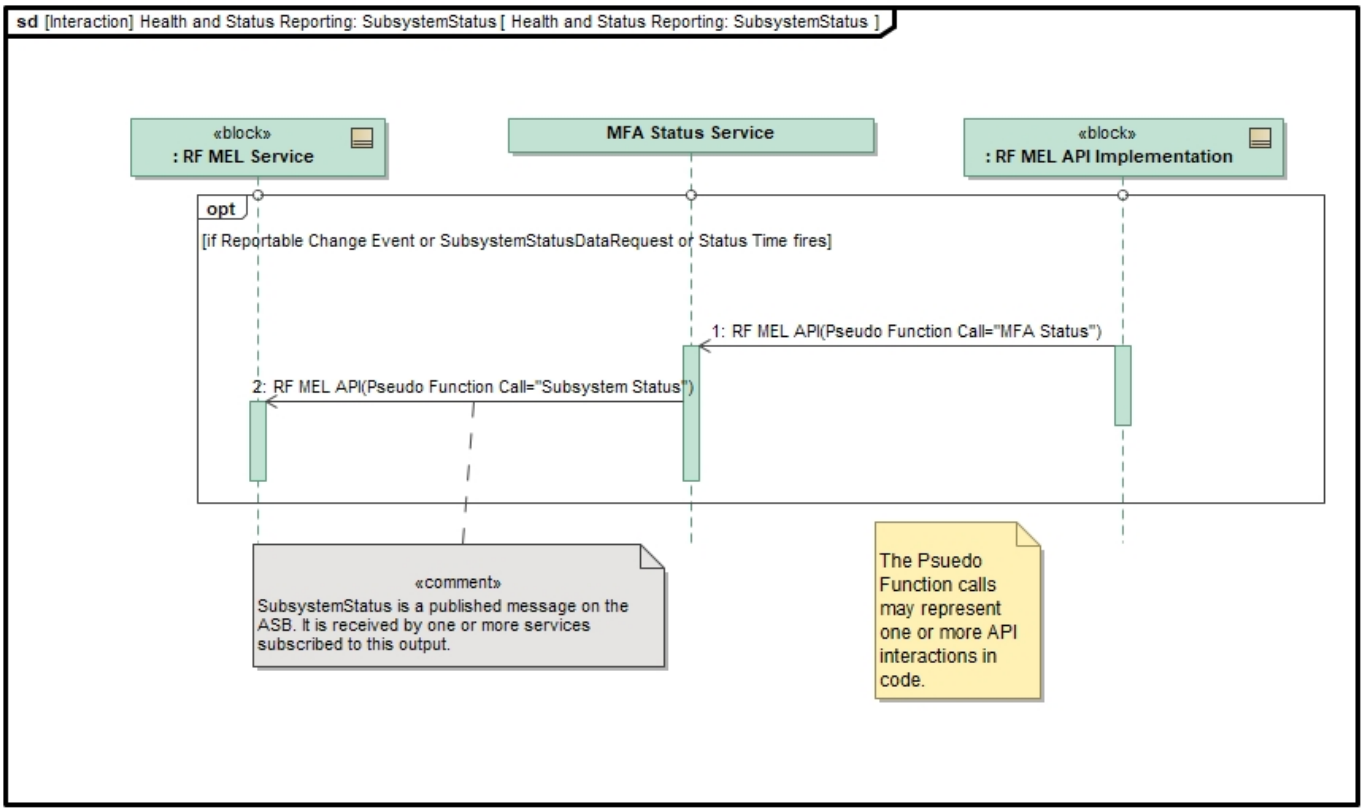


Figure 4.10.1: An MFA Reports Its Status in a `SubsystemStatus` Message Due to a Change in Status, a Request for Status, or a Periodic Timer.

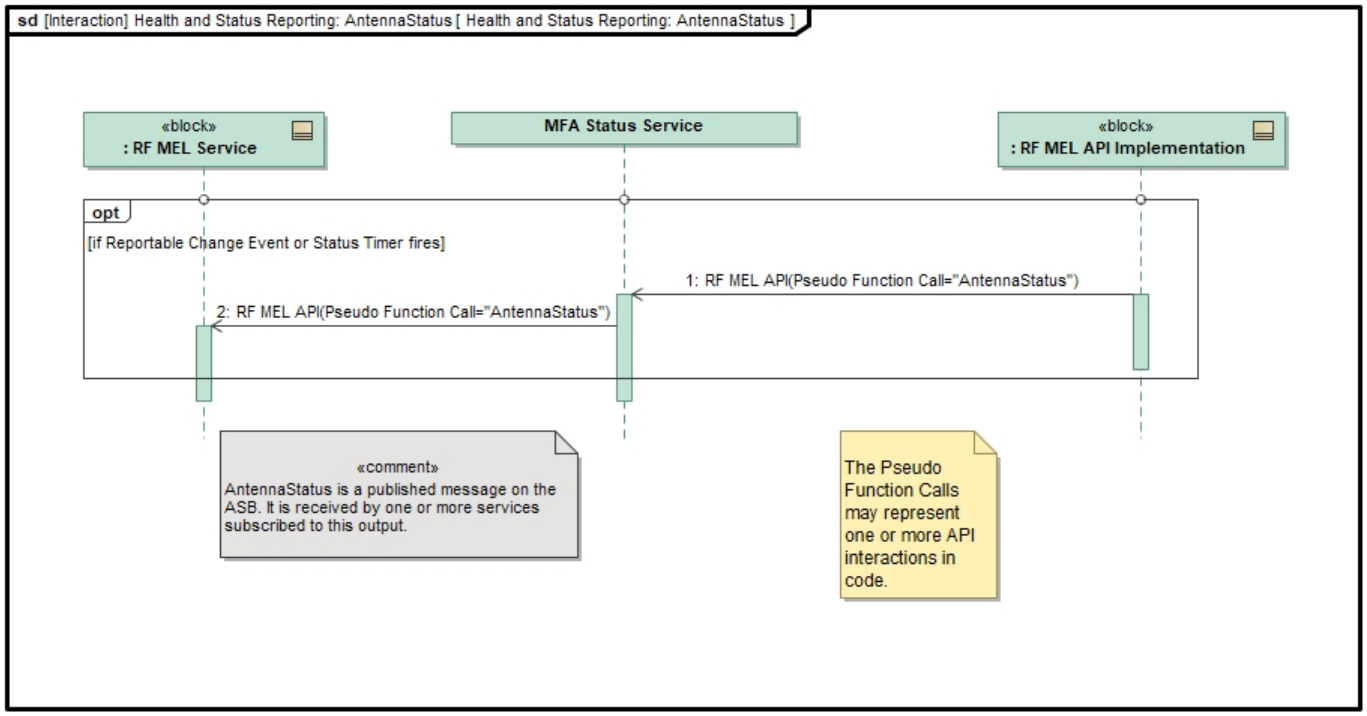


Figure 4.10.2: An MFA Reports Its Status in an `AntennaStatus` Message Due to a Change in Status or a Periodic Timer.

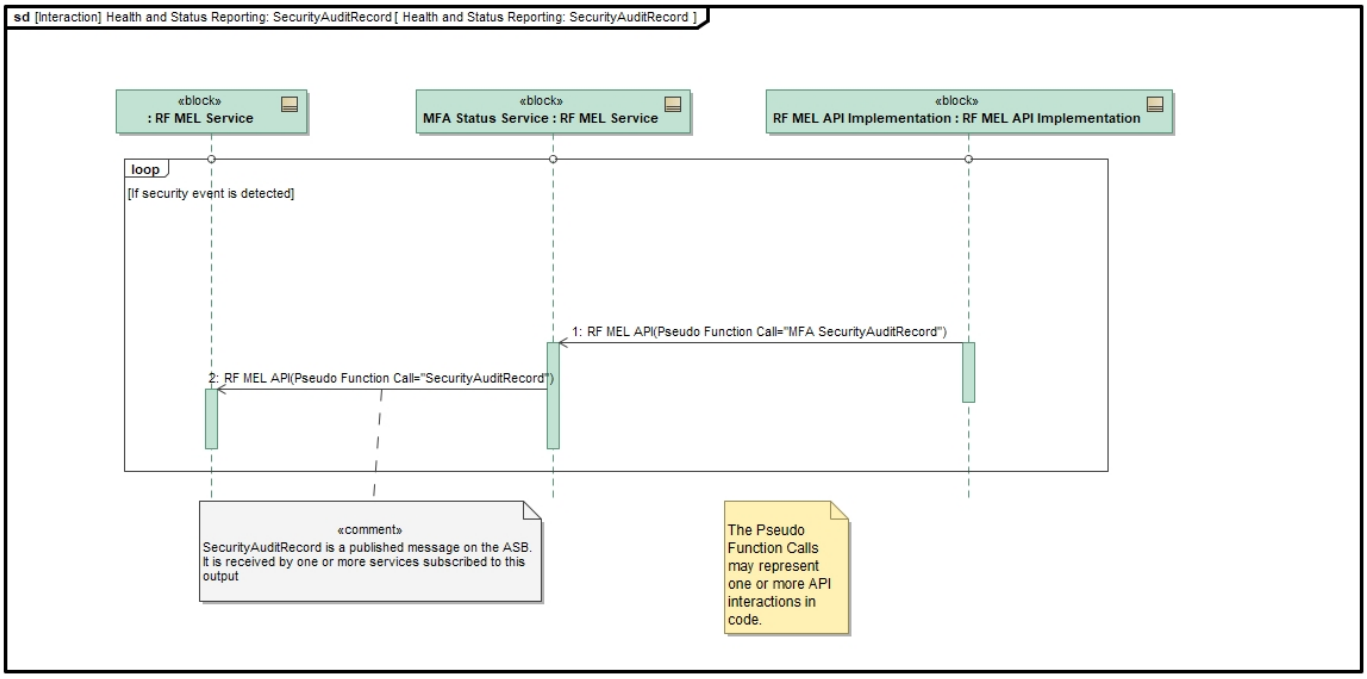


Figure 4.10.3: An MFA Reports Its Status in an `MFA_SecurityAuditRecord` *Message Due to the Detection of a Security Event.*

Local Function status primarily informs MFP layer services of the current state of specific, deployed capabilities that services running in the MFP require to perform as designed. Each Local Function should define the conditions for reporting a degraded state in the Local Function contract template. The degraded state conditions inform the service provider of the impacts to overall service functionality resulting from a Local Function with a degraded state of operation. **Figure 4.10.4** captures the pattern for reporting the status of Local Function. In this example, a third party develops a Local Function for the specific MFA and, as a result, the definition of the RF MEL interface for this MFA extends to include the status of the new Local Functions.

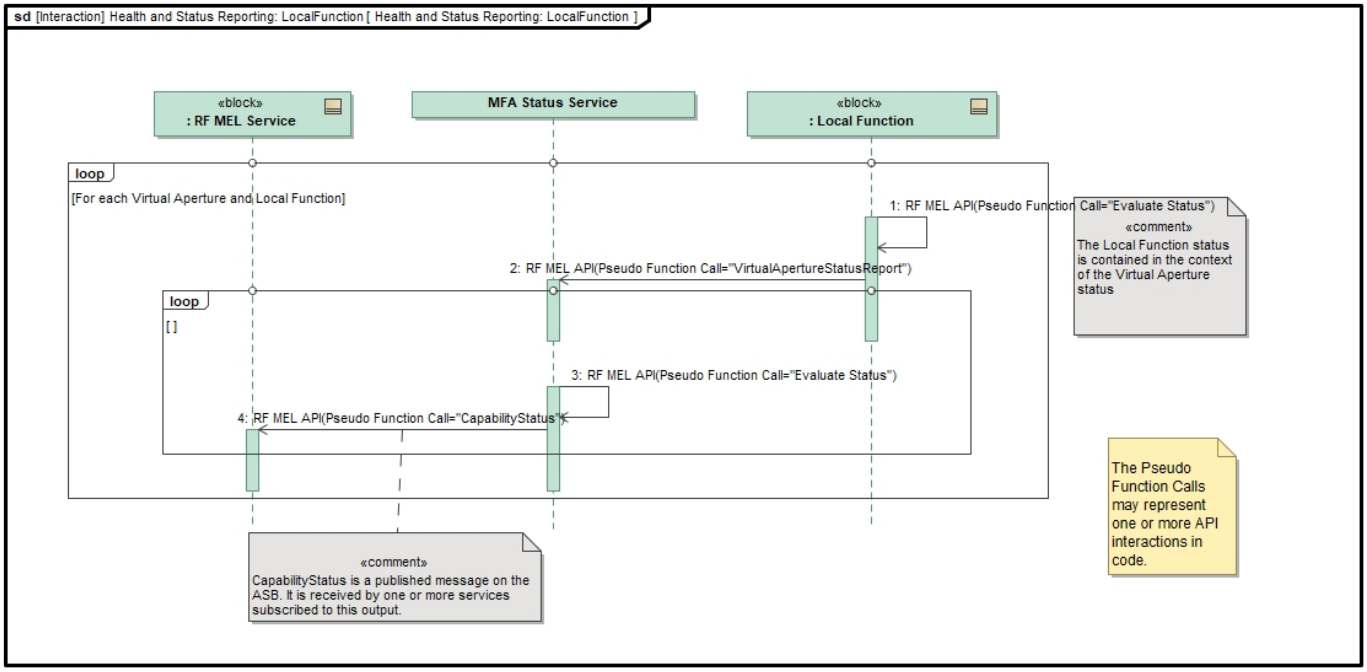


Figure 4.10.4: A Local Function Reports Its Status Through the RF MEL.

The following BIT interactions include two categories: queries and commands. Queries may be used by any service using the MEL. However, commands are expected to be performed by a single AMS GRA Service acting as an OMS Adapter. A MEL implementation may enforce this command restriction via an authentication mechanism, but this feature is not required.

A list of available BIT tests can be requested through `getConfiguration()`. This list can then be used to control individual BIT tests using `commandBIT()`. This command allows tests to be activated or canceled and provides a means to clear faults. Note that only one of these three (`initiateBIT_ID`, `cancelBIT_ID` or `clearFaultCode`) is expected to be populated in any given `BIT_Command` to match the expected behavior of the OMS SubsystemBIT_CommandMDT message.

BIT status faults are both reported through `BIT_Status` which corresponds with OMS/UCI `SubsystemBIT_StatusMDT` and can be requested via `getBIT_Status` or by registering for updates through the appropriate `setCallback` method. This alignment with OMS/UCI allows for easy translation and publishing of the data by the MEL OMS Adapter via UCI message(s). Both software and hardware faults from the MFA with the exception of faults in the MEL library are reported via `BIT_Status`. Any faults in the MEL library are reported via

the MEL API and the service using the MEL library is responsible for reporting any OMS faults as part of its own status. `BIT_Status` is reported via the MEL API asynchronously on change or periodically and by request through a `getBIT_Status`. Faults are not necessarily related to or exclusively detected by BIT. Faults shall populate the `faultCode` field with an unsigned integer-based number and the severity field. This fault and status behavior are shown in **Figure 4.10.5**.

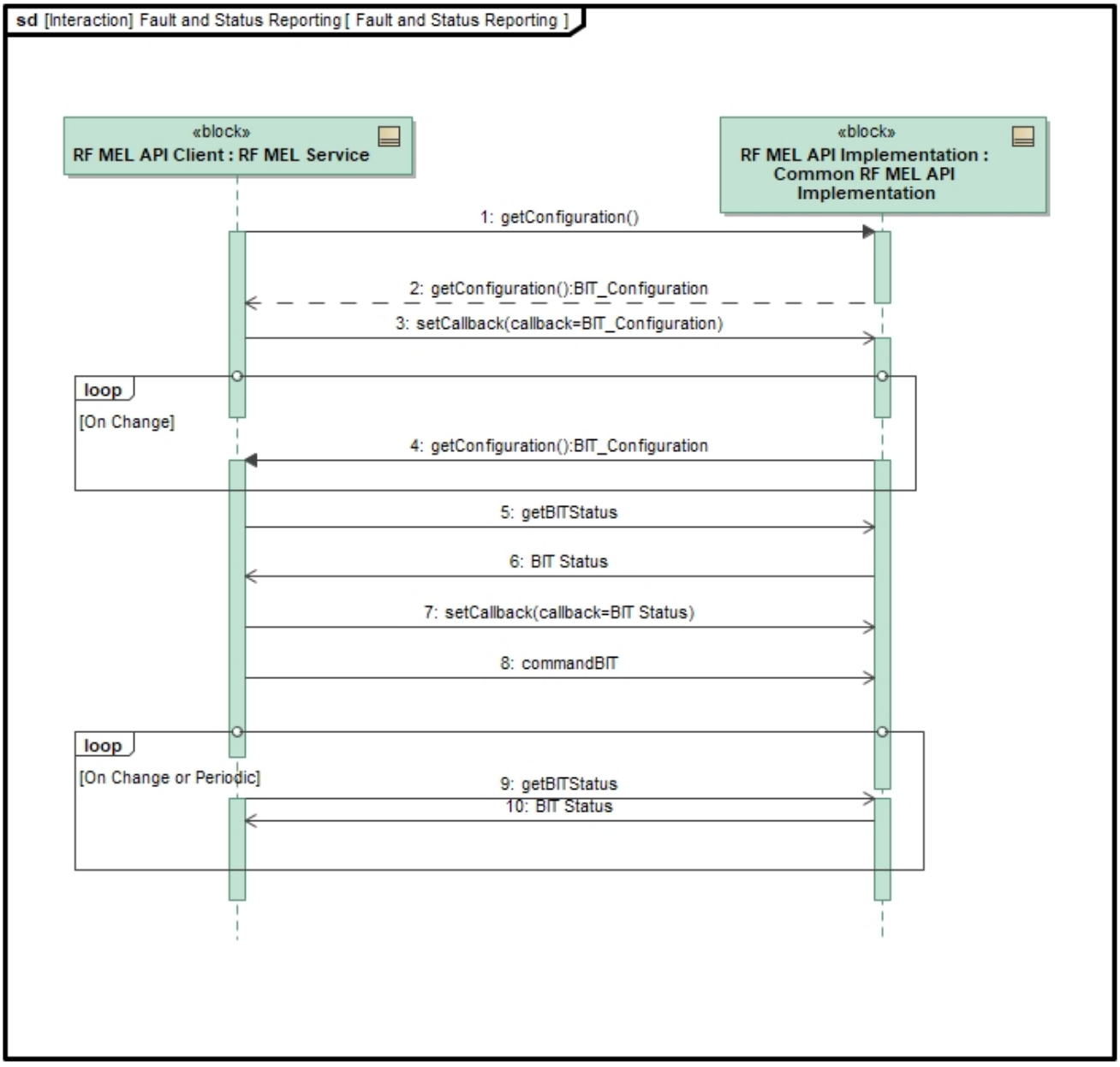


Figure 4.10.5: Faults With the RF MEL Library are Reported Via the RF MEL API.

4.11 Local Function Control

Description

This use case describes how a service specifies `LocalFunctionCommands` to be executed within a job `JobInterval` on a VA. It is shown below in **Figure 4.11.1**.

Preconditions

Before the service specifies `LocalFunctionCommands` to be executed within a job `JobInterval` on a Local Function, the following conditions must be true:

- The MFA is in an operational state and has loaded the Local Functions used by the service.
- The service has created the required endpoints and allocated its VA.
- The service has optionally used the VA class instance to query for the availability of the VA and Local Function instances.
- The service has a `JobDetail` for a scheduled job.

Post-Conditions

After the service specifies local function commands to be executed within a job interval on a VA, the following conditions must be true:

- A job executes on the MFA incorporating the desired Local Functions into its processing pipeline.
- Optionally, the Local Function replaces the packet content with its own content. This replacement is documented in the Local Function Service Contract.

Assumptions

The following functions and conditions are assumed true for the `LocalFunctionCommands` to be properly executed within a `JobInterval` on a VA:

- The VA is designed to incorporate Local Function processing in the MFA.
- Each instance of the VA may have its own instance of the Local Function. Alternatively, a Local Function may be shared across multiple instances (which are presumably not compatible with each other due to sharing the Local Function resources).

- The memory map of the Local Function is provided to the service developer, and the addresses and format of the Local Function control interface are known to the service implementation.
- In the case that the Local Function must make real-time decisions, the VAs must allocate all resources required to execute the Local Function capability even if they are not used.

Actors

The actors in the Local Function Control use case are:

- The service
- The MEL API Implementation
- The Local Function

Flow of Events

Figure 4.11.1 shows the service specifying Local Function commands to execute within a job interval on a VA:

1. Issue job requests a VA.
2. Populate commands as `JobIntervals` objects supplied to the `JobDetails` class.
3. For each `JobInterval` supplied by the service to the `JobDetails` class, the service specifies a `vector` of `LocalFunctionCommands` into its `if_commands` property to set register level values inside the Local Function instance selected by the MFA.

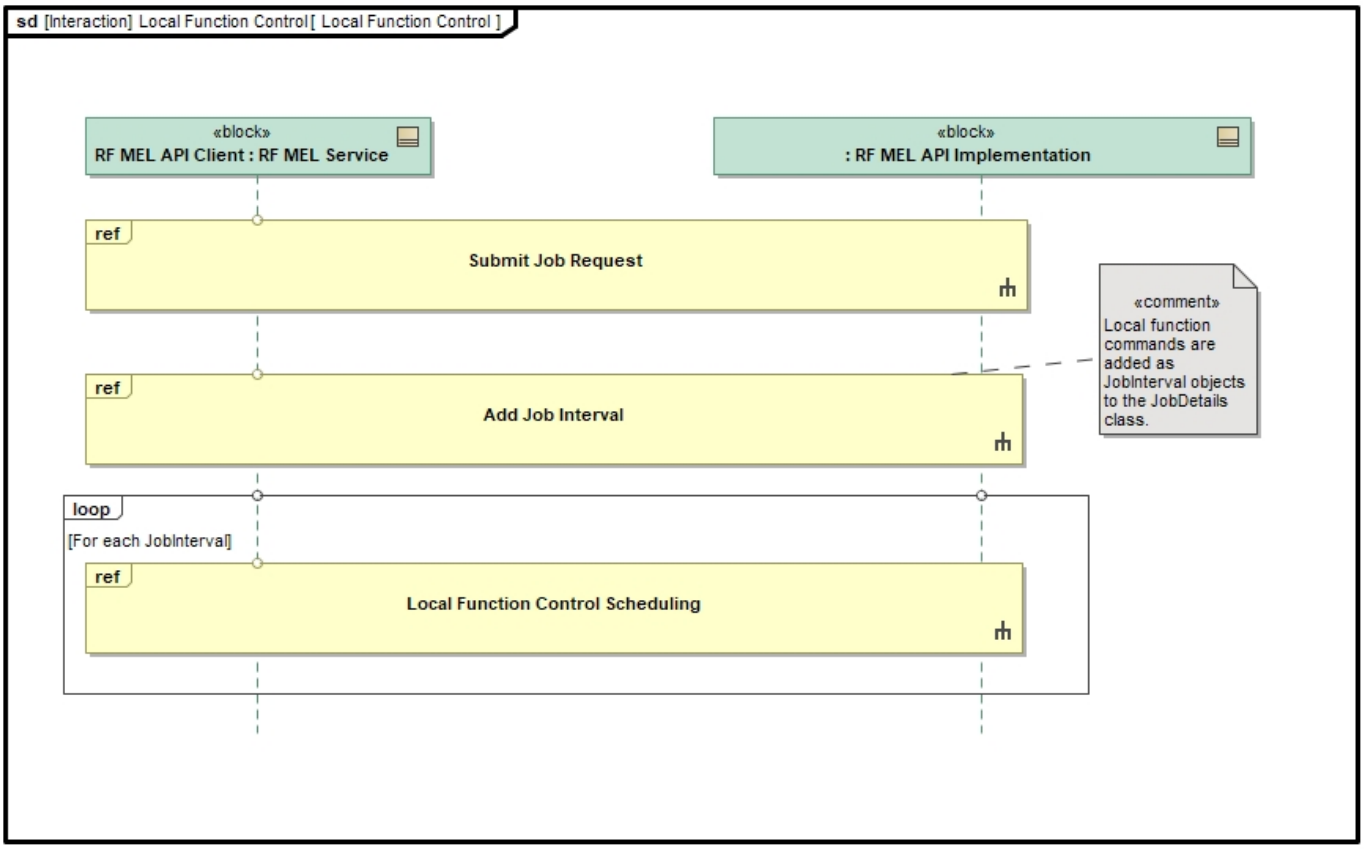


Figure 4.11.1: A Service Specifies Local Functions to be Executed on a VA.

Flow of Events

Figure 4.11.2 shows the service populating the `JobInterval` and details using the local function commands:

1. The service specifies a `vector` of `LocalFunctionCommands` to set register level values inside the Local Function instance selected by the MFA.
2. The service populates Local Function commands on the `JobInterval` using `setLfCommands`.
3. The service adds the populated `JobInterval` to the `JobDetail`.

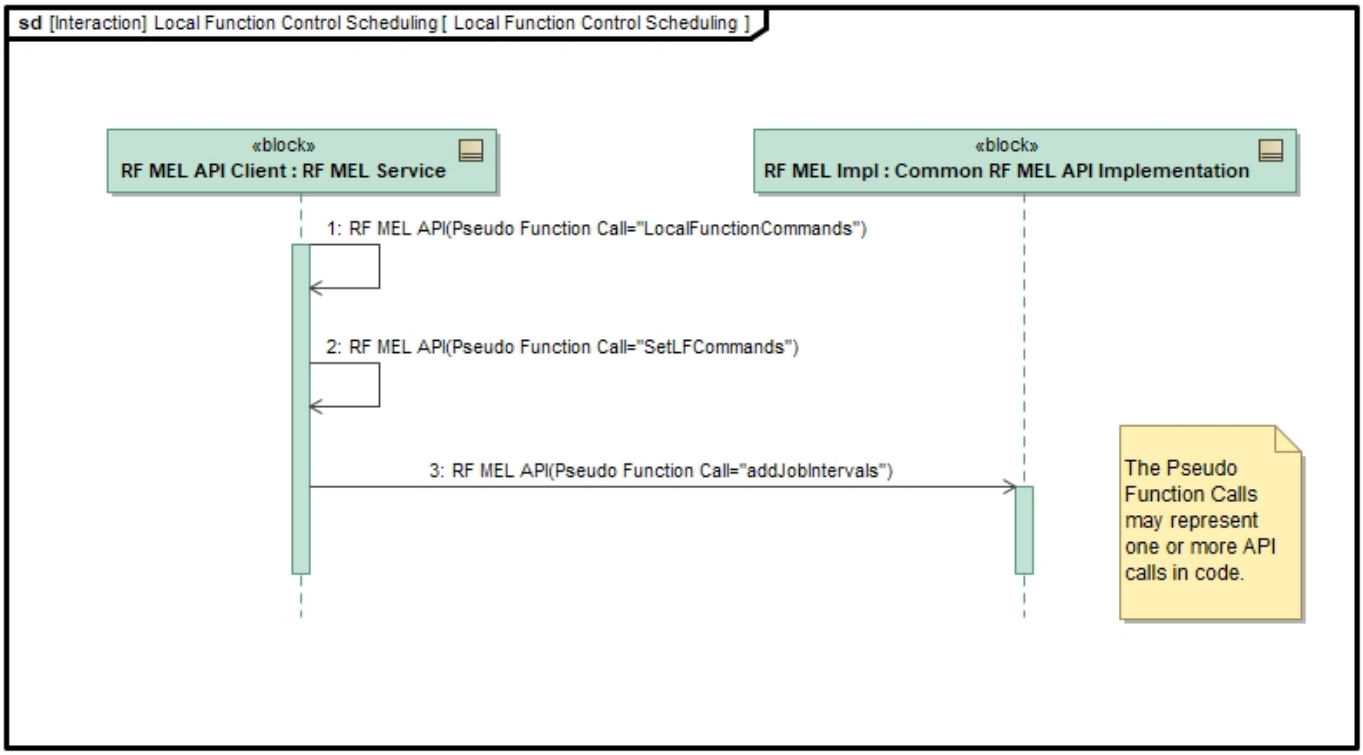


Figure 4.11.2: The Service Adds Job Details Using the Local Function Commands.

4.12 Inhibit VA

Description

The sequence diagram, show in **Figure 4.12.1** , captures how a service requests an inhibit VA using the RF MEL. An inhibit VA is a specialized VA that enables the requesting service to control and limit the RF transmissions from a specific MFA, in order to coordinate RF compatibility across multiple MFAs on a platform. An inhibit VA provides a narrowly focused set of functionalities and does not use any endpoint connections for its operation. Similar to other VA requests, the request for an inhibit VA requires a `VirtualApertureDefinitionID` that is established and defined at design time. This ID specifically references an inhibit VA definition. AMS GRA generally anticipates that only a limited set of services perform this function, but the ability for a service to request an inhibit VA is left as a decision for the MSI.

The request for an inhibit VA occurs during service initialization, and the function call includes the UUID of the requesting service. The service UUID is intended to provide a mechanism for the MFA to verify that the requesting service is permitted to request an inhibit VA, as using an inhibit VA represents a specialized service role for commanding an MFA. If service UUID verification is desired, the MFA requires a service UUID white list to be provided with MFA OEM software at initialization.

Upon successful establishment of an inhibit VA, the service registers a status callback function to receive status updates for the inhibit VA. This interface provides a simple mechanism for the service to maintain an accurate, summarized assessment of all of the supporting resources in the MFA, associated with this VA. This approach decouples services from detailed MFA implementations and component status. This Inhibit VA concept is elaborated in the Resource Management and MFA volumes.

Preconditions

Before the service can request a handle for an inhibit VA using the RF MEL, the following conditions must be true:

- The service has been successfully deployed.
- The RF MEL has been successfully initialized.
- The service has a valid instance of the RF MEL.
- The service has the `VirtualApertureDefinitionID` of the inhibit VA that it requests.

Post-Conditions

After the service has requested the inhibit VA, the following conditions should be true:

- Upon success, the service can submit requests to inhibit specific transmit actions to the MFA through the inhibit VA and can process inhibit VA status updates.
- Upon failure, the service takes appropriate actions based on the type of failure, as defined by the Service Contract.
- Upon failure, the service and MFA continue in an operational state.

Assumptions

- There are no further assumptions to execute the Inhibit VA use case.

Actors

The actors in the Inhibit VA use case are:

- The service.
- The MEL API Implementation

Flow of Events

Figure 4.12.1 shows the inhibit VA handle being requested by the service:

1. The service requests an inhibit VA from the RF MEL by calling the `requestInhibitVirtualAperture()` function with the `VirtualApertureDefinitionID` and priority.
2. The RF MEL returns to the service a `RequestFor<InhibitVirtualAperture>` object, which is a future type.
3. The service gets the response from the future type.
4. If the request was successful, the service gets the `InhibitVirtualAperture` object.

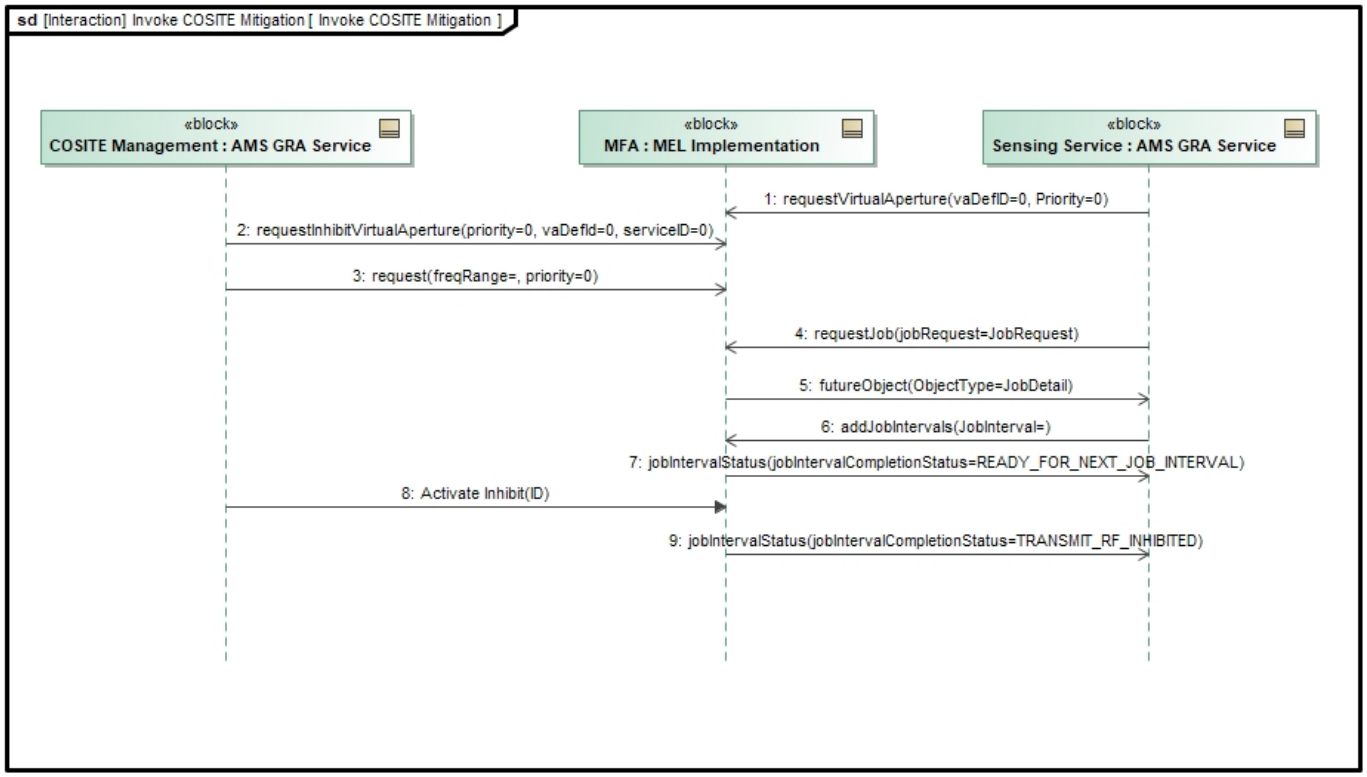


Figure 4.12.1: A Service Requests an Inhibit VA to Control and Limit RF Transmissions From the MFA.

4.13 Request RF Activity Notifications

Description

The sequence diagram, shown in **Figure 4.13.1**, captures how a service can use the RF activity interface in the RF MEL to gain awareness of RF activity on multiple MFAs. The RF activity interface is intended to provide a tailorable mechanism for services to access low latency updates describing ownship transmissions that may impact a service's processing functions. The interface is designed to inform a service of both scheduled and executing RF activities. This interface also allows an adapter for legacy equipment to gain insight into ownship emissions and communicate its spectral usage.

In the example sequence diagram, Service 1 is employing MFA 1 directly to accomplish commanded activities.

Simultaneously, Service 1 is using the RF activity interface to receive information about transmit events from MFA 2 for a specific region of the spectrum. This sequence represents an example of a distributed implementation of RF coordination, in which Service 1 is responsible for comprehending ownship emissions and responding appropriately. This interface can also support a more centralized RF coordination approach, in which a single service is evaluating spectral compatibility across multiple Skills and MFAs.

Using the knowledge of current and ownship RF activities, the centralized RF coordination Skill can command sensing Skills in a manner that limits RF interference between them. This approach decouples services from detailed MFA implementations, and also decouples the Skills from each other. This interface provides one of the key mechanisms for RF coordination across a platform; the inhibit VA interface provides the second mechanism for RF coordination.

Preconditions

Before the service can request RF activity callbacks using the RF MEL, the following conditions must be true:

- The services have been successfully deployed.
- The RF MELs for all MFAs of interest has been successfully initialized.
- The service has a valid instance of each RF MEL.

Post-Conditions

After the service has requested RF activity updates, the following conditions should be true:

- Upon success, the service will receive RF activity updates from the MFA when conditions specified in the RF activity filter are met. These updates will include notification of pending, executing, and completed RF activity.
- Upon failure, the service takes appropriate actions based on the type of failure, as defined by the Service Contract.
- Upon failure, the service and MFA continue in an operational state.

Assumptions

- There are no further assumptions to execute the Request RF Activity Notifications use case.

Actors

The actors in the Request RF Activity Notifications use case are:

- Sensing services.
- The MFAs

Flow of Events

Figure 4.13.1 shows services utilizing VAs on multiple MFAs, following a standard pattern, with Service 1 additionally requesting RF activity information from MFA 2:

1. The services request VAs using the appropriate RF MEL by calling the `requestVirtualAperture()` function with the `VirtualApertureDefinitionID` and priority.
2. Service 1 configures an RF activity callback on MFA 2 by calling `addActivityCallback` with the desired RF activity filter defined.
3. Service 1 requests timeline on MFA 1 to execute commanded tasking.
4. Service 2 requests timeline on MFA 2 to execute commanded tasking.
5. MFA 2 sends RF activity updates to Service 1 when jobs are scheduled.
6. MFA 2 sends RF activity updates to Service 1 when jobs are executing.

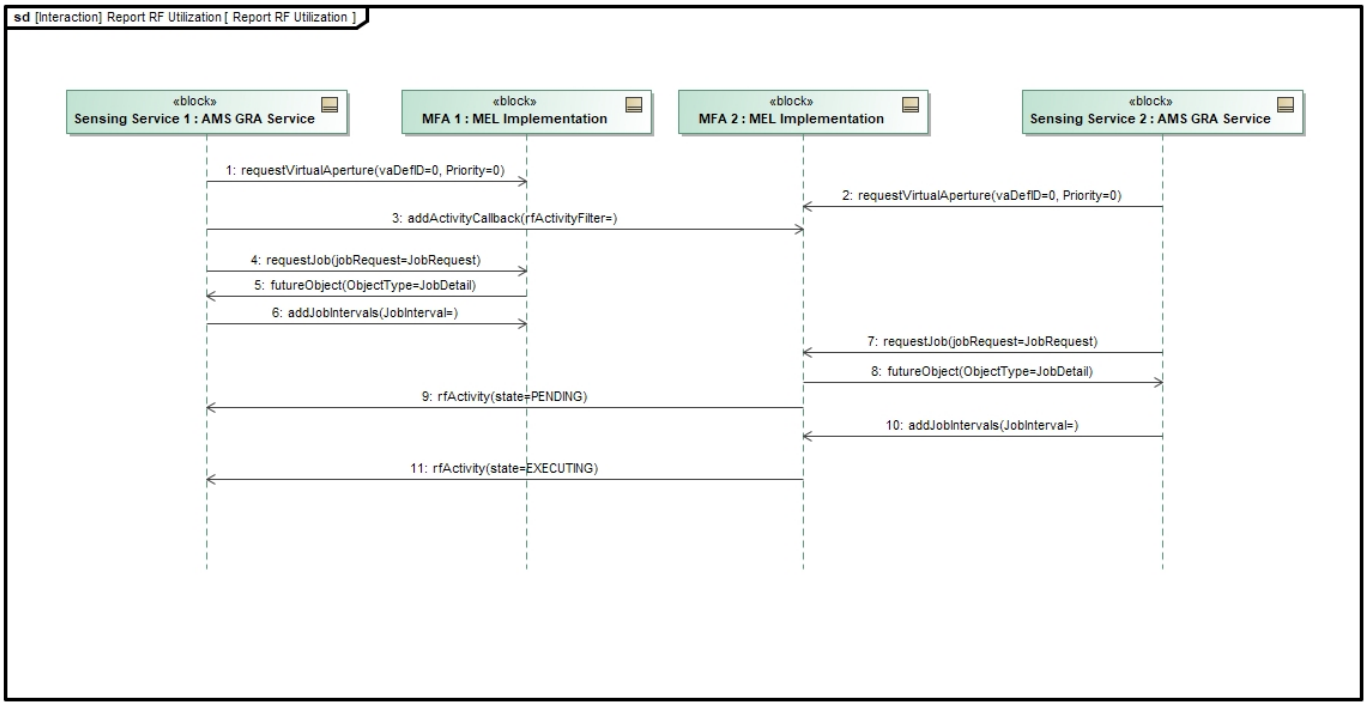


Figure 4.13.1: A Service Uses the RF Activity Interface to Access Scheduled and Executing RF Activities on MFAs.

5.0 Advanced Features Use Cases

The use cases in this chapter use features of the RF MEL to support advanced responses to a variety of operational scenarios.

5.1 Multi-Beam Use Case

Description

This use case, shown in **Figure 5.1.1**, describes how a service uses the RF MEL to command a job that forms multiple receive beams, all pointed to different arbitrary locations in space, with each beam formed by a separate Element Group. The Element Groups may overlap or use completely separate array elements on the MFA face depending on the design of the VA. Note that separate, unique commands are issued to each Element Group.

Preconditions

Before a service uses the RF MEL to command a job that forms multiple receive beams, the following conditions must be true:

- The service is using a VA with multiple Element Groups. For instance, this VA might support multiple independent receive beams with each commanded to a different pointing angle and frequency band.
- The service has completed the Load RF MEL (see **Section 4.1**) and Request VA (see **Section 4.4**) operations successfully.

Post-Conditions

After the service uses the RF MEL to command a job that forms multiple receive beams the following conditions must be true:

- Each Element Group within the MFA executes its portion of the requested job and each group streams its output to a different endpoint.
- After execution, each `JobInterval` is submitted as usual and the `finalize()` method completes the specification of the job as usual.

Assumptions

The following functions and conditions are assumed true for the RF MEL to command a job that forms multiple receive beams successfully:

- A set of VA definitions is provided with the RF MEL, or the MFA provider also delivers a set of tools to service developers to build VAs appropriate to the capabilities of the given MFA.
- Any pair of Element Groups in the VA may include overlapping elements on the array, depending on the simultaneity supported within the MFA.
- The MFA provides the capability to process `JobInterval`s independently for each Element Group, and the durations of those `JobInterval`s need not align. However, they must fit within the overall job duration.
- The number of Element Groups included in a given VA is known at design time.
- The number of endpoints required per Element Group in a given VA is known at design time.

Actors

The actors in the Multi-Beam cse case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 5.1.1 shows the service using the RF MEL to command multiple receive beams:

1. At initialization, for each of the Element Groups designed into the VA, the service creates a separate endpoint.
2. Inside each `JobRequest` issued by the service, the number of `ElementGroups` in the `rxElementGroups` vector is greater than zero but no greater than the number of groups defined in the VA. It is valid for a job to ignore a subset of Element Groups, leaving them allocated to the service but idle. The service sets up parameters for each `ElementGroup` as needed for the job.
3. For each `JobDetail` received from a successful `JobRequest`, the service populates a series of `ProductStreamParams` mapping to each Element Group referenced in the `JobRequest`.

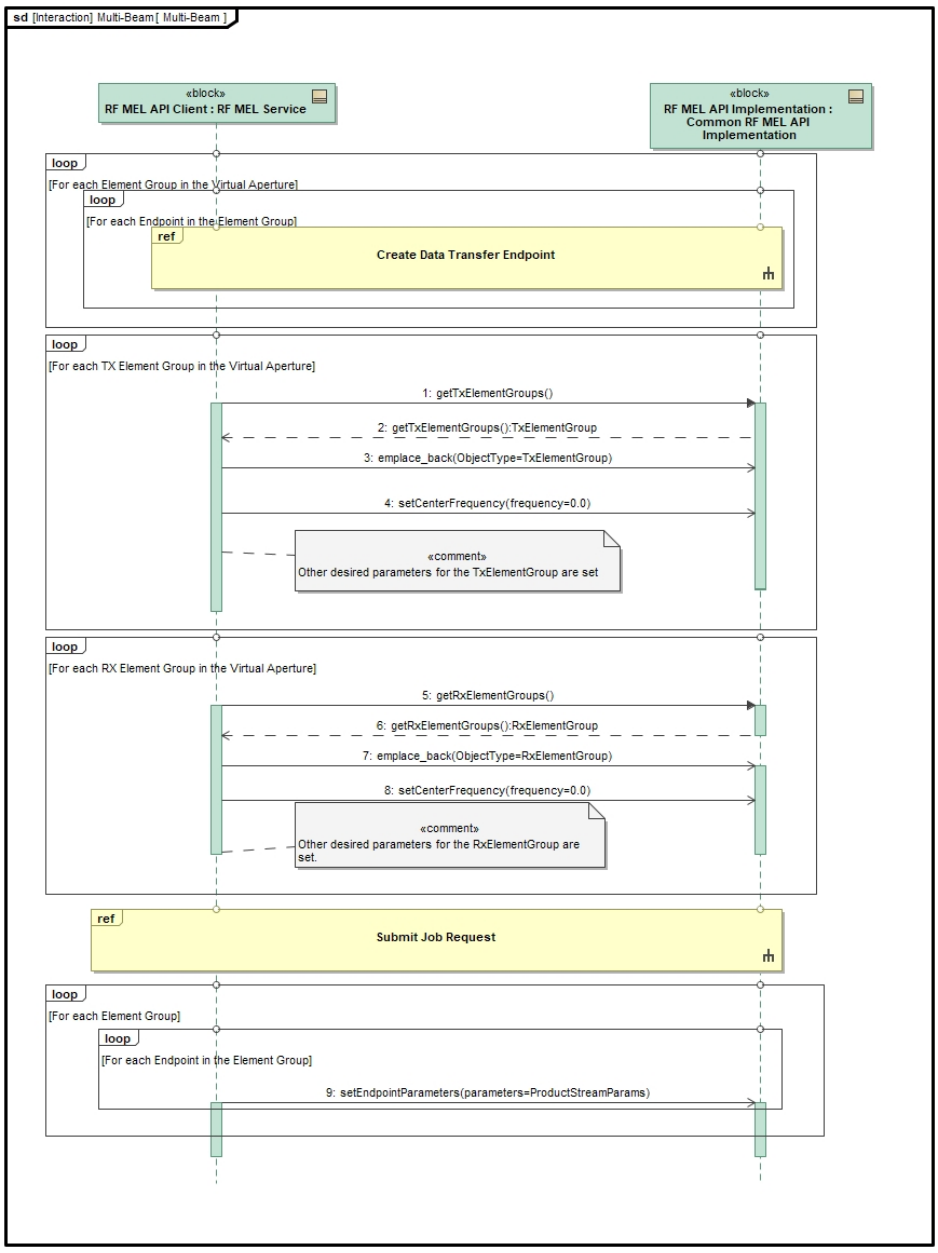


Figure 5.1.1: A Service Uses the RF MEL to Command a Job That Forms Multiple Receive Beams.

5.2 Multi-Channel Use Case

Description

This use case, shown in **Figure 5.2.1**, describes how a service uses the RF MEL to command a job from which multiple channels or streams of data flow from the MFA to services deployed to the DPPs in the MFP. An example would be a Radio Detection and Ranging (RADAR) Skill using a VA which produces multiple output channels that form the components of a monopulse antenna: sum, delta and guard.

Pre-Conditions

Before the service uses the RF MEL to command a job from which multiple channels or streams of data flow from the MFA to services deployed to the DPPs in the MFP, the following conditions must be true:

- The service developer is using a VA with multiple output channels. For instance, this VA might support monopulse direction finding by receiving separate channels for sum, delta-az, delta-el and guard.
- The service has completed the Load RF MEL and Request VA operations successfully.

Post-Conditions

After the service uses the RF MEL to command a job from which multiple channels or streams of data flow from the MFA to services deployed to the DPPs in the MFP, the following conditions must be true:

- The MFA executes the requested job, and each of the specified endpoints receives its associated stream of data.
- After execution, each `JobInterval` is submitted as usual and the `finalize()` method completes the specification of the job as usual.

Assumptions

The following functions and conditions are assumed true for the service to use the RF MEL to command a job with multiple channels or streams of data:

- Either a set of VA definitions is provided with the RF MEL, or the MFA provider delivers a set of tools to service developers to build VAs appropriate to the capabilities of the given MFA.
- The number of output channels produced by a given VA is known at design time.

Actors

The actors in the Multi-Channel use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 5.2.1 shows the service using the RF MEL to command a job with multiple channels or data streams:

1. At initialization, for each of the output channels designed into the VA, the service creates a separate endpoint.
2. Inside each `JobRequest` issued by the Skill, each Element Group in the `rxElementGroups` array contains an array, `endpointIDs`, whose size matches the number of output channels designed into the VA. This mapping of VA channels to endpoints allows the VAS to select resources available to execute the job on the MFA and route data to the assigned endpoints.
3. For each `JobInterval` received from a successful `JobRequest`, the service populates its `ProductStreamParams` with an array of `EndpointParameters` objects whose size matches the number of output channels in the VA, each of which references the endpoint ID of one endpoint owned by the service.

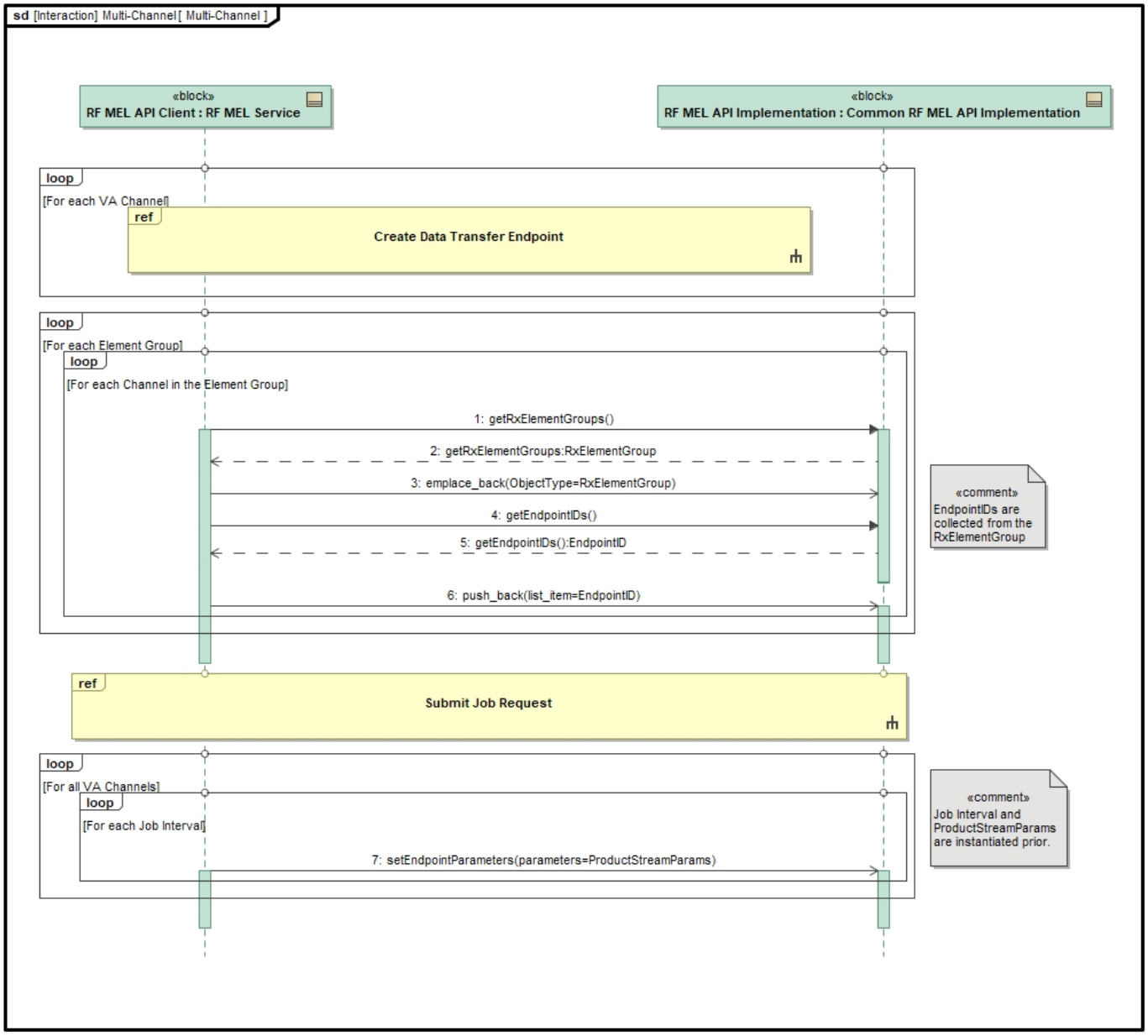


Figure 5.2.1: A Service Uses the RF MEL to Command a Job. This Job Entails Multiple Streams of Data Flowing From the MFA to Services on DPPs in the MFP.

5.3 Multi-Polarization Use Case

Description

This use case, shown in **Figure 5.3.1**, describes how a service uses the RF MEL to command a multiple-polarization job on the MFA. Commanding this job results in multiple channels or streams of data, one per commanded polarization.

Pre-Conditions

Before a service uses the RF MEL to command a multiple-polarization job on the MFA, the following conditions must be true:

- The service developer has selected a VA with either multiple Element Groups, potentially covering the same sets of array elements, or a single Element Group from which multiple channels are produced.
- The service has completed the Load RF MEL and Request VA operations successfully.

Post-Conditions

After the service uses the RF MEL to command a multiple-polarization job on the MFA, the following conditions must be true:

- The MFA executes the requested job and each commanded polarization results in a stream of output to a different endpoint.
- After execution, each `JobInterval` is submitted as usual and the `finalize()` method completes the specification of the job as usual.

Assumptions

The following functions are assumed true for the service to command multiple jobs on the MFA successfully:

- The MFA is capable of receiving and digitizing raw I/Q inputs from differently-polarized sub-elements and combining them according to coefficients provided in the `JobDetail`.
- The MFA documentation describes the orientation of the axes of polarization relative to the MFA faces.

Actors

The actors in the Multi-Polarization use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 5.3.1, shows a service using the RF MEL to command a multiple-polarization job on the MFA, resulting in multiple channels or streams of data, one per commanded polarization:

1. At initialization, the service creates a separate endpoint for each commanded polarization. In the event of a multiple-output-channel VA, the service repeats this process for each channel.
2. Inside each `JobRequest` issued by the service, each `ElementGroup` in the `rxElementGroups` array contains an array, `endpointIDs`, whose size matches the number of endpoints. If multiple `ElementGroups` are used, then the sum of all `endpointIDs` across all the `ElementGroups` matches the number of endpoints. The service assigns the `endpointIDs` per `rxElementGroups` as needed for the job.
3. For each `JobDetail` received from a successful `JobRequest`, the service populates its `ProductStreamParams` with an array of `EndpointParameters` objects whose size matches the number of endpoints, each of which references the `EndpointID` of one endpoint owned by the service. Each `ReceiveEvent` contains an array named polarization, whose size is the number of commanded polarizations, and whose elements are the coefficients describing how to combine the polarized sub-elements.

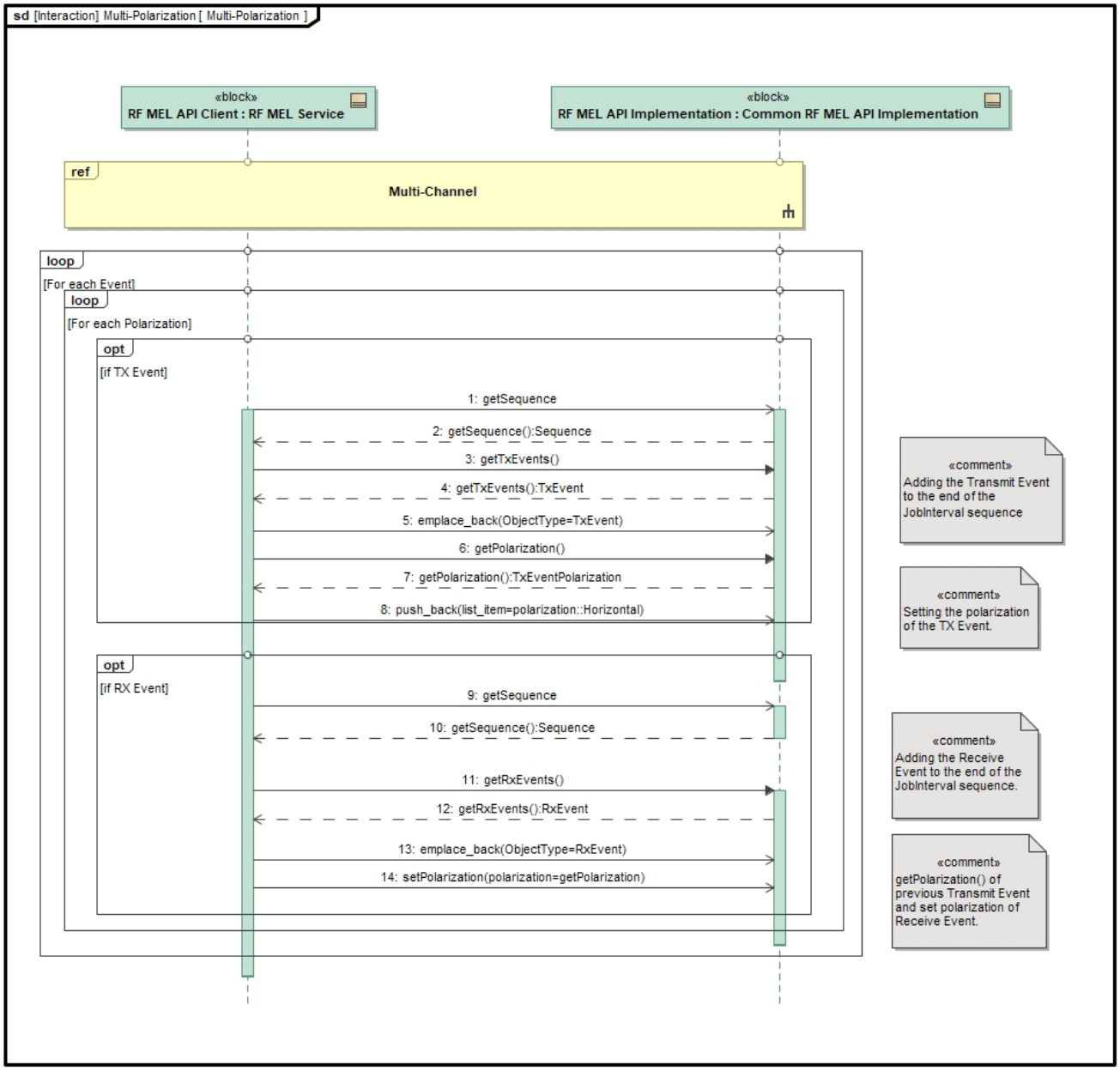


Figure 5.3.1: A Service Uses the RF MEL to Command a Multiple-polarization Job.

5.4 Round Robin Use Case

In Round Robin, sets of data produced by the MFA are transferred to different DPPs for processing. For example, each `JobInterval`'s output data may be sent to a different DPP for processing. All of the processing services must receive the control service's commands so that they can properly process the data. An example of the Round Robin layout is shown in **Figure 5.4.1**. The behavior realized by this setup is captured in **Figure 5.4.2**.

Round Robin highlights the following issues:

- Signal processing controls must be provided to all services that execute the signal processing chain.
- The need for multiple services on multiple DPPs to receive low latency data transfers concurrently or nearly concurrently to allow for parallel processing.

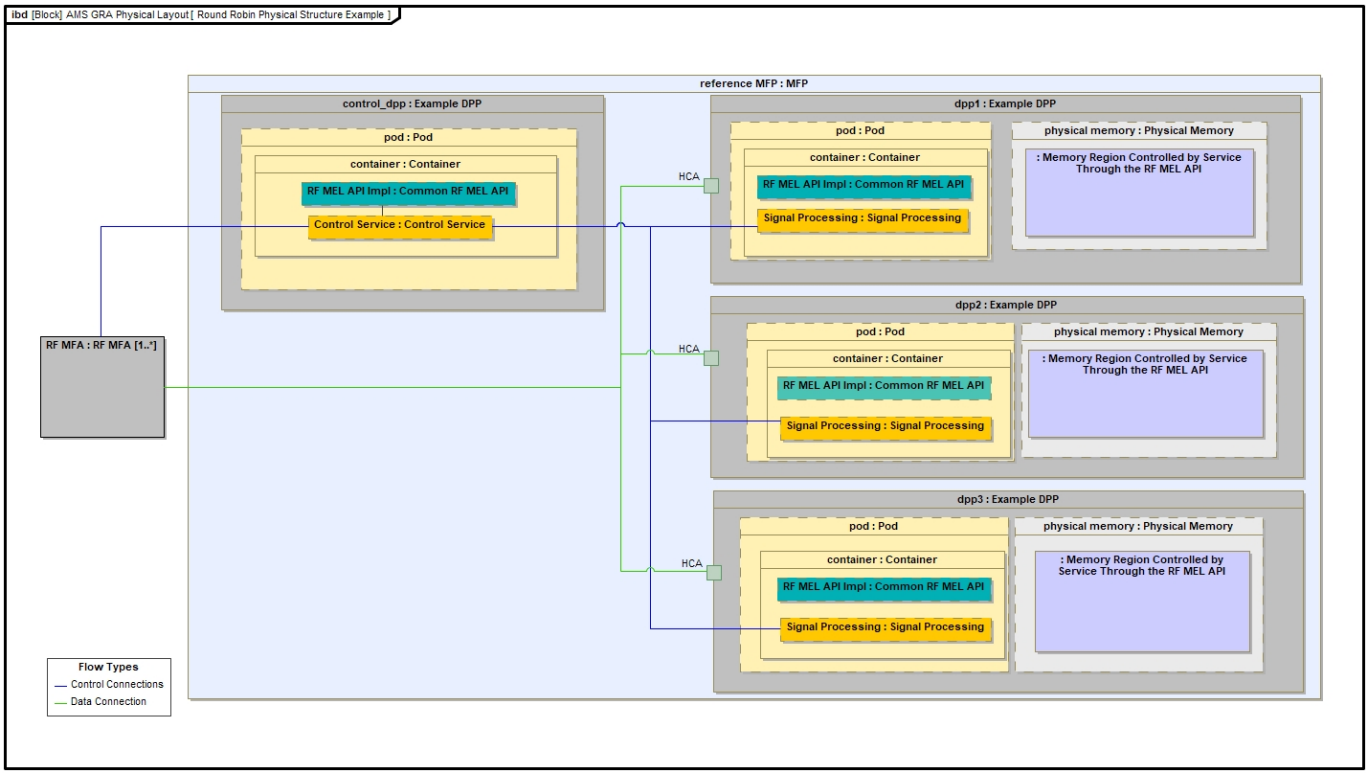


Figure 5.4.1: This is a Round Robin Physical Structure Example. Sets of Data From an MFA are Sent to Different DPPs for Signal Processing Via the RF MEL.

Pre-Conditions

Before the Round Robin sets of data produced by the MFA are transferred to different DPPs for processing, the following conditions must be true:

- The architecture, shown in **Figure 5.4.1**, has been successfully established.

Post-Conditions

After the Round Robin sets of data produced by the MFA are transferred to different DPPs for processing, the following conditions must be true:

- The data produced by the MFA can be processed by each DPP.

Assumptions

The following functions and conditions are assumed true for the data to be properly reported to the DPPs for processing:

- RDMA or RDMA over Converged Ethernet (RoCE) is implemented by the MFA.
- Under direction of the service, the RF MEL governs some portion of the physical memory on the DPP.
- Each service runs within a single container, and each container runs within its own pod.
- For simplicity, these DPPs only interface with a single MFA.
- The RF MEL can communicate with its MFA to receive the MFAs Internet Protocol address and port for the purpose of setting up a queue pair.
- The Host Channel Adapter (HCA) on the RF MFA has enough memory to create the necessary number of queue pairs (buffer and queue pair bookkeeping).
- Each service maps to an operating system process. There is some sharing of state across processes within a container.

Actors

The actors in the Round Robin use case are:

- The RF MFA.
- The five instantiations of the RF MEL.
- The four signal processing services.

Flow of Events

An example of the Round Robin layout is shown above in **Figure 5.4.1**. The behavior realized by this setup is captured below in **Figure 5.4.2**.

1. The RF MELs set up everything they need to use RDMA.
   (a) The RF MEL gets access to the DPP's HCA.
   (b) The RF MEL sets up a queue pair (has a priori knowledge of MFA, Internet Protocol Address, and Port).
   (c) The RF MEL allocates memory in the DPP physical memory.
   (d) The RF MEL associates memory with the queue pair.
2. The control service goes through the usual sequence to request a VA, request a job, receive a `JobDetail`, populate relevant parameters, and submit `JobIntervals`.
3. The control service provides its control parameters to the processing services (via the ABB).
4. The RF MFA performs the commanded functions and transfers the data from the MFA physical memory to the RF MEL controlled DPP physical memory.
5. The signal processing services access the data in the RF MEL controlled DPP physical memory for processing.
6. The signal processing services indicate when the memory can be freed/used again.

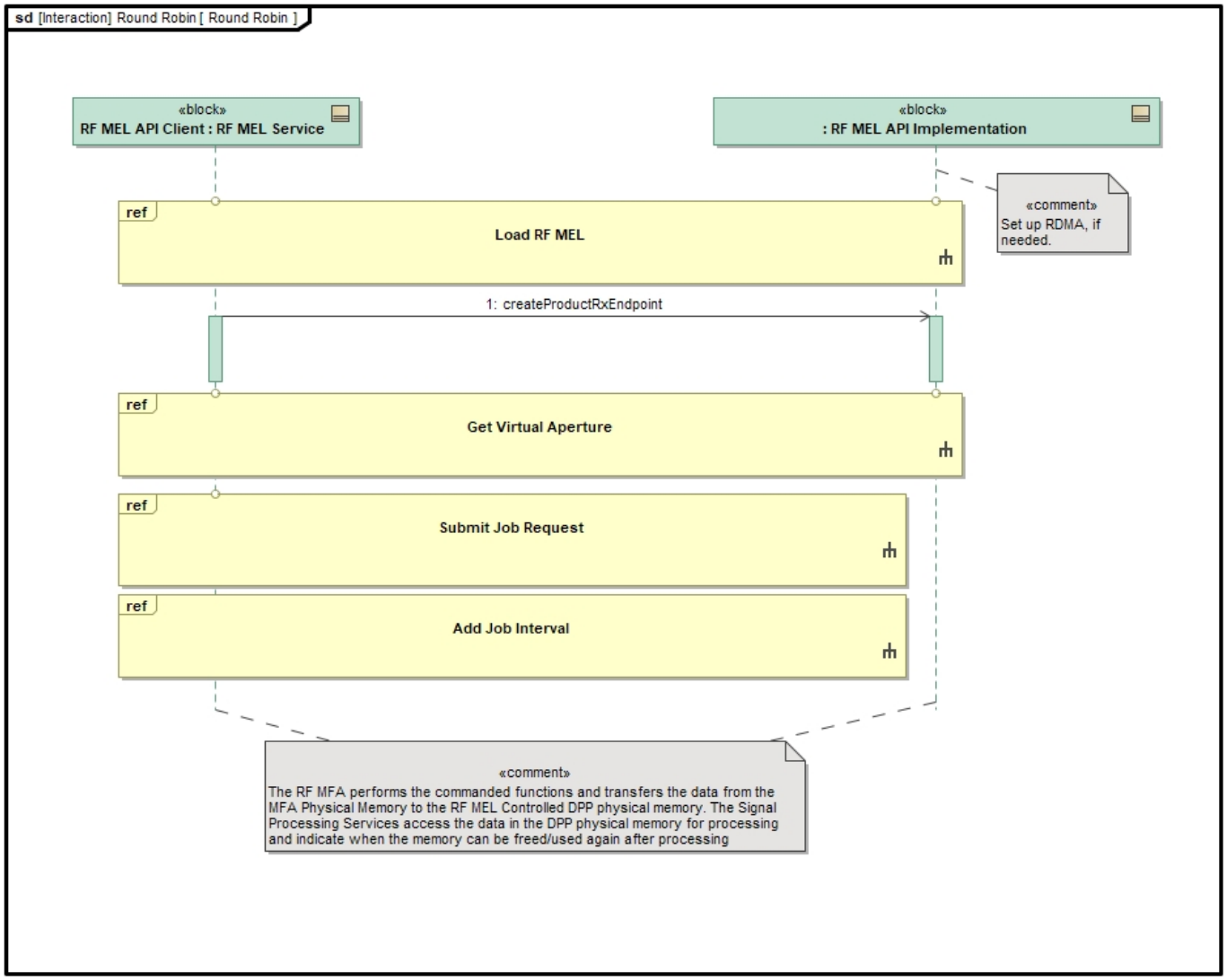


Figure 5.4.2: An MFA May Use a Round Robin Approach for Distributing Data to DPPs.

5.5 Adaptive Pulse Repetition Frequency

Description

A service uses the RF MEL to command a job on the MFA using a VA to perform a job using a waveform that includes adaptive Pulse Repetition Frequency (PRF) behaviors in which each `JobInterval` may utilize a different PRF within the overall `JobRequest`.

A service developer may instead issue separate `JobRequests` with different PRF. That approach requires many separate `JobRequests` and does not provide the benefits of an adaptive PRF behavior. This approach also requires the VAS to juggle more scheduled jobs.

Pre-Conditions

Before the VA can perform a job using a waveform that includes adaptive PRF behaviors, the following conditions must be true:

- The service has been successfully deployed.
- The RF MEL is loaded successfully.
- The service has a valid shared pointer to a VA through a previous successful VA request.
- The service has a valid pointer to an endpoint through a previous successful endpoint creation.

Post-Conditions

After the VA has performed a job using a waveform that includes adaptive PRF behaviors, the following conditions must be true:

- Upon success, the VA within the MFA executes its portion of the `JobRequest`, and streams its output to the defined endpoint.
- The service may submit additional `JobRequest(s)` and interact with the VA and endpoints.
- Upon failure, the service takes appropriate action based on the type of failure and may notify the system through a `Service_Status` messages.

Assumptions

The following functions and conditions are assumed true for the VA to correctly perform a job using waveform that utilizes adaptive PRF:

- Either a set of VA definitions is provided with the RF MEL, or the MFA provider delivers a set of tools to service developers to build VAs appropriate to the capabilities of the given MFA.
- Any pair of Element Groups in the VA may include overlapping elements on the array depending on the simultaneity support within the MFA.
- The MFA provides the capability to process `JobInterval` independently for each Element Group, and the durations of those `Intervals` need not align. However, they must fit within the overall job duration.
- The MFA provides the capability to process `JobRequest` including adaptive PRF behavior.
- The service developer has defined a waveform with adaptive PRF behavior that conforms to the instantaneous bandwidth limitations of the MFA.

Actors

The actors in the Adaptive PRF use case are:

- The service
- The MEL API Implementation

Flow of Events

Figure 5.5.1 shows a service using the RF MEL to command a job using a waveform that includes adaptive PRF behaviors:

1. The service initially requests access to the VA by submitting a `JobRequest` with minimum start times and maximum finish times to define a scheduling window for the job.
2. For each `JobRequest` issued by the service, an adaptive update for the PRF may be provided at every `JobInterval` with varied `ModulationParams` per `JobInterval`. These updates may occur while the current `JobInterval` is executing.
3. The `JobIntervals` are submitted as usual and the finalize method completes the specification of the job as usual resulting in adaptively modified PRF at each `JobInterval`.

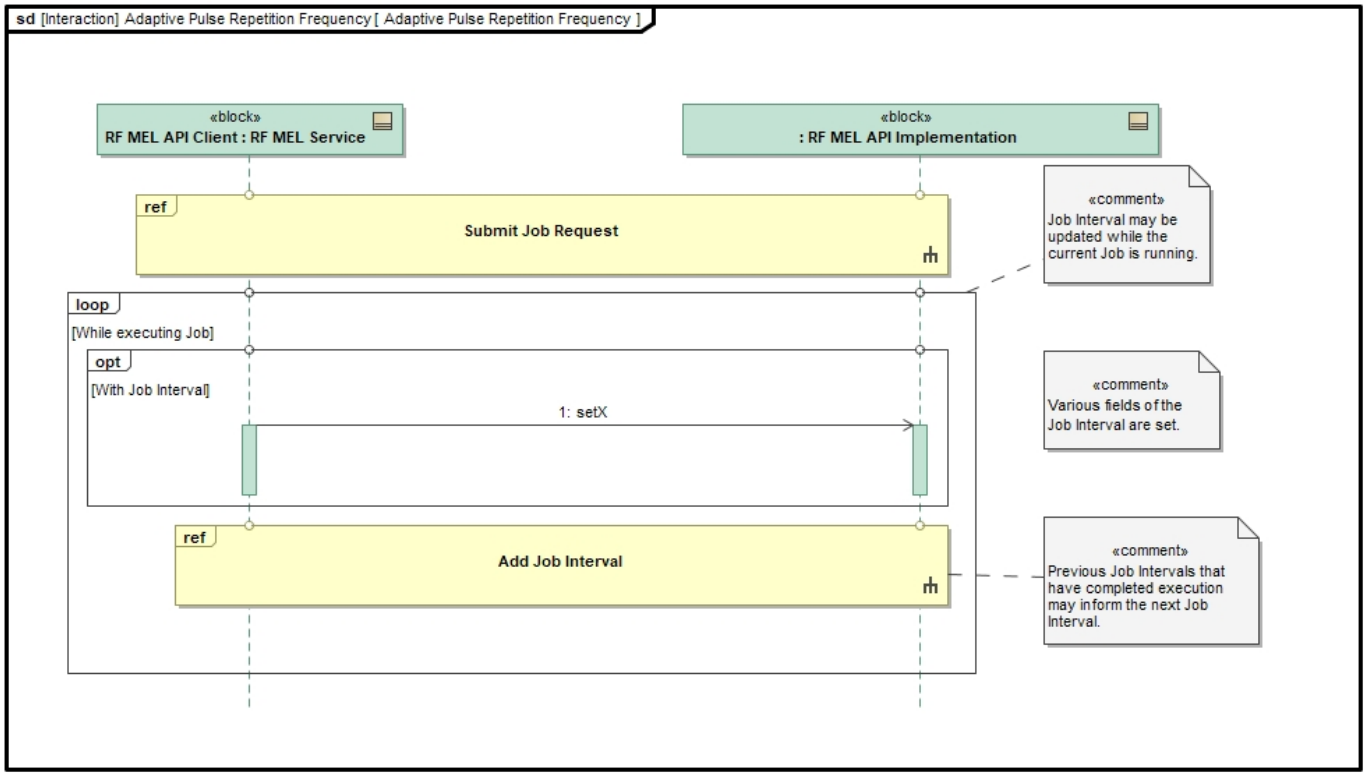


Figure 5.5.1: A Service Uses the RF MEL to Command a VA to Perform a Job Using a Waveform that Includes Adaptive PRF.

A.0 Acronyms

ABB	Avionics Backplane Bus
ACK	Acknowledgement
AGC	Automatic Gain Control
AMS	Agile Mission Suite
API	Application Programming Interface
ASB	Abstract Service Bus
BIT	Built-In Test
C2	Command and Control
CIF	Context Indicator Field
CPU	Central Processing Unit
DEA	Direct Endpoint Access
DP	Digital Payload
DPP	Digital Payload Processor
ECEF	Earth-Centered, Earth-Fixed
EMCON	Emissions Control
FAW	Frame Alignment Word
FPGA	Field-Programmable Gate Array
GPP	General Purpose Processor
GPU	Graphics Processing Unit
GRA	Government Reference Architecture
HCA	Host Channel Adapter
HTML	Hypertext Markup Language
I/Q	In-phase/Quadrature
IAW	In Accordance With
ID	Identification
IDD	Interface Description Document
IMPL	Implementation

IR	Infrared
LFM	Linear Frequency Modulation
LLA	Latitude, Longitude, and Altitude
LSB	Least Significant Bit
MEL	MFA Encapsulation Layer
MFA	Multi-Function Aperture
MFP	Multi-Function Processor
MPS	Mission Processing Subsystem
MPU	Minimum Procurable Unit
MSI	Mission System Integrator
NACK	Negative-Acknowledgement
OEM	Original Equipment Manufacturer
OMS	Open Mission Systems
OOK	On-Off Keying
PDW	Pulse Descriptor Word
PNT	Position Navigation and Time
PRF	Pulse Repetition Frequency
RADAR	Radio Detection and Ranging
RAM	Random Access Memory
RDMA	Remote Direct Memory Access
RF	Radio Frequency
RoCE	RDMA over Converged Ethernet
Rx	Receive
SK	Starter Kit
STL	Standard Template Library
Tx	Transmit
UCI	Universal Command and Control Interface
UUID	Universally Unique Identifier
VA	Virtual Aperture

VADB	Virtual Aperture Database
VAI	Virtual Aperture Instance
VAS	Virtual Aperture Scheduler
VITA	VME bus International Trade Association
VME	Versa Module Eurocard
VRL	VITA Radio Link Layer
VRT	VITA Radio Transport

B.0 Revision History

The Revision History table, **Table B.0.1**, captures the major changes made for a given AMS GRA volume as the SK continues to mature.

Revision	Description	Date	Notes
-	This revision adds a revision table to the volume.	May 22, 2024	SK11.0
A	This revision adds 1. Prose updates and clarifications. 2. Additional failures that have been identified and added to JobIntervalCompletionStatus. 3. New Prose added to Section 2 documenting the RF MEL split into 6 separate domain libraries.	July 24, 2025	SK13.0
B	This revision did not introduce any significant changes.	April 24, 2026	AMS GRA 14.0

Table B.0.1: Revision History Captures the Major Changes Made to the RF MEL IDD Between Documentation Releases.